

学校代码：10730

分类号：391

密级：公开

# 兰州大学

## 硕士学位论文

( 专 业 学 位 )

论文题目（中文） 基于数据与指令解耦的  
张量计算调度系统设计与实现

论文题目（外文） Design and Implementation of a Data-Instruction  
Decoupled Tensor Computation Scheduling System

作 者 姓 名 陈杰

学 科 专 业 通信工程

研 究 方 向 架构设计

教 育 类 型 学历教育

指 导 教 师 王兆滨 教授

合 作 导 师 何安平 副教授

论文工作时间段 2025 年 9 月 至 2026 年 3 月

答 辩 日 期 2026 年 5 月

校址：甘肃省兰州市城关区天水南路 222 号

## 原创性声明

本人郑重声明：本人所呈交的学位论文，是在导师的指导下独立进行研究所取得的成果。学位论文中凡引用他人已经发表或未发表的成果、数据、观点等，均已明确注明出处。除文中已经注明引用的内容外，不包含任何其他个人或集体已经发表或撰写过的科研成果。对本文的研究成果做出重要贡献的个人和集体，均已在文中以明确方式标明。

本声明的法律责任由本人承担。

论文作者签名：\_\_\_\_\_

日期：\_\_\_\_\_

## 关于学位论文使用授权的声明

本人在导师指导下所完成的论文及相关的职务作品，知识产权归属兰州大学。本人完全了解兰州大学有关保存、使用学位论文的规定，同意学校保存或向国家有关部门或机构送交论文的纸质版和电子版，允许论文被查阅和借阅；本人授权兰州大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用任何复制手段保存和汇编本学位论文。本人离校后发表、使用学位论文或与该论文直接相关的学术论文或成果时，第一署名单位仍然为兰州大学。

本学位论文研究内容：

☒ 可以公开

☐ 不宜公开，已在学位办公室办理保密申请，解密后适用本授权书。

（请在以上选项内选择其中一项打“√”）

论文作者签名：\_\_\_\_\_

导师签名：\_\_\_\_\_

日期：\_\_\_\_\_

日期：\_\_\_\_\_

# 基于数据与指令解耦的张量计算调度系统设计 与实现

## 中文摘要

随着深度学习模型规模的不断扩大，多节点系统中的张量计算任务面临计算密集、通信开销高以及调度灵活性不足等问题。传统系统通常采用数据传输与任务指令紧耦合的执行方式，在多节点环境下易引发重复数据搬运、通信拥塞以及资源利用率下降，进而限制系统性能与扩展能力，尤其在通信开销占主导的场景下这一问题更加突出。

针对上述问题，本文基于数据与指令解耦思想，面向小规模多 FPGA 节点环境，设计并实现了一种张量计算任务调度原型系统。系统采用“主机-调度器-加速器”三层架构，在体系结构层面分离数据通路 with 指令通路，实现数据传输与任务执行的异步协同，从而在执行过程中有效隐藏通信开销，并提升系统整体执行并行度与资源利用效率。

在任务管理与调度方面，主机端基于有向无环图建模任务依赖关系，并通过就绪驱动机制生成调度任务流；调度层构建统一的元数据表，对张量块的标识、位置及一致性状态进行管理，并结合数据本地性、节点负载与内存约束等因素，设计以数据迁移代价为核心的多因素调度策略，在保证负载均衡的同时降低跨节点数据传输开销，从而实现计算效率与通信开销之间的有效权衡。

在系统实现方面，完成了任务分块与指令流生成、数据-指令双通路控制逻辑、数据搬运与同步机制以及多节点协同执行流程的软硬件协同设计，并在自研 FPGA 计算阵列上构建原型系统进行验证。实验结果表明，该系统在保证稳定运行的前提下，显著降低跨节点数据搬移开销，并通过通信与计算的重叠执行有效隐藏通信延迟，在典型工作负载下表现出良好的性能提升效果与系统稳定性。

综上，本文从系统架构、数据管理与调度机制等多个层面验证了数据-指令解耦在多节点张量计算中的有效性，为资源受限环境下提升通信效率与系统并行度提供了一种具有工程可行性的解决方案，并为后续多节点异构计算系统的设计与优化提供了参考依据。

**关键词：**张量计算；多节点调度；数据-指令解耦；FPGA 系统；数据本地性

# DESIGN AND IMPLEMENTATION OF A DATA-INSTRUCTION DECOUPLED TENSOR COMPUTATION SCHEDULING SYSTEM

## Abstract

With the rapid growth of large-scale deep learning models, tensor computation in multi-node systems becomes increasingly computation-intensive and communication-bound. Traditional tightly coupled execution between data movement and task instructions often leads to redundant data transfers, communication congestion, and limited scalability.

To address these challenges, this thesis presents a tensor computation scheduling prototype based on data-instruction decoupling for resource-constrained multi-FPGA systems. The proposed system adopts a three-layer architecture consisting of host, scheduler, and accelerator nodes, where the data path and instruction path are decoupled to enable asynchronous execution and effectively hide communication overhead.

The host models task dependencies using a directed acyclic graph (DAG) and generates ready tasks through a readiness-driven mechanism. The scheduler maintains a unified metadata table to manage tensor blocks and employs a multi-factor scheduling strategy considering data locality, node load, and memory constraints, with data movement cost as the primary metric, achieving a balance between communication efficiency and load balancing.

The system is implemented through hardware-software co-design, including task partitioning, dual-path control logic, and data movement mechanisms, and is validated on a custom FPGA platform. Experimental results show that the proposed approach reduces cross-node data movement and effectively overlaps communication with computation, improving execution efficiency while maintaining stable system operation.

Overall, this work demonstrates the effectiveness of data-instruction decoupling in multi-node tensor computation and provides a practical solution for improving communication efficiency and system parallelism in resource-constrained environments.

**Keywords:** tensor computation; multi-node scheduling; data-instruction decoupling; FPGA-based scheduling system; data locality

# 目 录

|                             |    |
|-----------------------------|----|
| 中文摘要 .....                  | I  |
| Abstract.....               | II |
| 第一章 绪论 .....                | 1  |
| 1.1 研究背景与意义 .....           | 1  |
| 1.2 国内外研究现状 .....           | 2  |
| 1.2.1 张量分块与任务划分技术 .....     | 3  |
| 1.2.2 数据/指令解耦机制 .....       | 4  |
| 1.2.3 多节点调度策略与数据本地性.....    | 5  |
| 1.2.4 现有系统对比分析 .....        | 6  |
| 1.3 研究内容与技术路线 .....         | 8  |
| 1.4 本文主要贡献 .....            | 9  |
| 1.5 论文结构安排 .....            | 10 |
| 第二章 张量计算建模与解耦执行理论基础 .....   | 11 |
| 2.1 张量计算与任务依赖模型 .....       | 11 |
| 2.1.1 张量运算的并行结构.....        | 11 |
| 2.1.2 张量分块与任务抽象.....        | 12 |
| 2.1.3 面向张量任务的 DAG 调度基础..... | 13 |
| 2.2 多节点系统与调度问题 .....        | 15 |
| 2.2.1 多节点执行模型 .....         | 15 |
| 2.2.2 数据一致性管理机制.....        | 16 |
| 2.2.3 数据本地性与任务放置 .....      | 17 |
| 2.2.4 调度策略概述 .....          | 17 |

|                               |    |
|-------------------------------|----|
| 2.3 指令-数据双通路异步执行模型 .....      | 18 |
| 2.3.1 耦合式执行模型及其局限性 .....      | 18 |
| 2.3.2 指令-数据分离与解耦执行模型 .....    | 19 |
| 2.3.3 异步执行与通信-计算重叠 .....      | 20 |
| 第三章 张量计算调度系统总体架构 .....        | 21 |
| 3.1 系统整体架构设计 .....            | 21 |
| 3.1.1 面向解耦执行的三层系统架构 .....     | 21 |
| 3.1.2 数据通路指令通路解耦设计 .....      | 25 |
| 3.1.3 整体执行流程分析 .....          | 27 |
| 3.2 指令体系设计 .....              | 28 |
| 3.2.1 指令类型与格式 .....           | 29 |
| 3.2.2 指令通路握手机制 .....          | 31 |
| 3.2.3 异步指令队列与执行模型 .....       | 33 |
| 第四章 张量数据管理机制与实现 .....         | 35 |
| 4.1 面向张量块的数据标识与元数据管理 .....    | 35 |
| 4.1.1 张量块 ID 分配机制 .....       | 35 |
| 4.1.2 元数据表结构设计 .....          | 37 |
| 4.1.3 数据驻留与副本管理策略 .....       | 38 |
| 4.2 稀疏压缩与数据传输机制 .....         | 38 |
| 4.2.1 块稀疏张量的统一表示 .....        | 39 |
| 4.2.2 数据的序列化与传输 .....         | 40 |
| 4.2.3 基于环形缓冲区的数据接收与缓存机制 ..... | 41 |
| 4.2.4 数据迁移流程与 DMA 实现 .....    | 42 |
| 4.3 数据一致性与正确性保障机制设计 .....     | 43 |
| 4.3.1 一致性模型定义 .....           | 43 |
| 4.3.2 元数据驱动的副本管理与状态更新机制 ..... | 44 |

|       |                          |    |
|-------|--------------------------|----|
| 4.3.3 | 调度层缓存一致性与无死锁保证 .....     | 45 |
| 4.4   | 数据管理实验与分析 .....          | 46 |
| 4.4.1 | 数据本地性与跨节点传输开销分析 .....    | 46 |
| 4.4.2 | 稀疏率对数据传输带宽利用的影响 .....    | 51 |
| 4.4.3 | SWMR 一致性与副本管理正确性验证 ..... | 56 |
| 第五章   | 多因素任务调度策略与实现 .....       | 60 |
| 5.1   | 面向多计算单元的调度问题建模 .....     | 60 |
| 5.1.1 | 基于 DAG 的就绪任务模型 .....     | 60 |
| 5.1.2 | 调度目标与约束 .....            | 61 |
| 5.2   | 多因素调度决策模型设计 .....        | 62 |
| 5.2.1 | 数据本地性评分模型 .....          | 63 |
| 5.2.2 | 节点负载与内存压力建模 .....        | 64 |
| 5.2.3 | 综合调度决策与迁移约束机制 .....      | 65 |
| 5.3   | FPGA 调度器实现 .....         | 67 |
| 5.3.1 | 调度流程与状态更新机制 .....        | 67 |
| 5.3.2 | FPGA 调度器内部结构设计 .....     | 68 |
| 5.3.3 | 调度状态的预测更新与反馈校正机制 .....   | 69 |
| 5.4   | 调度策略实验 .....             | 70 |
| 5.4.1 | 实验环境与参数指标设计 .....        | 70 |
| 5.4.2 | 整体性能对比分析 .....           | 72 |
| 5.4.3 | 动态收敛性分析 .....            | 78 |
| 5.4.4 | 状态误差下的稳定性分析 .....        | 79 |
| 第六章   | 系统实现与实验分析 .....          | 84 |
| 6.1   | 系统原型实现 .....             | 84 |
| 6.2   | 实验环境与评估方法 .....          | 87 |
| 6.2.1 | 实验目的 .....               | 87 |

|           |                |     |
|-----------|----------------|-----|
| 6.2.2     | 实验平台 .....     | 87  |
| 6.2.3     | 工作负载 .....     | 88  |
| 6.2.4     | 评估指标 .....     | 89  |
| 6.3       | 实验结果与分析 .....  | 90  |
| 6.3.1     | 运行稳定性分析 .....  | 90  |
| 6.3.2     | 解耦机制性能分析 ..... | 92  |
| 6.3.3     | 实验总结 .....     | 94  |
| 第七章       | 总结与展望 .....    | 95  |
| 7.1       | 研究成果总结 .....   | 95  |
| 7.2       | 系统局限性与不足 ..... | 95  |
| 7.3       | 未来工作展望 .....   | 96  |
| 参考文献      | .....          | 97  |
| 在学期间的研究成果 | .....          | 102 |
| 致 谢       | .....          | 103 |

# 第一章 绪论

## 1.1 研究背景与意义

近年来,以 BERT<sup>[1]</sup>、GPT 系列<sup>[2-3]</sup>、PaLM<sup>[4]</sup> 等模型为代表的国外大语言模型 (Large Language Models, LLMs), 以及国内的盘古<sup>[5]</sup>、文心一言<sup>[6]</sup>、GLM<sup>[7]</sup> 与 DeepSeek<sup>[8]</sup> 等模型在自然语言处理、代码生成与多模态理解等领域取得了突破性进展。随着深度学习的发展,神经网络的复杂度及其对硬件计算资源的需求量与日俱增。现如今,参数量在几十亿甚至万亿级别的大模型层出不穷,在算法功能愈发完善的同时,也向硬件算力和存储空间提出了新的挑战<sup>[9]</sup>。其背后的计算结构高度依赖大规模张量 (Tensor) 运算与高效数据流管理。国内方面,围绕大规模模型训练构建的智能计算系统逐渐形成体系化架构,从算力组织、互连拓扑到任务调度机制均进行了系统优化<sup>[10]</sup>。然而,相关研究多聚焦于大规模 GPU 集群或专用加速器平台,对于资源受限的小规模多 FPGA 节点系统缺乏针对性探索。

机器学习模型和算法本质上就是对高维张量的各种算术运算。基础的张量运算构成了网络模型中的算子,如矩阵乘 (gemm)、卷积 (convolution)、池化 (pooling) 等,而多种算子的连接组合又进一步构成了模型的计算图。其计算过程适合通过分块、流水化与并行化方式提升性能<sup>[11]</sup>。随着模型规模不断扩大,单设备难以独立完成全部计算任务,系统需将高维张量分为多个子任务,由多节点协同执行。

因此,多节点任务组织、调度策略设计及跨节点数据传输优化成为影响系统性能与扩展性的关键问题。进一步来看,多节点系统中的通信开销主要来源于通信次数、数据规模以及通信与计算之间的时序关系。节点间通信优化主要有三种方法:一种是减少通信次数,一般需要从算法层面或通过合并通信操作进行设计解决;二是减少通信量,一般采用压缩的方法实现,如深度学习中的梯度参数压缩方法;三是实现通信与计算重叠,一般需要从应用通信特性角度考虑,实现梯度参数通信与计算的重叠优化。

此外,实际系统中常采用张量压缩或稀疏表示以降低通信与存储开销,但该过程会引入额外的数据编码与格式转换,使数据访问与计算流程更加复杂。在多节点环境下,这种复杂性进一步放大,对任务调度在数据可用性、传输时机及依赖关系协调方面提出了更高要求。

在主流深度学习系统中,TensorFlow<sup>[12]</sup>、PyTorch<sup>[13]</sup>、Megatron-LM<sup>[14]</sup>、DeepSpeed<sup>[15]</sup> 以及 Ray<sup>[16]</sup> 等通过算子融合、流水线并行及参数分片等技术提升训练

效率。然而，从系统架构角度看，这些系统通常将任务指令与数据传输绑定，即采用隐式“指令携带数据”的执行方式。该紧耦合模式在单节点中可简化控制，但在多节点环境下会导致重复数据搬运、通信拥塞及数据复用率下降；在引入张量压缩后，还会进一步增加格式转换带来的额外开销，限制跨节点调度灵活性。本文在此基础上，于小规模多节点环境中验证数据与指令解耦机制的可行性，重点关注其工程实现与运行效果。

另一方面，近年来专用加速器架构逐渐体现出“数据-指令解耦”的设计思想，该思想可追溯至 decoupled access/execute 体系结构<sup>[17]</sup>。例如，NVIDIA Hopper 架构中的 Tensor Memory Accelerator (TMA) 支持异步张量预取以减少计算等待<sup>[18]</sup>；Google TPU v4 通过部分解耦的数据流执行模式，实现数据加载与计算执行的分离，从而提升资源利用率<sup>[19]</sup>。类似思想在 FPGA 神经网络加速研究中亦有体现，论文<sup>[20]</sup> 对目前 FPGA 用于神经网络的发展现状进行总结，并提出所面临的问题和挑战。论文<sup>[21]</sup> 总结了 FPGA 的设计理论及其用于神经网络加速的技术原理和实现方法。然而，这些机制通常依赖特定硬件架构，缺乏面向通用多节点系统的可迁移实现路径。

与此同时，在异构与多节点计算场景中，复杂的计算流程常被分割为更细粒度的多个任务，并表示为有向无环图 (Directed Acyclic Graph, DAG) 建模的工作流，以利用集群并行计算加速任务执行，并被 Legion<sup>[22]</sup>、StarPU<sup>[23]</sup> 与 PaRSEC<sup>[24]</sup> 等系统用于调度与数据局部性优化。DAG 能够有效表达张量计算间的依赖关系，是实现并行调度与任务重叠执行的重要基础。

在上述背景下，如何在系统层面实现任务调度与张量数据管理的解耦，以减少跨节点重复数据传输并提升调度灵活性，成为值得深入研究的问题。尤其在资源受限的小规模多 FPGA 节点系统中，已有研究虽探索了 FPGA 集群在深度学习中的应用<sup>[25]</sup>，但仍面临数据搬运成本高于计算开销的问题。因此，围绕张量数据复用、数据位置管理及调度器侧轻量化控制机制的研究具有重要意义。

基于此，本文以张量任务的 DAG 调度与数据-指令解耦机制为核心，在自研 FPGA 计算阵列上实现验证性原型系统，用于探索该机制在多节点环境中的工程实现路径与实际效果。

## 1.2 国内外研究现状

随着深度学习模型发展，张量计算已成为高性能计算系统中的核心负载。近年来，研究主要集中在任务划分、数据与指令解耦、多节点调度及稀疏压缩传输等方向，为本文的工作提供了重要理论与技术基础。

### 1.2.1 张量分块与任务划分技术

张量分块 (Tensor Tiling) 是实现大规模并行计算与高效数据管理的基础技术。通过将高维张量划分为若干小块 (Tile)，系统可在保持计算正确性的前提下同时提升数据局部性与并行粒度，从而显著降低访存冲突并减少跨节点通信量<sup>[26]</sup>。在深度学习计算框架中，张量分块通常与算子融合、流水线并行及任务调度联合使用，以实现端到端的性能优化。

在软件系统方面，主流框架如 TensorFlow XLA、TVM 与 Mind Spore AKG 等均采用静态编译期分块策略。XLA(Accelerated Linear Algebra) 是 TensorFlow 的图优化编译器，并在 TensorFlow2.0 版本中作为正式特性发布。该架构的一个显著优点是，在定义计算图时，开发者不必提前指定张量的形状和数值，而是可以在图执行时通过分析操作之间的依赖关系推导张量的形状<sup>[27]</sup>；TVM 将算子划分为 4 类，即映射算子、约简算子、复杂算子和不可融合算子。并根据支配树的结构，为每一类算子设计相应的融合规则<sup>[28]</sup>；Hidet 则提出任务映射驱动的张量程序执行模型，以进一步提升张量程序优化能力<sup>[29]</sup>；AKG 编译器则结合硬件拓扑结构生成分块与调度方案，实现算子级别的自适应划分<sup>[30]</sup>。这些方法能够在单节点环境下有效提升内存带宽利用率，但分块策略一旦确定便无法根据运行时负载或通信延迟动态调整，灵活性不足。

在硬件加速研究中，VTA(Versatile Tensor Accelerator) 作为 TVM 框架的扩展，是一个开放、通用、可定制的深度加速学习加速器，具有完整且适用于 TVM 的编译器堆栈。与 TVM 共同构成了端到端的软硬件协同加速堆栈解决方案<sup>[31]</sup>。该机制主要面向单节点加速，缺乏跨节点划分能力。赵桦等改进了一种针对 VTA 张量处理器的细粒度结构化稀疏方法。该方法有效解决了传统分块剪枝方法在粗粒度稀疏计算时精度下降的问题，并将任务领域进行了拓展<sup>[9]</sup>。此外，已有研究如 DNNWeaver<sup>[32]</sup> 与 FINN<sup>[33]</sup> 通过软硬件协同设计实现神经网络在 FPGA 上的高效部署，但其优化重点主要集中在单节点加速。类似地，裴建华等提出一种基于 FPGA 芯片使用 RTL 逻辑实现 YOLO 后处理的硬件加速方案，计算精度损失约 0.2%，运算效率相对 CPU 提升了 7 倍，验证了数据流优化与算子级划分在单节点环境下的有效性<sup>[34]</sup>，但其研究重点仍集中于单设备性能提升。其他研究如 Tensor Algebra Compiler、Tensor IR 等也在高层张量 DSL 与编译器框架中探索自动化分块与调度策略<sup>[35-36]</sup>。此外，COSA 框架通过约束优化方法系统性探索空间加速器的数据流与调度空间，为张量计算映射提供了结构化建模方法<sup>[37]</sup>，但其优化目标仍主要集中在单芯片加速器架构。

总体来看，现有张量分块与任务划分研究的核心问题在于静态性与局部性：分块策略多在编译阶段确定，无法在运行时依据数据分布、节点负载与通信状态

进行动态调整；任务划分粒度与算子特性紧耦合，缺乏跨算子、跨设备的统一任务抽象。为此，本文在原型系统中采用基于 DAG 的任务描述与触发机制，用于验证跨节点任务分配与块级数据复用的可行性，为后续调度策略奠定基础。

### 1.2.2 数据/指令解耦机制

数据与指令解耦 (Data-Instruction Decoupling) 是近年来在异构系统设计中广受关注的体系结构思想，其目标是在控制层和数据层之间建立独立的通信通路，使数据搬运与计算指令可异步并行，从而减少等待时间并提升资源利用率<sup>[38]</sup>。该思想可追溯至 decoupled access/execute 架构及数据流计算模型<sup>[17, 39]</sup>。随着 AI 模型的算子数量与数据依赖复杂度持续增长，这一思想被重新引入现代深度学习加速器设计。

在硬件层面，为了更完美地适配人工智能时代的张量运算需求，业界开发了领域特定的张量加速器，如 GPU Tensor Core、NPU 以及 TPU 等用于加速张量运算。Google TPU v4 采用基于数据流执行模型的控制逻辑，将数据加载与计算触发解耦，通过独立的指令调度单元实现张量的异步搬运与计算核并行<sup>[19]</sup>。NVIDIA Hopper 架构中的 Tensor Memory Accelerator (TMA) 进一步发展了这一思想，支持在片上网络中独立传输张量块，实现计算单元与内存控制器之间的全流水异步协作<sup>[40]</sup>。Tensor Core 是 Nvidia GPU 上专门针对矩阵乘而设计的专用计算单元，可以通过对应的 WMMA 指令进行矩阵乘法的加速运算。然而，这类机制深度绑定于特定硬件架构与指令集，缺乏可通用的软件接口与跨平台迁移能力。

在软件与编译层，主流框架通过中间表示 (IR) 及编译器优化，典型的机器学习编译器包括 TVM、Tensor Comprehensions、nGraph、XLA 等，在一定程度上体现了数据与指令的静态解耦。编译器通常将神经网络模型按照计算图和算子两个不同的粒度进行划分，并转换成对应层级的 IR，从而进行编译优化。计算图将被转换为图层的低级 IR，而图内部的算子则转换为算子层的低级 IR，之后编译器便可进行各个层相关的优化和下降 (lowering)。例如，XLA 编译器在 HLO 优化阶段执行数据流分析与缓冲区分配 (buffer assignment)，并通过调度策略重叠计算与通信以减少内存高峰使用与延迟<sup>[41]</sup>；TVM 的 Ansor 生成编译期调度表，对算子执行顺序、内存布局及线程映射进行优化，以提升单节点的执行并行度<sup>[42]</sup>；Hidet 提供的数据流 DSL 允许开发者在编译期显式描述算子之间的数据依赖与流水化执行关系<sup>[29]</sup>。类似机制有助于在软件层减少数据访问冲突、提高计算资源利用率，但本质仍属于编译期静态优化，缺乏运行时动态调度能力。

近年来，国内研究也开始关注数据/指令解耦机制。Mind Spore 的 AKG 在算子生成阶段通过多面体变换 (polyhedral transformations) 实现数据与计算的静态解

耦<sup>[30]</sup>；清华大学提出的 KAPLA 框架利用时空调度 (Temporal-Spatial Scheduling) 统一建模任务依赖与数据流，初步探索了系统级的解耦思想<sup>[43]</sup>。但总体而言，这些研究仍停留在算子内核或编译层优化，对多节点协同与运行时元数据驱动的调度支持不足。

综上所述，数据与指令解耦机制的研究正在从硬件级设计向系统级实现扩展。现有方案虽已在单节点加速中验证有效，但在多节点运行时系统中仍缺乏统一框架与可行实现路径。针对这一问题，本文在原型系统的调度层采用数据-指令双通路的设计思想，实现异步传输与任务执行的功能验证。使任务指令与数据传输可独立管理、异步执行；结合元数据表索引机制，实现数据访问与任务调度解耦，减少通信冗余与节点等待时间。

### 1.2.3 多节点调度策略与数据本地性

在分布式计算系统中，调度策略与数据本地性 (Data Locality) 是决定系统吞吐量与资源利用率的关键因素。经典分布式计算框架如 MapReduce<sup>[44]</sup>、Spark<sup>[45]</sup> 以及面向 AI 工作负载的 Ray<sup>[16]</sup> 等系统均通过任务划分与数据本地性感知调度提升系统性能。随着深度学习模型参数规模的增长，单设备计算能力难以支撑完整任务执行，跨节点调度成为提升系统可扩展性的主要途径。然而，通信带宽与节点资源有限使得任务分配与数据迁移的平衡愈发困难。如何在保证负载均衡的同时减少跨节点数据搬运，是当前分布式调度系统的核心难题。

现有深度学习框架主要采用静态任务映射与分布式通信优化相结合的方式。Horovod、Megatron-LM 与 DeepSpeed 等系统通过通信优化与并行策略提升分布式训练效率<sup>[14, 46-47]</sup>，而 PipeDream 等方法通过流水线并行实现计算与通信重叠，但其任务映射多在初始化阶段确定，缺乏运行时动态调整能力<sup>[48]</sup>。

在异构加速平台中，尤其是 FPGA 集群中，跨节点协同计算仍面临数据分布不均、任务划分缺乏灵活性以及运行时反馈不足等关键问题。Johnson 等人设计的 FPGA 集群深度学习加速器展示了多节点协同的可行性，但其任务分配与资源调度依赖静态的编译期分析，难以适应运行时的负载变化<sup>[49]</sup>。Geng 等人提出的 FPDeep 框架利用深度流水化 (deep pipelining) 将神经网络映射到多块 FPGA，并通过阶段化数据传输优化吞吐量，但其节点划分与通信模式在设计时即固定，不支持运行时动态重构<sup>[50]</sup>。Zeng 与 Prasanna 针对 CPU-FPGA 异构环境下的图神经网络训练研究了通信与负载不均问题，指出数据本地性与跨节点传输成本是影响调度效率的主要瓶颈<sup>[51]</sup>。近年来也有研究尝试在 FPGA 集群环境下探索深度学习调度策略，例如基于可配置加速器的 FPGA 集群调度方法<sup>[25]</sup>。

此外，在异构多核平台环境下，多类型 DAG 调度问题也得到了深入研究。首

先, 异构多核环境下的任务调度问题属于 NP-难问题, 即任务调度问题只能得到近似最优解, 因此异构多核环境下的任务调度问题对处理器系统的性能和效率提高具有非常重要的作用. 多核处理器任务调度的目标就是按照某种策略将每个任务合理分配到各个处理器内核上并行有序地执行, 从而减少并行运行任务间的通信开销和等待时间, 使任务的执行时间最短<sup>[52]</sup>。

有研究利用深度强化学习方法对 DAG 任务进行自适应调度, 以在复杂资源环境中提升系统吞吐率与资源利用率<sup>[53]</sup>。运行时系统如 Legion<sup>[22]</sup>、StarPU<sup>[23]</sup> 与 PaRSEC<sup>[24]</sup> 等通过 DAG 描述任务依赖关系, 并结合数据本地性与资源状态实现动态调度。国内方面, 宋俊苏设计应用复杂任务树算法模型进行简单任务组合、父(子)节点任务执行, 任务执行成功后从消息队列取出执行后的二进制流数据, 提升分布式复杂任务调度处理质量<sup>[54]</sup>。然而, 该类方法主要面向 CPU/GPU 多核系统设计, 对 FPGA 多节点环境下的数据本地性与通信代价考虑不足。与此同时, 国内关于分布式复杂任务调度与数据管理的研究也指出, 分布式异构算力环境下的任务调度是移动协同计算技术中的关键步骤, 需要一种适应异构资源、具备高稳定性和容错能力的调度机制<sup>[55]</sup>。左俊杰等的研究指出: 现有调度方法通常使用多类型 DAG 任务模型进行模拟, 但调度方法往往忽略不同核上的通信开销, 或未考虑处理器与节点的对应关系, 导致调度时间开销较大, 处理器资源未充分利用, 任务效率低<sup>[56]</sup>。此外, 刘斌等提出一种新颖的计算任务调度策略, 充分考虑了任务的时效性, 同时为机动计算节点设计了容错机制可以更好地应用于复杂环境<sup>[57]</sup>。

总体而言, 现有的专用张量计算硬件与访存技术多依赖于针对特定单芯片内层级存储的编译期静态映射。尽管通过异步直接内存访问或专用控制指令在单节点内实现了较好的计算与通信重叠, 但在面对多 FPGA 计算节点互联环境时, 硬件架构通常缺乏运行时资源状态的动态自适应调整能力, 其软硬件强耦合的设计也限制了跨平台的工程迁移与扩展。为了更系统地评估前述各类软件编译框架、分布式调度系统以及专用硬件在多节点张量计算场景下的特性, 有必要对其调度机制、数据管理及解耦策略进行集中对比与深入探讨。

#### 1.2.4 现有系统对比分析

综合上述对国内外研究现状的梳理, 现有的主流张量计算框架以及专用硬件架构在调度机制和数据管理上各具特色。为了更清晰地阐明不同系统架构的设计侧重点及其适用场景, 表1-1从调度机制、数据管理方式、解耦方式等维度对上述代表性工作与本文原型系统进行了系统性对比。

表 1-1 主流张量计算与调度系统特性对比

| 系统/框架      | 调度机制          | 数据管理方式                      | 数据与指令解耦方式                | 适用场景                | 设计侧重点与局限性                      |
|------------|---------------|-----------------------------|--------------------------|---------------------|--------------------------------|
| TVM / XLA  | 编译期静态图调度      | 静态内存预分配与 Buffer 复用          | 软件层编译解耦, 底层执行为强绑定        | 单节点异构环境或固定拓扑集群      | 侧重编译期算子融合优化; 缺乏运行时的动态调整能力。     |
| Legion     | 运行时动态调度       | 基于逻辑区域 (Logical Regions) 管理 | 软件运行时层面的任务与数据解耦          | 大规模超算、多节点异构集群       | 侧重分布式一致性与扩展性; 但运行时软件调度开销较大。    |
| StarPU     | 运行时异构动态调度     | 分布式共享内存 (DSM) 与自动预取         | 运行时框架层解耦                 | 单节点多 GPU/CPU 异构平台   | 侧重多硬件资源动态并行利用; 控制流仍集中在主机端。     |
| NVIDIA TMA | 硬件级异步调度       | 硬件级异步直接内存访问 (Async DMA)     | 硬件指令驱动的异步传输与计算重叠         | 单个 GPU 内部 (SM 与显存间) | 侧重硬件专用数据搬运效率; 依赖特定硬件架构, 跨平台困难。 |
| Google TPU | 编译期静态调度       | 软件管理的片上暂存器                  | 软硬件深度协同, 数据流动受指令控制       | 专用同构 TPU 数据中心集群     | 侧重特定矩阵乘法的流水线能效比; 硬件结构灵活度较低。    |
| 本文系统       | 结合多因素的运行时动态调度 | 元数据表 + 环形缓冲区的硬件一致性管理        | 数据通路 with 指令通路物理/逻辑双通路分离 | 小规模多 FPGA 节点环境      | 侧重在多节点通信延迟可控的前提下, 实现软硬件协同解耦。   |

由表1-1可知，以 TVM 和 XLA 为代表的深度学习编译器主要依赖编译期的静态图优化，虽然能有效减少单节点内的访存开销，但在面对多节点网络环境时，难以灵活应对运行时的动态延迟。Legion 与 StarPU 等运行时框架虽然支持丰富的动态调度策略，但其复杂的软件管理层在处理细粒度张量计算时往往带来不可忽视的调度开销。在底层架构设计上，NVIDIA TMA 引入了硬件级异步传输机制，而 Google TPU 则依赖软硬件深度协同的片上存储精细管理。两者虽然都能高效地实现访存与计算的重叠，但此类高度定制化的设计往往与特定专用芯片架构深度绑定，难以直接迁移至通用的多 FPGA 互联平台。

综上所述，现有的张量计算调度系统多服务于云端大规模数据中心或特定的专用芯片平台。面对几十甚至数百个计算节点的大规模分布式部署，或在大模型调度主流的多 GPU 异质混合集群生态中，系统的可延展性、跨平台兼容性以及复杂容错往往依赖高带宽的数据中心级高速网络与庞大的软件运行时环境。

鉴于此，面向资源受限的小规模多 FPGA 节点这一特定场景，本文将研究边界明确限定在实验室特定原型环境下，重点探索如何在维持软硬件结构相对简单、开销可控的前提下，通过数据与指令的解耦设计实现高效的通信延迟隐藏与灵活任务调度。这为在嵌入式边缘阵列或特定硬件验证平台上，利用多 FPGA

进行流式张量计算调度提供了一种可行、轻量化的软硬件协同设计参考方案。为此，本文开展了基于数据与指令解耦的张量计算调度系统设计与实现工作。

### 1.3 研究内容与技术路线

本文针对多节点张量计算中数据传输与指令控制紧耦合、调度灵活性不足的问题，从系统工程角度出发，在自建的小规模多 FPGA 平台上设计并实现基于数据-指令解耦思想的任务调度原型系统。研究重点在于探索该机制在资源受限多节点环境下的实现方式，并通过实验评估其对数据复用率、调度灵活性与通信效率的影响。

本文主要研究内容包括以下五个方面：

#### (1) 原型系统架构设计

构建由 Host、FPGA 调度器和多个 FPGA 计算节点组成的三层结构系统。系统采用数据通路 with 指令通路分离的双通路架构，实现数据与控制的异步传输。

#### (2) 任务描述与运行时管理

在 Host 端利用 DAG 描述张量算子依赖关系，生成任务序列及数据引用信息。运行时系统根据任务就绪状态进行调度，并通过统一接口向 FPGA 调度层下发轻量化指令。

#### (3) 双通路调度与指令体系实现

在 FPGA 调度层设计数据通路 with 指令通路：数据通路负责张量块搬运与缓存管理，指令通路负责任务指令解析与分发。两条通路通过独立有限状态机 (Finite State Machine, FSM) 实现异步协作。

#### (4) 元数据驱动的数据管理与调度机制

设计元数据表记录张量块标识、驻留节点与一致性状态，并结合数据本地性、节点负载与内存占用构建多因素加权调度模型，实现任务动态分配与数据迁移控制，提升系统整体并行度与资源利用率。

#### (5) 系统实现与性能验证

完成 Host 端任务管理模块、调度通信协议以及 FPGA 调度器模块的实现，在多 FPGA 平台上对不同任务规模与调度策略进行实验验证，评估数据-指令解耦架构的性能表现。

本文整体技术路线如图1-1所示：从任务依赖建模出发，经软件层生成调度逻辑，在硬件层实现双通路调度与数据管理，在多 FPGA 平台上完成系统验证。

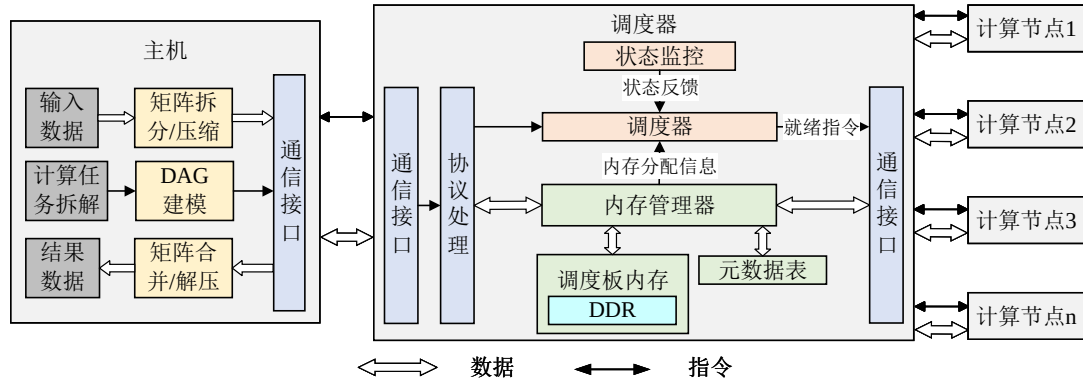


图 1-1 整体技术路线图

## 1.4 本文主要贡献

围绕小规模多 FPGA 节点环境下张量计算任务调度中存在的搬运开销大、调度灵活性不足等问题，本文基于数据与指令解耦思想，从系统架构设计、关键机制实现到实验验证进行了系统研究。论文的主要工作与贡献体现在以下几个方面：

(1) 设计并实现了一种面向多个 FPGA 张量加速器的张量计算调度原型系统

构建了由 Host、FPGA 调度器及多个张量加速器组成的三层协同架构，实现了张量任务的描述、下发、调度与执行流程，为在资源受限的多节点 FPGA 平台上验证运行时调度与数据管理机制提供了工程基础。

(2) 提出并实现了数据通路与指令通路分离的双通路调度结构

在 FPGA 调度层中将任务控制与张量数据传输在体系结构上显式分离，通过独立通路进行管理。结合异步队列、DMA 机制与握手机制，实现指令下发与数据搬运的并行执行，验证了数据-指令解耦在多节点环境下的可行性。

(3) 设计了基于元数据表的张量数据管理与一致性维护机制

构建统一的元数据映射表，对张量块标识、驻留位置及一致性状态进行集中管理，并结合单写者多读者 (SWMR) 一致性模型，实现跨节点数据访问与副本维护，为调度决策提供运行时数据状态信息。

(4) 实现了融合数据本地性与节点状态的多因素动态调度策略

在调度层综合考虑数据本地性、节点负载及内存占用等因素，设计多因素加权调度模型，并基于 DAG 就绪驱动机制完成任务动态分配，在保证执行正确性的前提下降低数据迁移开销，提高系统资源利用率。

(5) 完成了系统级功能验证与性能评估

在自研 FPGA 计算阵列上对调度架构与关键机制进行了实验验证，从数据传输开销、任务执行时延及系统稳定性等方面进行分析。实验结果表明，该数据-指令解耦调度架构能够减少重复数据搬运并提升数据复用率。

综上,本文从工程实现角度探索了数据与指令解耦思想在多节点张量计算调度中的应用,为更大规模或异构计算系统的调度与数据管理研究提供了参考。

## 1.5 论文结构安排

本文围绕基于数据与指令解耦的张量计算调度系统设计与实现展开研究,全文共分为七章,具体结构安排如下:

第一章为绪论。介绍论文的研究背景与意义,综述国内外在张量分块、数据与指令解耦、多节点调度及数据本地性等方面的研究现状,分析现有方法的不足,在此基础上明确本文的研究内容、技术路线及主要研究贡献。

第二章为张量计算建模与解耦执行理论基础。系统介绍支撑本文研究的关键理论与技术,包括张量计算与任务依赖建模、多节点执行模型与数据一致性管理、数据本地性与调度策略,指令-数据解耦与异步执行模型等,为后续系统设计提供理论依据。

第三章为张量计算调度系统总体架构设计。阐述系统的三层体系结构及模块划分,重点说明数据通路 with 指令通路的解耦设计思想、系统整体执行流程以及指令体系与软硬件交互机制。

第四章为张量数据管理机制与实现。围绕多节点环境下的张量块管理需求,介绍张量块标识与元数据表结构、数据驻留与副本管理策略、稀疏压缩与数据传输机制,以及一致性保障与无死锁设计,并通过实验对数据管理效果进行分析。

第五章为多因素任务调度策略与实现。基于 DAG 就绪任务模型,对多节点调度问题进行建模,提出融合数据本地性、节点负载和内存占用的多因素调度策略,并给出 FPGA 调度器的具体实现与硬件开销分析,最后通过实验验证调度策略的有效性。

第六章为系统实现与实验分析。介绍原型系统在 FPGA 平台上的具体实现,包括硬件结构、资源利用情况及系统部署方式;在此基础上,构建实验环境与评估方法,对系统在不同任务规模与通信压力场景下的运行表现进行系统级实验分析,重点评估数据-指令解耦机制在通信隐藏、任务完成时间及系统稳定性等方面的性能收益。

第七章为总结与展望。对全文研究工作进行系统总结,归纳本文在系统架构设计、数据管理机制及调度策略方面的主要成果与创新点;同时分析当前系统在规模扩展、真实计算负载支持及复杂异构环境适配等方面的不足,并对未来在大规模多节点协同计算、调度优化及系统工程化实现等方向的发展进行展望。

## 第二章 张量计算建模与解耦执行理论基础

### 2.1 张量计算与任务依赖模型

#### 2.1.1 张量运算的并行结构

张量运算，如矩阵乘法、卷积、逐元素操作和归约，通常表现为规则的多维数组访问模式，其计算结构多由规则嵌套循环组成，使得编译器能够对其进行分析与调度优化<sup>[36, 58]</sup>。张量本质上是一种通过多个维度索引的多维数组结构<sup>[59]</sup>。在智能计算系统架构中，这种规则计算模式为专用加速器设计提供了基础<sup>[60]</sup>，并有利于任务划分、并行调度和跨设备执行。总体而言，张量运算主要具有以下两个结构特征：

##### (1) 数据访问的规律性与可预测性

张量运算通常以规则嵌套循环 (regular loop nests) 形式实现，其循环边界、访问模式及数据依赖关系可在编译期确定。例如内积算法：传统上的矩阵-矩阵乘法是基于内积算法执行的。也即将左矩阵的行元素和右矩阵的列元素进行一系列点积运算，并得到最终结果：

$$C_{ij} = \sum_k A_{ik} \times B_{kj} \quad (2-1)$$

式(2-1)中  $A$ 、 $B$ 、 $C$  为输入与输出矩阵， $i$ 、 $j$ 、 $k$  为循环索引。计算过程按矩阵  $A$  的行和矩阵  $B$  的列访问数据并生成  $C$  的对应元素。这种规则访问模式使数据存取在编译期即可预测，从而有利于任务划分、数据局部性优化和并行执行。

##### (2) 空间与时间维度上的自然并行性

张量运算通常可以拆分为多个相互独立的子任务并行执行，其并行性主要体现在以下三个方面：

##### 1) 数据并行性 (data parallelism)

数据并行是指对张量不同子集进行独立计算，是并行计算中最常见的模式之一<sup>[61]</sup>。例如矩阵乘法中，矩阵  $C$  的不同行或子块可由不同计算单元独立完成，通过一系列的乘法和累加操作得到结果矩阵的每个元素，没有中间矩阵的产生。作为深度学习分布式训练中的核心技术，旨在将大规模训练数据集分割并分发至多个计算节点，实现并行计算以加速模型训练。

##### 2) 算子并行性 (operator parallelism)

算子并行指多个相互独立的算子可同时执行。例如 MC-CNN 的卷积核具有

多个通道,每个通道的卷积核在对应的输入数据上独立应用。通过将不同通道的卷积结果进行叠加,MC-CNN 能够融合来自多个通道的信息。不同通道的卷积或归一化算子彼此独立,可分配到不同计算单元并行执行。

### 3) 流水并行性 (pipeline parallelism)

流水并行是将多个计算阶段按数据流方式连接形成流水线结构,每个阶段负责处理数据流中的不同部分。以前向传播为例,输入数据依次经过卷积、激活、池化等层,例如 CNN 的核心运算如常规卷积、池化、全连接以及激活等各阶段可在不同时间并行处理不同数据,从而提升系统吞吐量。

## 2.1.2 张量分块与任务抽象

在实际系统中,完整张量往往难以在单个加速器上直接容纳或高效处理,因此将张量划分为较小子块 (tiles) 成为深度学习系统与张量编译框架中的常用优化技术,其目标是提升数据局部性与并行效率<sup>[36, 62]</sup>。张量分块不仅缓解存储限制,还提高数据复用率与任务并行度,是构建可调度张量任务的重要基础。

### (1) 计算硬件的存储约束

实际计算硬件通常受存储容量限制。例如在 FPGA 加速神经网络方面,Venieris 等人<sup>[63]</sup>采用同步数据流架构设计了一种将 CNN 模型计算任务映射到 FPGA 平台的流水处理方法,为了避免因数据缺失而导致流水线停顿,只能将需要加速的 CNN 模型权重和各层之间的特征图全部缓存到片上,但因为片上内存受限,此类方法只能适用于规模较小的 CNN 模型,难以容纳大规模张量;GPU 虽具有较大显存,但单次内核执行通常无法加载全部中间数据。在分布式环境中,完整张量复制还会带来较高的通信与访存开销。因此,需要将张量划分为适应硬件存储规模的子块并逐块计算,以减少片外访存并提升数据复用效率。

### (2) 分块后的数据局部性与复用

分块后的张量具有更好的数据局部性,每个数据块在计算过程中可被多次复用,从而减少主存访问。例如在李亿渊等的研究中通过将矩阵以行为基础分成矩阵带,再以分块的方式计算每个矩阵带内的非零元,从而解决稀疏矩阵向量乘法固有的数据局部性差、写冲突和负载不均衡的问题,且充分利用了粗粒度访问并降低了手动缓存判断次数<sup>[64]</sup>。在分布式系统中,以数据块为粒度进行迁移或同步也可避免完整张量级通信,从而降低节点间数据交换开销并提升计算效率。

### (3) 基于分块的任务抽象

在完成 tile 化后,系统通常将数据块及其计算操作抽象为可调度任务 (task)。任务包含输入与输出数据块,例如矩阵乘法任务使用矩阵  $A$  的行块和矩阵  $B$  的列块,并生成矩阵  $C$  的结果块。任务之间通过数据依赖关系连接,这些依赖由计

算图结构和数据块位置决定，并构成任务调度的有向无环图 (DAG)，为就绪驱动调度提供基础。

#### (4) 分块原则

根据算子访问模式及硬件结构，可采用不同分块策略优化性能。常见方式主要包括以下三类：

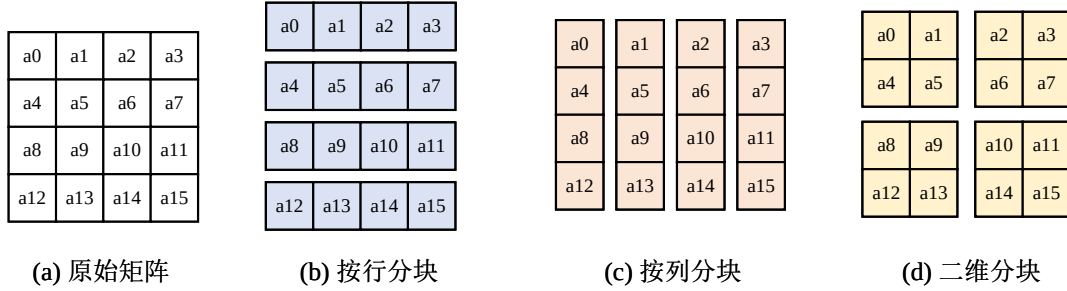


图 2-1 常见分块方式

按行分块（图2-1(b)）适用于行方向访问占主导的算子，如向量—矩阵运算或行归一化操作。按列分块（图2-1(c)）适用于列访问频繁的计算任务，如矩阵累加或列聚合运算。二维分块（图2-1(d)）是最通用策略，将矩阵划分为如  $128 \times 128$  或  $64 \times 64$  的子块，适合矩阵乘法等同时访问两个维度的算子，可显著提升数据局部性与并行度。

综上，张量分块不仅缓解存储压力、提升数据复用与局部性，也是构建任务抽象和可调度 DAG 的重要基础。

### 2.1.3 面向张量任务的 DAG 调度基础

在张量运算被分解为细粒度任务后，需要结构化方式描述任务之间的依赖关系与执行顺序。DAG 中节点表示子任务，边表示数据传输的形式提供了子任务及其依赖关系的数据结构表示，这为解决依赖任务的调度问题提供了模型<sup>[61, 65]</sup>。其无环性保证图具有拓扑可排序性，从而确保计算按顺序执行。

#### (1) DAG 的基本结构与数学建模

图2-2 给出了几类典型有向图结构，其中图2-2(b) 为典型 DAG，其节点表示任务或事件，边表示依赖关系。无环性保证沿任一有向路径无法返回起点，从而具有明确的因果关系。

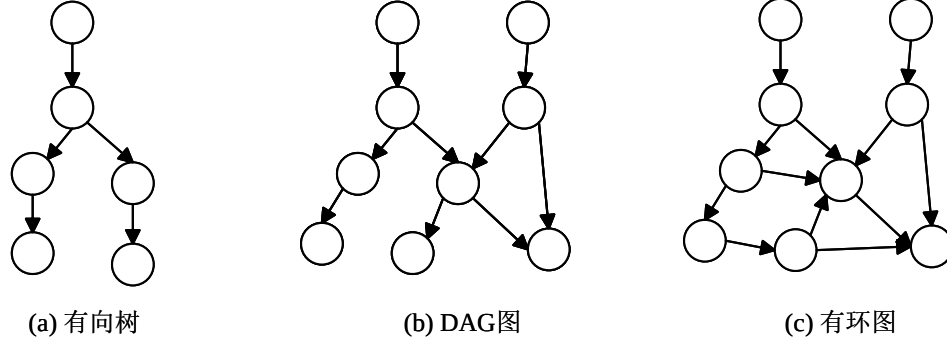


图 2-2 典型有向图结构

形式上，DAG 可表示为：

$$G = (V, E) \quad (2-2)$$

式(2-2)中， $V = \{v_1, v_2, \dots, v_n\}$  表示包含  $n$  个任务的集合， $E$  表示任务间的依赖关系以及通信成本，边的数量用  $e$  表示。若  $(u, v) \in E$ ，表示任务  $v$  依赖于任务  $u$ 。

对于节点  $v$ ，其前驱与后继集合定义为：

$$\text{Pred}(v) = u \in V | (u, v) \in E, \quad \text{Succ}(v) = w \in V | (v, w) \in E \quad (2-3)$$

其无环性保证 DAG 至少存在一种拓扑排序，从而确保所有任务能够在满足依赖关系的条件下依次执行。通过这种方式可以保证任何一个任务图 DAG 均只有一个入口节点和一个出口节点。

## (2) DAG 在张量计算表达与框架系统中的应用

张量运算通常具有结构化依赖模式，如算子链式执行、并行算子结构以及多输入多输出关系。DAG 能自然表达这些依赖关系：顺序依赖形成路径，无依赖算子可并行执行。因此，DAG 成为描述张量计算图结构的常用抽象。

在异构计算系统中，大量研究基于 DAG 提出任务调度算法以提升资源利用率，其中 HEFT 等经典算法被广泛用于异构处理器调度研究：Topcuoglu 等人提出的 HEFT 算法是一种使用广泛的列表调度算法。HEFT 算法使用任务的平均计算成本和平均通信成本计算任务的向上排名  $\text{rank}_u$ ，并将其作为优先级。在处理器选择阶段，HEFT 算法每次选择具有最高优先级的任务，并将其分配到具有最早完成时间的处理器上，并且在任务分配时考虑处理器上是否有足够的空闲时间完成任务的插入<sup>[66]</sup>。这一模型也成为现代深度学习系统的标准表示方式。例如 TensorFlow 的数据流计算图<sup>[12]</sup>、PyTorch 的 TorchScript/FX 图表示<sup>[13]</sup>，以及 XLA、TVM 等编译器优化流程均基于 DAG，用于表达算子依赖、识别并行性、执行拓扑排序及算子融合等优化。

综上，DAG 为张量计算提供了严格且可拓扑排序的依赖表达方式，为深度学习编译器与执行框架的调度与优化提供了重要基础。

## 2.2 多节点系统与调度问题

随着大规模张量计算在机器学习与科学仿真中的广泛应用，单一计算节点已难以满足计算与存储需求，多加速器协同执行逐渐成为主流体系结构，如多 GPU、GPU-CPU 混合平台及多 FPGA 系统等<sup>[12-13, 15, 46]</sup>。在此类系统中，任务与数据分布于多个节点，任务调度、数据放置与一致性管理成为影响系统性能的重要因素。本节介绍多节点执行模型、数据一致性机制、数据本地性问题及典型调度策略，为后续章节提出的多因素调度机制奠定理论基础。

### 2.2.1 多节点执行模型

多节点加速系统通常由多个结构同构的计算节点组成，每个节点包含独立计算资源、本地存储体系与高速互连接口<sup>[67]</sup>。图2-3展示了多节点系统的一般结构：数据以块形式分布在各节点，本地节点执行计算，节点之间通过高速互联通信。

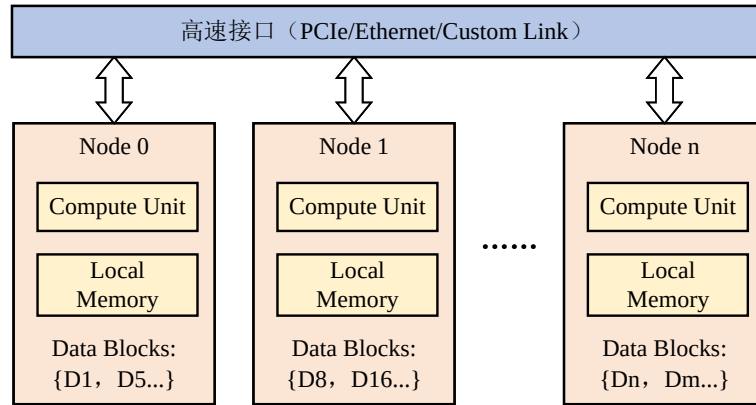


图 2-3 多节点系统结构图

该体系结构具有以下特征：

#### (1) 分布式存储与数据驻留

输入数据与中间结果通常以数据块形式分布式存储。分布式存储通过数据分散存储与并行处理机制，能够有效提升系统吞吐能力。由于任务执行位置与数据驻留位置密切相关，调度器在确定任务执行节点时需考虑数据位置。本地访问开销远低于远端访问，不必要的数据迁移会增加调度成本并降低系统效率，导致分布式存储系统在处理数据时陷入性能瓶颈：存储负载分配失衡导致部分节点资源过度消耗，而其余节点却处于空闲状态，直接拉低了系统的整体吞吐表现。

#### (2) 跨节点通信开销高

跨节点通信受网络带宽与链路延迟限制，其代价通常高于节点内部访问<sup>[67]</sup>。例如 GPU HBM 带宽可达 TB/s，而跨节点互联带宽通常低两个数量级。当任务依赖远端数据时，通信延迟往往成为性能瓶颈，因此调度器需要尽量减少跨节点数据传输。如在边缘计算场景中只需上传模型参数，而无需传输大量的原始数据，

减少了数据传输量。

### (3) 节点运行状态具有动态差异

虽然节点硬件通常同构，但由于负载、缓存命中率及数据分布差异，系统会表现出“有效异构性”。调度器需要动态调节任务分布策略，最大限度提升系统整体吞吐能力与稳定性，以避免节点负载不均。

### (4) 任务依赖关系约束执行顺序

任务之间的依赖关系通常以 DAG 形式表示。边  $(v_i, v_j)$  表示子任务  $v_i$  和子任务  $v_j$  之间的优先条件，指定  $v_i$  为  $v_j$  的前继子任务， $v_j$  为  $v_i$  的后继子任务，也即  $v_j$  必须在  $v_i$  完成执行后执行。只有当前序任务完成后，后续任务才能执行。在多节点环境中，依赖关系不仅影响任务触发时间，还限制系统可利用的并行度<sup>[68]</sup>。

## 2.2.2 数据一致性管理机制

由于数据可能在不同节点之间复制或迁移，多节点系统需要维护数据副本一致性<sup>[60, 69]</sup>。典型一致性模型包括：

### (1) 单副本模型 (Single Copy Model)

系统在任意时刻仅维护一个有效副本，所有读写操作需通过该副本节点完成。该模型实现简单，但存在单点瓶颈。

### (2) 多副本只读 (Multiple Read-Only Replicas)

允许多个节点持有只读副本，而写操作仅在唯一节点进行<sup>[69]</sup>。该策略可提高读取并行度，但写入后需要同步其他副本。当客户端发起数据更新操作时，系统将更新请求同时广播至所有副本节点。系统需等待集群内所有副本节点成功完成数据更新并返回确认信息，然后才向客户端反馈操作成功。

### (3) 多副本可读可写 (Multi-Writer Replication)

允许多个节点同时读写数据，需要依赖一致性协议保证版本顺序。该方案并行度高，但一致性维护成本较高。常见的一致性维护是基于时间戳的，当节点进行数据更新时，系统会为新生成的数据版本赋予一个递增的时间戳；在读取数据时，节点通过比较时间戳的大小来判断数据的新旧程度。若读取的数据版本时间戳小于当前最新的时间戳，说明该数据并非最新版本，需要重新获取最新数据。

### (4) 单写者、多读者模型 (SWMR)

允许多个节点持有只读副本，但任一时刻仅允许一个节点写入。当发生写操作时，其余副本失效，写入完成后按需复制新版本。该模型适用于“读多写少”的张量计算场景，因此成为本系统采用的一致性策略。其四个阶段如图2-4所示。

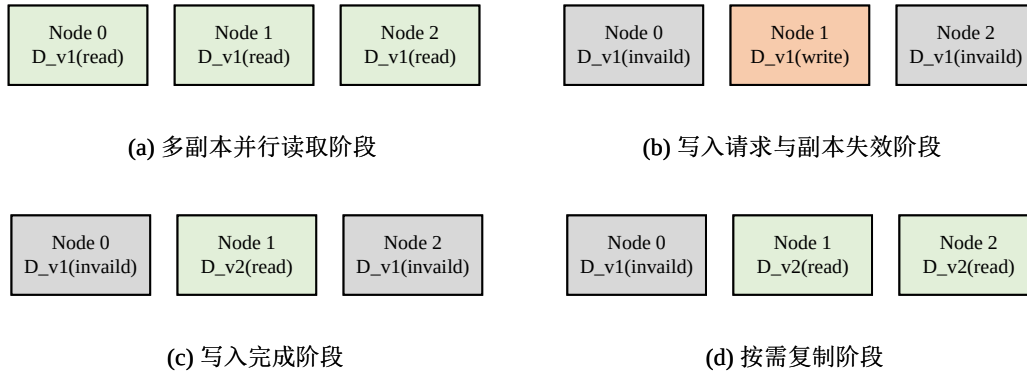


图 2-4 SWMR 一致性示意图

表2-1对不同一致性模型进行了对比。

表 2-1 数据一致性模型特性对比

|         | 单副本   | 多副本只读 | 多副本可读可写 | SWMR    |
|---------|-------|-------|---------|---------|
| 读并行度    | 低     | 高     | 高       | 高       |
| 写并行度    | 低     | 单写者   | 高       | 单写者     |
| 一致性维护成本 | 无     | 中     | 高       | 低       |
| 优点      | 实现简单  | 并行读   | 并行度高    | 避免写冲突   |
| 缺点      | 单点瓶颈  | 写代价高  | 实现复杂    | 写频繁需复制  |
| 适用场景    | 小规模系统 | 读多写少  | 强一致系统   | 多节点张量计算 |

### 2.2.3 数据本地性与任务放置

数据本地性是影响多节点执行效率的重要因素<sup>[62]</sup>。若任务能够在数据所在节点执行，则无需跨节点数据迁移，从而减少通信开销并提高执行效率。任务放置问题决定任务在哪个节点执行，其目标是减少数据搬移并提高资源利用率。任务执行的数据搬移成本可表示为：

$$cost = \sum_{d \in inputs} size(d) \times latency(d)_{\omega} \quad (2-4)$$

式(2-4)中  $size(d)$  为数据块大小， $latency(d)$  为访问该数据的网络延迟。由于任务放置同时受到数据位置、资源状态与网络条件影响，其优化通常采用启发式策略，例如优先选择数据所在节点执行任务。

### 2.2.4 调度策略概述

调度问题是一个广泛的优化问题，通常出现在任务需要被安排在有限资源上并且必须满足一系列约束条件时。具体来说，调度问题旨在通过合理安排任务的执行顺序和资源分配来优化某个目标。在多节点系统中，调度器负责将任务分配

到各计算节点。由于任务依赖关系、节点负载与通信成本具有动态性，调度问题通常被视为多目标优化问题<sup>[65-66]</sup>。常见调度策略包括：

(1) 轮询调度 (Round-Robin)

按顺序在节点之间循环分配任务，实现简单但忽略数据位置与节点状态。

(2) 负载感知调度 (Load-Aware Scheduling)

根据节点负载选择执行节点，可避免节点拥塞，但可能增加数据迁移开销。

(3) 多因素加权调度 (Multi-Factor Scheduling)

综合考虑数据本地性、节点负载与资源利用率，通过评分函数选择执行节点，例如：

$$score(node) = w_1 L_{locality} + w_2 L_{load} + w_3 L_{resource} \quad (2-5)$$

式(2-5)中  $w_1, w_2, w_3$  为各因素权重。该策略能在通信成本与资源利用率之间平衡。

表2-2总结了不同调度策略的特点。

表 2-2 不同调度策略特性对比

|       | 轮询调度      | 负载感知调度   | 数据本地性调度 | 多因素加权调度 |
|-------|-----------|----------|---------|---------|
| 数据本地性 | 否         | 否        | 是       | 是       |
| 负载状态  | 否         | 是        | 否       | 是       |
| 优点    | 实现简单      | 提高资源利用率  | 降低通信成本  | 综合性能较好  |
| 缺点    | 忽略负载与数据位置 | 数据迁移可能增加 | 易形成热点节点 | 权重配置复杂  |
| 适用场景  | 简单系统      | 负载变化明显系统 | 数据规模大系统 | 多目标调度系统 |

## 2.3 指令-数据双通路异步执行模型

多计算节点运行时系统中，任务执行一般包含指令下发、数据加载、运算执行、结果回传等阶段。传统耦合式模型里，数据传输与指令控制紧密绑定、串行执行，指令常需等待数据搬运完成，指令处理易受存储与通信延迟影响，跨节点场景下并行度受限。为提高吞吐、隐藏延迟，现代架构普遍采用指令与数据路径分离设计，让控制逻辑和数据搬运并行执行，从而实现更高效的任务执行模型。

本节首先分析传统耦合式执行模型的局限性，然后介绍指令-数据双通路结构及其解耦执行模型，最后讨论异步执行与通信-计算重叠机制，为后续系统设计提供理论基础。

### 2.3.1 耦合式执行模型及其局限性

传统加速器通常采用耦合式执行模型 (coupled execution model)，其执行流程具有严格顺序关系：任务指令触发数据搬运，数据到达后才能启动计算，当前任

务结束后才能解析下一条指令<sup>[67-68]</sup>。在此模式下，指令流与数据流形成强同步关系，其执行路径具有以下特点。

(1) 指令流与数据流的顺序依赖

耦合式模型本质上是一种阻塞式控制结构。指令下发依赖数据搬运完成，任务执行依赖数据通路空闲，任意阶段延迟都会导致指令停顿，使系统性能受数据路径最慢阶段限制。

(2) 跨节点通信延迟直接暴露给控制路径

在多节点系统中，当任务放置与数据驻留不一致时，需要通过网络传输数据。通信链路的延迟、带宽波动或竞争冲突会阻塞指令路径，使调度器无法持续推进任务调度，从而降低系统并行度<sup>[69]</sup>。

(3) 流水线难以形成深度重叠

耦合式模型将数据搬运、指令解析和计算准备串联为单一流程，难以并行推进，流水线重叠度较低。在远端数据访问频繁时，流水线中断更加明显。

(4) 缺乏对运行时动态性的适应能力

多节点系统中的网络负载、节点资源与数据驻留位置具有动态变化。耦合式模型要求控制流与数据流严格同步，难以吸收这些变化，容易造成部分资源空闲而部分资源过载。

因此，耦合式执行模型难以充分利用多节点系统中的通信-计算并行性，需要通过更高层次的执行结构解耦指令与数据之间的时间依赖。

### 2.3.2 指令-数据分离与解耦执行模型

为突破耦合式执行模型的串行瓶颈，现代加速器体系结构普遍采用指令通路 with 数据通路分离 (dual-path separation) 设计，使控制路径与数据路径能够在时间上独立推进<sup>[38, 67]</sup>。该结构在物理链路与逻辑功能上分离控制操作与数据搬运，使控制流持续推进，而数据流承担高带宽的数据迁移任务。

(1) 指令-数据双通路的结构特征

双通路结构通过区分控制平面 (control plane) 与数据平面 (data plane) 实现。控制平面负责指令解析、依赖判断和调度决策等控制操作；数据平面负责数据加载、片上搬运及跨节点传输。通过控制与数据路径分离，系统能够避免控制流程被数据搬运阻塞。

(2) 解耦执行模型的理论基础

双通路结构为解耦执行模型 (decoupled execution model) 提供体系结构基础，其核心思想是将数据访问延迟从控制路径中分离，使控制路径能够提前推进，而数据路径在后台完成数据准备。根据控制与数据路径之间的依赖程度，解耦执行

模型主要分为两类：

#### 1) 完全解耦模型 (Fully-Decoupled Execution)

完全解耦模型通常包含访问流水线与执行流水线，二者通过队列或令牌进行弱同步。访问路径可提前发起数据搬运，执行路径在数据就绪时触发计算，该机制与数据流计算模型具有一定相似性<sup>[39]</sup>。该模型能够有效隐藏访存和通信延迟，但需要更复杂的状态管理与同步机制。

#### 2) 部分解耦模型 (Partially-Decoupled Execution)

部分解耦模型保持控制流主导地位，数据操作由指令触发，但两者在时间上无需严格同步。该结构实现简单且可控性高，在动态调度和分布式执行环境中具有良好适应性。

由于实现复杂度较低且能够提升并行度，部分解耦模型已成为多加速器执行框架中的常见形式，而指令-数据分离结构正是其重要基础。

### 2.3.3 异步执行与通信-计算重叠

在双通路结构与解耦执行模型基础上，异步执行 (asynchronous execution) 成为提升系统性能的重要运行时机制。异步执行不要求任务等待所有前置阶段完成，而是依据任务状态与数据可用性推进执行，使不同阶段能够在时间上重叠。

#### (1) 基于就绪状态的任务推进模型

在异步执行模式中，任务由运行时维护状态，如 Ready、Running、Blocked 等。任务进入可执行状态需满足两类条件：a. 依赖关系满足 (DAG 入度归零)；b. 输入数据可用 (由数据路径提供状态信息)。任务推进由事件驱动或状态更新触发，而非整体同步，因此形成非阻塞的事件驱动调度过程。

#### (2) 异步语义下的流水线构造

异步模式允许指令处理、数据搬运和计算准备在时间上并行推进。执行过程可抽象为三阶段结构：调度与指令处理阶段  $S(t)$ ，数据传输阶段  $D(t)$ ，算子执行阶段  $C(t)$ 。

在同步模型中， $S$ 、 $D$ 、 $C$  顺序执行；在异步模型中，三者可同时位于时间轴不同位置，形成跨阶段流水化 (pipelining)。

#### (3) 通信-计算重叠的系统效应

通过解耦结构与异步语义的结合，系统能够实现：数据搬运延迟隐藏 (latency hiding)；调度器持续推进任务图前沿；减少计算单元因数据未就绪而产生的空闲时间；提升跨节点执行场景下的吞吐率与资源利用率<sup>[38, 67]</sup>。

因此，异步执行不仅是一种运行时优化机制，也是解耦执行框架在时间维度上的延伸，其有效性依赖于指令-数据分离结构与部分解耦执行范式。

### 第三章 张量计算调度系统总体架构

在第二章的理论分析基础上,本文针对传统控制流与数据流强耦合执行模型在多节点环境中的通信瓶颈与流水线停顿问题,提出一种面向张量分块计算的软硬协同调度系统。该系统基于“指令-数据解耦”的设计思想,引入FPGA硬件调度器,构建主机、调度器与张量加速器协同工作的执行架构。本文关注的重点在于系统级任务调度与数据管理机制,而非具体算子实现。因此,在后续设计中将对加速器进行统一抽象,视为具备标准接口的计算节点,以便从系统层面对多节点协同执行过程进行建模与分析。

本章将对该系统的总体架构设计与核心控制机制进行详细阐述。首先,3.1节将从宏观层面介绍面向解耦执行的三层系统架构,明确各层次的功能划分,并详细分析控制通路 with 数据通路的分离设计及整体执行流程。随后,3.2节将聚焦于系统的软硬交互接口,定义面向张量计算的领域专用指令体系,阐述指令的类型、格式以及基于异步队列的握手与执行模型。本章的设计为后续第四章的数据管理机制与第五章的任务调度策略提供了基础架构支撑。

#### 3.1 系统整体架构设计

##### 3.1.1 面向解耦执行的三层系统架构

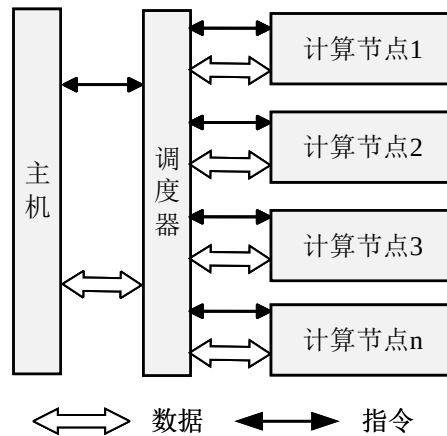


图 3-1 整体架构图

如图3-1所示,系统架构遵循“软硬优势互补”与“逻辑-物理分离”的设计原则,主机软件层负责任务编排与控制管理,FPGA硬件层负责实时资源映射与

执行调度。

在主机软件层，系统构建了一套支持动态依赖图 (Dynamic DAG) 管理的运行时环境。该环境支持任务异步注入，允许在前一批任务未完成时动态加入新的算子节点，同时系统实时更新依赖关系并筛选就绪任务。主机利用通用处理器的大容量内存与灵活控制能力，实时监控 DAG 节点拓扑状态，并依据 FPGA 调度层反馈的任务完成信号更新图结构，从而筛选出当前满足依赖条件（入度为 0）的就绪指令流。通过将复杂的依赖分析与图搜索逻辑保留在软件层，系统确保下发至硬件的任务在逻辑上均处于可执行状态。

在 FPGA 调度器层，系统将核心职责从依赖分析转移至物理资源映射决策。面对主机下发的就绪指令流，调度器无需处理算子间的逻辑顺序，而是依据计算节点的实时负载与数据驻留状态进行硬件级调度。通过多因素调度策略，计算任务可优先分发至数据所在节点或负载较低节点，从而减少上下文切换开销并提升计算阵列的吞吐率与数据复用率。

#### （1）主机软件层

主机层作为系统的编译管控中心，主要运行于通用处理器 CPU 上，负责高层任务的解析、数据预处理与全局状态管理。该层旨在实现软硬件协同的初始化与控制流管理，其软件架构与逻辑模块划分如图3-2所示，主要包含以下三个核心子系统。

1) 张量数据预处理与管理子系统：图3-2中灰色部分模块，该子系统负责计算任务的前置数据准备。它将用户输入的高维张量数据切分为硬件友好的固定尺寸图块 (Tiling)，完成精度转换与内存映射 (Memory Mapping)，为后续 FPGA 侧 DMA 搬运提供标准化的数据布局。

2) 动态任务编排与指令生成子系统：图3-2中绿色部分模块，该子系统是主机层的逻辑控制核心。它负责维护动态依赖图的生命周期，实时监控算子节点的依赖状态。一旦检测到节点入度归零，即判定其逻辑就绪，并将其序列化为“就绪指令流”下发，确保下层硬件仅需关注物理资源映射。

3) 全局状态监控与反馈子系统：图3-2中浅蓝色部分模块，该子系统是软硬件交互的信息反馈闭环。它负责实时捕获 FPGA 上报的中断信号与状态报文，解析任务完成状态并反馈至编排模块，从而触发 DAG 图的依赖消除与新一轮指令发射，实现自动化的闭环控制。

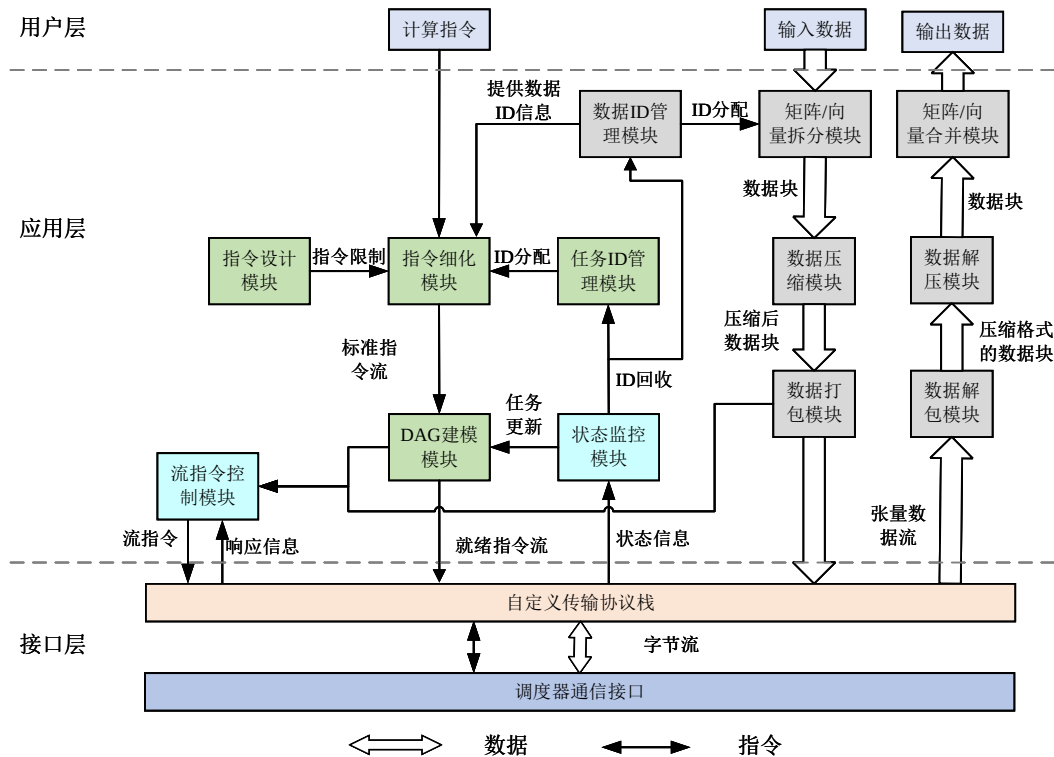


图 3-2 主机软件层架构图

## (2) FPGA 调度器层

FPGA 调度器层是异构系统的运行时管控中枢，位于主机与计算节点之间。其核心职能是实现指令的硬件级解析、数据的自动化搬运、数据的多副本一致性管理及计算任务的动态分发。其整体架构及模块交互关系如图3-3所示，该层由以下功能单元组成：

1) 主机通信接口单元：作为顶层物理入口，负责适配 PCIe 等高速总线协议，提供统一的硬件抽象层 (HAL)，实现主机与 FPGA 间的高带宽通信。

2) 协议解析与前端分发单元：负责对输入数据流进行硬件级解码，依据协议特征将混合码流实时分离为流控制指令、计算任务指令与张量数据流。

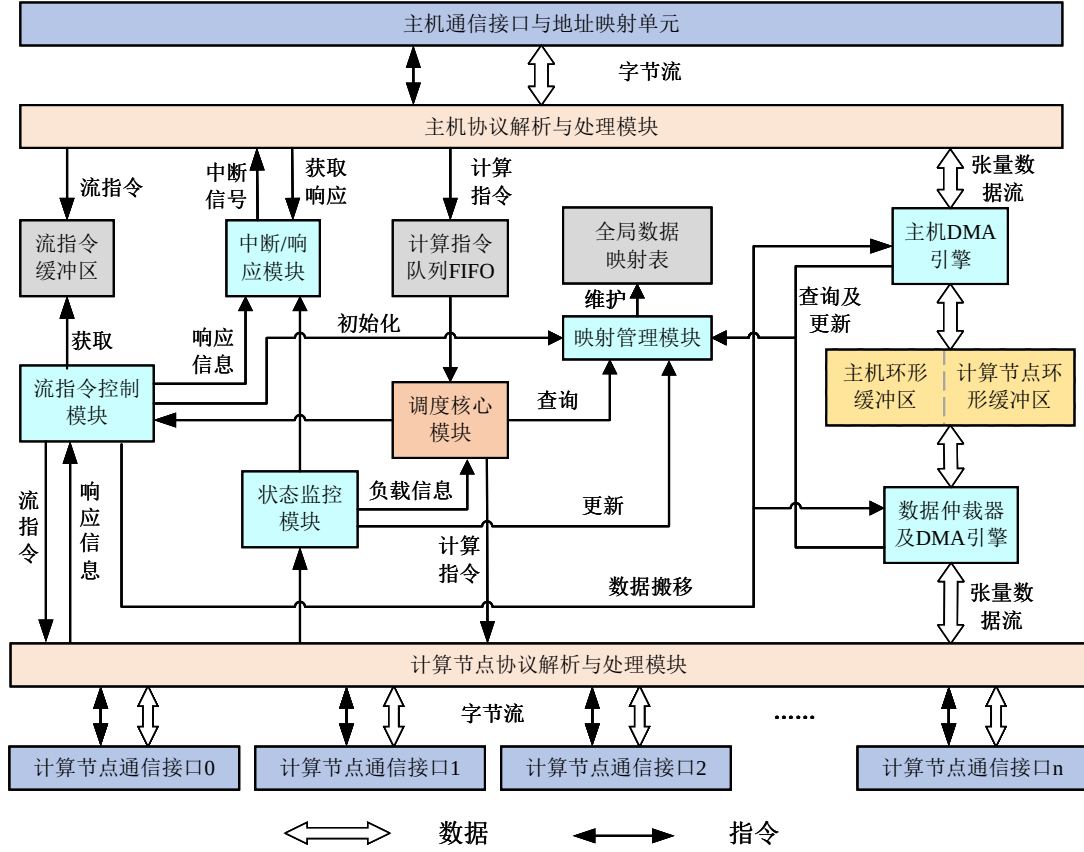
3) 流指令控制与交互协同单元：作为系统交互的控制核心，负责解析管理类指令，驱动 DMA 引擎与状态寄存器，并统一管理对外（主机及节点）的请求-响应握手逻辑。

4) **数据驻留管理与散列-聚集 DMA 单元:** 构成系统的数据平面核心。该单元维护全局数据映射表以管理数据驻留状态,并利用散列-聚集 (Scatter-Gather)DMA 引擎实现大规模张量数据在异构存储层级间的高效流转与仲裁。

5) 调度核心单元：作为决策中枢，负责根据数据在节点间的分布情况及节点实时负载状态，动态将计算任务映射至最优计算节点。

6) 全局状态监控与反馈单元：负责收集各计算节点的执行状态与负载信息，同步更新全局数据映射表，并向主机上报任务完成情况，形成控制闭环。

7) 计算节点互联与通道隔离单元：负责构建调度器与计算阵列间的物理互联拓扑，提供独立的指令发送通道与数据传输通道，并实现核心逻辑与计算阵列间的跨时钟域隔离。



### (3) 计算节点层

计算节点层是异构计算任务的物理执行终端。在本设计中，计算节点被抽象为具备标准化接口的“通用算力黑盒”，对外屏蔽具体的算子实现逻辑（如脉动阵列、向量单元等）。典型计算节点单元如图3-4所示，其对外特征包含三个维度：

1) 标准化双通道接口：提供物理分离的指令接收通道与数据读写通道，分别用于接收调度器下发的控制指令与 DMA 引擎传输的张量数据。

2) 事件驱动执行机制：采用从属模式运行，可接收并缓存多条计算指令；内置独立任务控制器，支持指令队列的自主调度与生命周期管理，在接收到有效指令后触发内部状态机启动计算流程，任务全部执行完毕后自动挂起。

3) 标准状态反馈能力：具备硬件级的状态汇报机制，能够实时向调度层反馈任务完成信号 (Done) 及通信握手信号 (ACK)，支持上层进行精确的依赖管理。

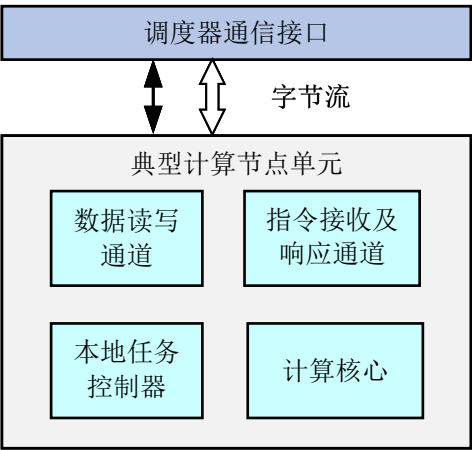


图 3-4 典型计算节点单元架构图

3.1.2 数据通路与指令通路解耦设计

传统异构计算架构中，指令控制与数据传输往往共享同一物理链路，导致大数据搬运期间控制信号无法及时下发。为此，本系统采用了“物理通道解耦、逻辑握手协同”的设计策略。通过构建相互独立的指令与数据硬件通路，配合严格的请求-响应机制，实现了大吞吐数据传输与实时控制指令的并行处理。其整体架构如图3-5所示。

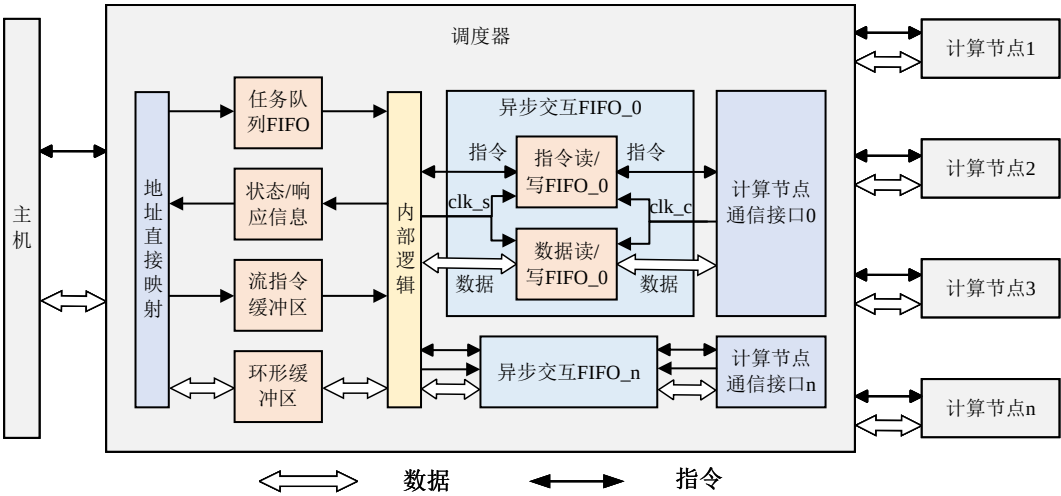


图 3-5 双通路解耦设计图

(1) 双通道物理拓扑架构

为支撑控制流与数据流的物理分离架构设计，系统在主机接口与计算节点接口两侧构建了独立的硬件传输通路，实现两类数据流的物理隔离与高效传输。

在主机侧映射接口层面，系统采用直接内存访问映射 (Direct Memory Mapping) 机制，将 FPGA 内部关键存储资源映射为主机可直接访问的地址空间，通过地址范围划分实现不同类型访问流的精准区分。该映射空间包含四类核心存储组件：用于交互控制信令的流指令缓冲区与状态 / 响应缓冲区、用于缓存计算

任务的指令队列 FIFO，以及用于暂存大规模张量数据的环形缓冲区；主机通过对不同地址范围的定向读写操作，从硬件层面完成控制指令流与张量数据流的物理分离，避免两类流的相互干扰。此外为主机端对环形缓冲区的读写设计了专门的 DMA 引擎以高效完成张量数据传输。

在计算节点侧双通道接口层面，系统在调度器与计算节点之间部署了两条物理隔离的总线通路。其一为指令控制通路，连接调度器的逻辑控制核心与计算节点的指令接收端口，专门负责传输计算任务指令及用于启动数据传输、查询设备状态的流控制指令；其二为张量数据通路，连接调度器的 DMA 引擎与计算节点的数据读写端口，仅用于传输存储在环形缓冲区中的大规模张量数据。通过总线级别的物理隔离，进一步保障控制流与数据流传输的独立性、稳定性和高效性。

### （2）逻辑协同与执行重叠机制

在物理通路解耦的基础上，系统在逻辑层面进一步采用“先握手、后传输”的协同机制，通过可靠的交互确认保障数据传输稳定性，并在握手完成后实现多阶段执行的并行重叠，最大化硬件资源利用率。

主机侧的协同交互遵循严格的握手 - 传输流程：当主机存在数据传输需求时，首先向流指令缓冲区写入传输请求指令，随后通过状态缓冲区实时查询调度器反馈状态，在接收到“就绪”响应后，启动 DMA 引擎将数据批量写入环形缓冲区。依托物理双通道的隔离优势，数据传输启动后仅占用数据通路带宽，指令通路即刻恢复空闲状态；此时主机无需等待数据传输完成，可通过指令通路继续向指令队列 FIFO 写入后续计算任务指令，成功实现“数据搬运”与“指令下发”的时间重叠，避免单一流程阻塞导致的效率损耗。

计算节点侧则通过流水线化设计实现多任务并行重叠，调度器与计算节点的交互同样遵循“先握手、后传输”逻辑：调度器首先通过指令控制通路下发数据交互类流指令，在接收到计算节点反馈的确认响应后，触发 DMA 引擎通过张量数据通路执行张量搬运操作。在 DMA 进行长周期张量传输的过程中，指令控制通路处于空闲状态，调度器可充分利用该窗口期，通过指令通路向当前节点预下发下一阶段计算任务指令，或并行向其他计算节点下发任务指令，实现数据传输与指令配置的流水线重叠。

该协同机制兼具可靠性与高效性：一方面通过握手确认确保接收端处于就绪状态，保障数据传输的逻辑可靠性；另一方面充分发挥物理双通道的隔离特性，将耗时的“数据传输阶段”与轻量级的“指令配置阶段”在时间轴上并行化，显著提升了系统整体执行效率。

### （3）跨时钟域异步缓冲设计

考虑到 FPGA 调度器核心逻辑（运行于系统时钟  $clk_s$ ）与计算节点阵列（运

行于计算时钟  $\text{clk}_c$ ) 工作在不同的物理硬件上, 本设计在调度器与计算节点的互联接口处, 分别在指令通路 with 数据通路上部署了双向异步 FIFO, 如图3-5所示。

该异步 FIFO 负责处理跨时钟域 (CDC) 的信号传递。调度器将指令或数据写入 FIFO 输入端, 计算节点在各自的时钟域下从 FIFO 输出端读取, 从而在物理层面实现了调度逻辑与计算逻辑的时钟域解耦, 保障了在握手与传输过程中的信号稳定性。

### 3.1.3 整体执行流程分析

本系统的单次计算任务生命周期严格遵循“控制流驱动、数据流就绪”的原则, 其执行流程包含“主机发射—调度器映射—节点执行—闭环反馈”四个关键阶段, 详细层级交互关系如图3-6所示。

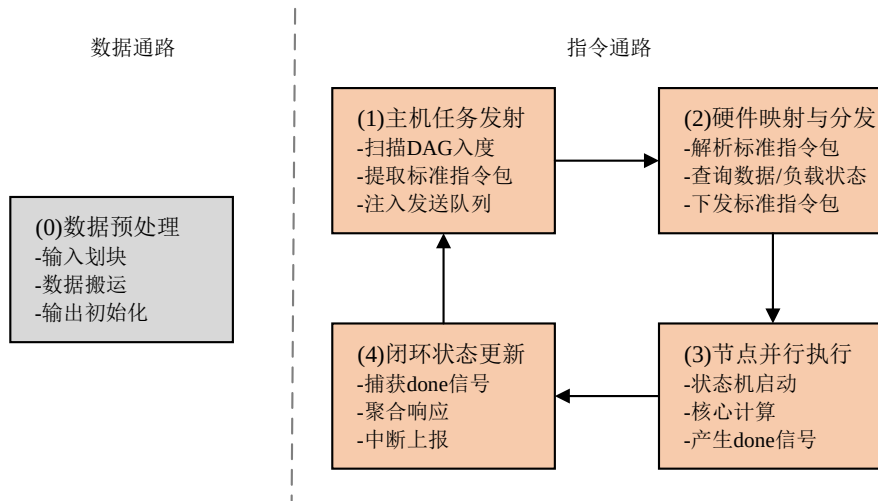


图 3-6 数据 - 指令双通路执行流程图

#### (1) 阶段一：主机依赖解析与任务发射

主机端的任务编排子系统实时维护全局动态依赖图。系统通过节点入度信息监控算子状态, 当检测到某算子节点的前驱任务全部完成 (即入度计数器归零) 时, 将其标记为“逻辑就绪”。

此时, 系统不再进行耗时的编译或序列化操作, 而是直接从该算子节点的预编译结构体中提取已封装好的标准指令包。随后, 该指令包被推入发送队列, 通过“指令通路”异步下发至 FPGA。

值得注意的是: 在此阶段之前的阶段 0 中, 张量数据预处理子系统已通过流指令完成了输入数据的切分与搬运, 以及输出结果缓冲区的预分配, 确保指令下发时相关数据在物理层面上已进入“在途”或“就绪”状态。

#### (2) 阶段二：硬件级依赖检查与动态映射

FPGA 调度器接收到标准指令流后, 调度核心首先根据指令中的全局数据 ID

查询全局数据映射表，以确认操作数的物理驻留状态。

情形 A（数据未就绪）：若因传输延迟导致输入数据尚未完全写入片上缓存，或前序节点的输出尚未落盘，调度器会将该指令暂时挂起至保留站。等待数据就绪信号触发唤醒，而非阻塞后续无关指令的发射。

情形 B（数据就绪）：若数据状态校验通过，调度器采用一定的调度策略，优先将指令分发至持有数据副本的计算节点；若目标节点繁忙，则根据实时负载信息选择最优空闲节点，完成从逻辑指令到物理微操作的映射。

### （3）阶段三：节点并行执行与反馈

目标计算节点在接收到标准指令包后，立即启动本地状态机。得益于节点内部独立的片上存储架构，计算逻辑与存储访问互不冲突。在计算核心读取本地数据进行运算的同时，DMA 引擎可利用片上存储的剩余空间，并行写入下一阶段的预加载数据或迁出中间结果。

节点作为“黑盒”独立完成运算后，不直接干预复杂的数据回传逻辑，而是仅通过控制通路拉高完成信号 (Done Signal)，并在本地更新状态寄存器，表明该计算单元已释放并准备好接收新任务。

### （4）阶段四：闭环状态更新与负载上报

FPGA 调度器的状态监控单元在捕获计算节点上报的执行完成信号 (Done 信号) 后，启动关键闭环操作，实现状态同步、负载更新与任务流转自动衔接。

首先，进行状态更新：调度器在硬件层面修改全局数据映射表中输出结果数据的状态位，将其置为“有效 (Valid)”，解锁所有依赖该数据的后续指令，确保指令流的连续推进。

其次，刷新负载信息：调度器同步采集计算节点的负载指标，包括计算核心空闲状态、片上内存占用等，实时更新至调度核心的决策表中，为本地性分发和负载均衡提供精准数据支撑，确保动态适配。

最后，中断上报与任务闭环触发：调度器向主机发送中断或状态报文反馈执行结果。主机收到反馈后，递减任务依赖图中后继节点的入度计数；当某后继节点的入度计数为零时，触发“阶段一：任务发射”流程，形成自动化流水线任务执行闭环。

## 3.2 指令体系设计

本系统构建了一套分层级的“流-计算协同指令体系”。该体系在架构设计上摒弃了传统单一指令流混杂控制与计算的模式，而是采用了“载体-负载”的分层封装模型。

其中，变长流指令 (Stream Instruction) 充当系统的传输与控制协议层，负责

跨物理链路如 PCIe 的可靠握手、数据搬运与任务分发；定长计算指令 (Compute Instruction) 则作为被流指令携带的核心负载，专注于描述计算节点内部的算术逻辑。这种设计使得底层的指令传输通路与上层的计算执行模型实现了解耦，无论底层物理接口采用 PCIe、Aurora 还是光互连，上层的指令交互逻辑均保持一致。

### 3.2.1 指令类型与格式

本系统的指令体系由两层协议构成：外层采用变长帧结构的流指令，作为全系统通用的通信载体，承载控制信令、数据描述符及状态字；内层采用 128-bit 定长的计算指令，作为算术逻辑的最小执行单元，被封装在流指令中进行分发。详细的指令格式如图3-7所示。

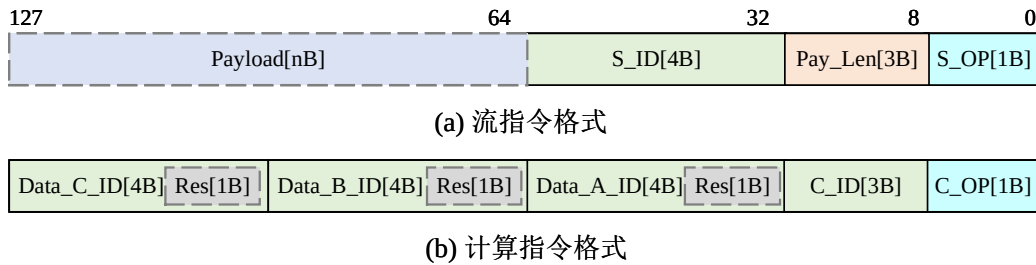


图 3-7 指令格式图

#### (1) 变长流指令

流指令遵循“定长头部加变长负载”的帧结构，头部包含路由与控制信息，负载则根据操作码的不同承载计算指令块、张量数据 ID 或状态字。流指令的统一帧头为 64bit，负载长度则根据不同的操作码对应不同的长度，为适配底层总线位宽，流指令负载长度强制向 64bit 对齐。不足部分填充无效数据 (Padding)。其位域定义如表3-1所示。

表 3-1 流指令帧位域定义

| 字段名称    | 位宽          | 对应位       | 功能描述              |
|---------|-------------|-----------|-------------------|
| S_OP    | 8-bit(1B)   | [7:0]     | 操作码，控制流指令的解析与去向。  |
| Pay_Len | 24-bit(3B)  | [31:8]    | 负载字节数，用于对齐硬件解码。   |
| S_ID    | 32-bit(4B)  | [63:32]   | 流指令 ID，用于请求与响应匹配。 |
| Payload | $n$ B (不定长) | [帧最高位:64] | 负载内容，具体由操作码决定。    |

依据 S\_OP 字段所定义的控制语义，本系统的流指令体系被划分为任务分发、数据管理与状态反馈三大功能类别。各类指令在遵循统一帧头标准的基础上，采用了差异化的负载 (Payload) 编码格式与交互逻辑，以适配异构架构场景下的多样化传输需求。如表3-2所示。

#### 1) 任务分发类指令

表 3-2 流指令类型与功能分类

| 类别   | 指令名称      | OP 码 | 功能说明          |
|------|-----------|------|---------------|
| 任务分发 | SEND_INST | 0x0A | 批量发送计算指令序列    |
| 数据管理 | SEND_DATA | 0x0B | 输入数据到达通知      |
|      | INIT_DATA | 0x11 | 预分配输出数据空间     |
|      | GET_DATA  | 0x0C | 请求回读计算结果      |
|      | FREE_DATA | 0x0D | 释放片上缓存资源      |
| 状态反馈 | STATUS    | 0x0E | 节点状态与任务完成信息上报 |
|      | ACK       | 0x0F | 指令确认与事务匹配     |

负责计算任务的下行传输，其核心指令为 SEND\_INST(OP=0x0A)。该指令的 Payload 作为计算指令容器，用于封装若干条 128-bit 定长计算指令。接收端根据 Payload 长度将其拆分为微指令序列并推入 Instruction FIFO，实现“一次握手、批量发射”的任务注入机制。为避免硬件 FIFO 溢出，单个流指令帧最多承载 256 条计算指令。

## 2) 数据管理类指令

数据管理类指令用于维护系统的全局数据映射关系，是实现控制流与数据流解耦的重要机制。此类指令仅传输数据描述符而不携带实际张量数据，其 Payload 由若干 32-bit 描述符组成，每个描述符包含数据属性与全局数据 ID。系统通过 SEND\_DATA、INIT\_DATA、GET\_DATA 与 FREE\_DATA 四类操作实现数据注册、缓冲区初始化、结果回读及缓存释放，从而完成数据生命周期管理。

## 3) 状态与反馈类指令

状态反馈类指令由 FPGA 或计算节点主动发起，用于维护软硬件执行闭环。其中 STATUS 指令用于上报节点运行状态及已完成任务信息，而 ACK 指令用于确认流指令事务，通过回填 S\_ID 实现异步事务匹配，从而保证控制信令在异步环境下的可靠传输。

## (2) 定长计算指令

区别于流指令的变长特性，核心计算层采用了 128-bit 超长指令字 (VLIW) 架构。该设计强制所有算子遵循统一的编码布局，旨在充分利用 FPGA 的并行位宽优势简化硬件解码逻辑。其核心设计理念体现在“任务标识的全局性”与“操作数寻址的冗余扩展性”两个维度，具体位域定义如表3-3所示。

具体而言，指令的低位段由计算操作码 (C\_OP) 与全局任务 ID(C\_ID) 构成。其中，位于 [7:0] 的操作码支持最多 255 种算子类型，覆盖了从基础 GEMM、CONV 到非线性激活 (ReLU) 等多种运算，预留了充足的算子库扩展空间；而位

表 3-3 计算指令字段功能定义

| 字段名称      | 位域范围     | 位宽     | 功能描述                        |
|-----------|----------|--------|-----------------------------|
| C_OP      | [7:0]    | 8-bit  | 计算操作码，定义算术逻辑类型，配置核心数据通路。    |
| C_ID      | [31:8]   | 24-bit | 全局任务 ID，用于执行状态回写与 DAG 依赖消除。 |
| Data_A_ID | [63:32]  | 32-bit | 输入 A 数据索引，高位保留，低位为数据映射表索引。  |
| Data_B_ID | [95:64]  | 32-bit | 输入 B 数据索引；单操作数或立即数算子中可被复用。  |
| Data_C_ID | [127:96] | 32-bit | 输出结果索引，指示计算结果的全局回写目标。       |

于 [31:8] 的 24-bit 任务 ID 则是贯穿任务生命周期的唯一凭证，计算节点在执行完毕后，将通过携带该 ID 的 Done 信号向调度器反馈，从而实现精确到指令级的闭环追踪。

指令的高 96 位被划分为三个标准化的 32-bit 操作数索引字段，分别对应输入操作数 A、B 与输出结果 C。为提升寻址灵活性并支持未来异构数据类型，系统针对每个操作数索引采用了“8+24”的冗余位宽设计：低 24-bit 作为全局数据 ID 索引，用于在全局映射表中检索数据状态及副本位置信息；而高 8-bit 则设计为属性扩展位 (Attribute/Reserved)。在当前版本中，该属性只在涉及立即数操作时与 24bit 一起构建 32bit 位作为立即数存储空间，未定义的位则作为保留资源，为后续系统支持混合精度计算或稀疏编码格式预留了底层架构接口。

### 3.2.2 指令通路与握手机制

在高并发任务调度场景下，控制指令的传输可靠性和处理效率直接影响系统整体吞吐性能。为解决变长指令在物理通路上传输时带来的解码复杂度、时序压力及队列堵塞问题，本节设计了一套基于流指令的可靠传输机制。流指令作为系统的传输层载体，在主机与调度器之间以及调度器与计算节点之间建立了统一的逻辑通道。系统采用“强制对齐传输”与“软硬协同分级流控”机制，确保了控制信令在高并发场景下的处理效率与通信稳定性。

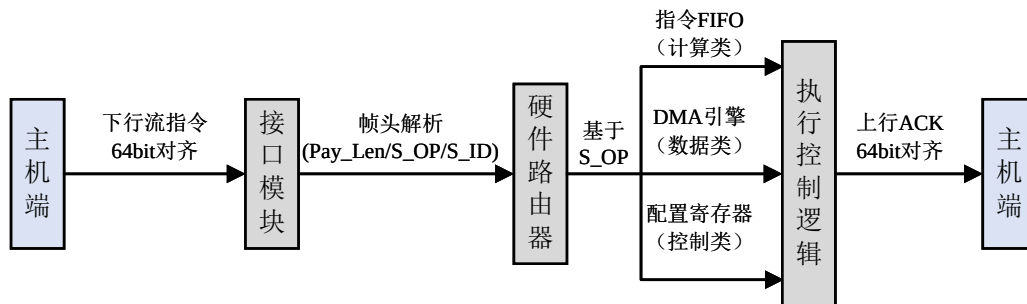


图 3-8 流指令传输与处理流程示意图

图3-8展示了流指令在主机与调度端之间的物理传输流程，包括基于 64-bit 对齐的指令封装、帧头字段解析、基于 S\_OP 的硬件分流处理，以及基于事务标

识的异步 ACK 反馈机制。该传输与确认模型同样适用于调度端与计算节点之间的指令通路。

#### (1) 下行传输：强制对齐与解析路由

在变长指令场景下，跨周期拼接带来的硬件复杂度和时序压力是主要瓶颈，下行指令传输采用强制对齐策略。尽管流指令负载是变长的，但系统要求所有传输事务必须以 64-bit 为边界进行对齐。

对齐填充机制：发送端在封装流指令时，若负载长度非 64bit 整数倍，会在尾部自动填充无效数据 0 直至补齐 64-bit。接收端硬件接口首先解析帧头中的 Pay\_Len，向上取整计算读取周期数，并在最后一拍自动丢弃填充位。这种设计虽然牺牲了微小的带宽，但避免了跨周期拼接数据的时序开销，显著提升了硬件解析器的吞吐率。

多级路由分发：帧头解析完成后，硬件路由器依据 S\_OP 执行分流：计算任务指令被推入指令 FIFO；数据管理类指令（如 SEND\_DATA）仅解析其中的数据描述符（地址、长度），随即触发 DMA 引擎在独立的数据通路上启动张量搬运；控制类指令则直接作用于配置寄存器。

#### (2) 上行反馈：基于 S\_ID 的异步事务匹配

鉴于不同流指令的执行延迟根据负载的不同具有不确定性，系统采用异步事务匹配机制而不是同步等待阻塞以提高通信效率。

异步闭环：每个下行指令包在帧头携带唯一的 S\_ID。当 FPGA 完成指令接收时，通过上行通道回传 ACK 流指令，并回填相同的 S\_ID。

乱序确认：主机侧维护一个“在途事务表”。收到 ACK 后，依据 S\_ID 消除对应表项。这一机制允许后发的短任务先于先发的长任务返回确认，充分释放了物理链路的并发能力。

#### (3) 可靠性设计：软硬协同分级流控

为防止指令队列溢出导致的死锁，系统构建了两级流控防线：一级防线（软件预测）：主机驱动代码周期性读取 STATUS 指令反馈的“队列深度”快照，在软件层维护虚拟信用积分。在发射新指令前，驱动代码预判剩余空间，若空间不足则主动挂起发送线程，减少无效的总线事务。二级防线（硬件反压）：当发生突发流量导致硬件 FIFO 满载时，调度器接口层直接拉低物理接口的 Ready 信号。该信号触发底层链路的反压机制，从硬件物理层暂停数据泵入，确保内部逻辑不发生溢出丢包。

### 3.2.3 异步指令队列与执行模型

本系统的指令执行模型涵盖了从主机生成任务到最终结果回收的全生命周期。系统采用“批量发射、异步执行、乱序完成”的策略，通过全链路 ID 追踪机制，在保证算力利用率的同时维护了逻辑执行的正确性。图3-9 中展示了计算指令从 DAG 依赖建模、批量发射、队列缓存、调度决策到并行执行及乱序完成回传的完整生命周期。系统通过 C\_ID 对指令进行全流程追踪，使得主机能够在不关心指令完成顺序的情况下更新 DAG 状态并完成资源回收。

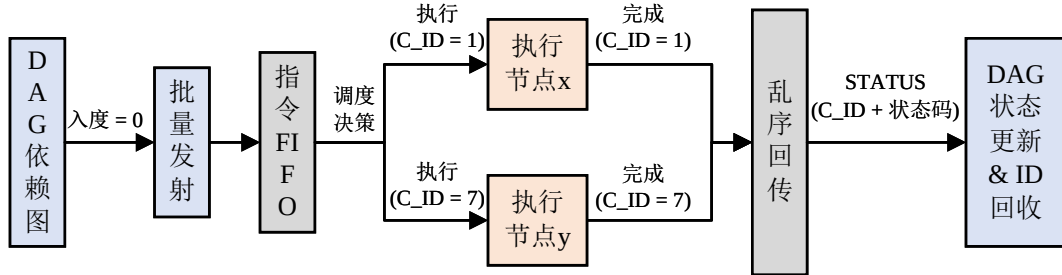


图 3-9 计算指令全生命周期与乱序完成执行模型示意图

#### (1) 计算指令全生命周期管理

为支持多计算节点环境下的高并发任务执行，本系统在指令层面引入了以调度决策为核心的生命周期管理机制。每条计算指令从生成到回收需经历生成与依赖建模、指令注入、调度决策、并行执行以及结果回收五个阶段，共同构成一个可重入的执行状态机，如图3-9所示。

在主机侧，用户提交的计算任务首先被抽象为标准计算指令，并注入 DAG 依赖管理模块。系统通过对指令间数据依赖关系进行显式建模，仅当指令所有前驱节点完成后，才将其标记为就绪状态，从逻辑层面保证了执行顺序的正确性。

就绪指令随后由驱动程序以批量方式封装为流指令包，并通过物理链路下发至 FPGA。该批量注入机制减少了主机与设备之间的交互频率，为后续大规模并发调度提供了高效的指令供给能力。

指令到达 FPGA 后被剥离传输封装并进入指令 FIFO 缓冲区。该缓冲层位于主机高带宽发射域与调度决策判定域之间，用于平滑指令到达速率与节点消耗速率之间的差异，并为调度决策提供稳定的指令输入。

调度层从指令 FIFO 中提取就绪指令，并依据当前各计算节点的负载状态及资源可用性完成调度决策，选择合适的目标计算节点执行指令。该调度过程将计算指令与具体硬件执行资源动态绑定，是系统支持多节点并行执行与负载均衡的关键环节。

指令在目标计算节点内独立执行。由于不同节点算力特性及任务规模差异，指令在执行阶段呈现明显的异步特性，其完成时间与发射顺序无直接对应关系。

当计算节点完成任务后，执行结果及对应指令标识被回传至调度层。调度器对来自不同节点的完成事件进行聚合处理，并统一上报主机侧。主机依据返回信息更新 DAG 状态并回收指令标识，从而完成计算指令生命周期的闭环管理。

### （2）乱序完成机制与基于 ID 的全链路追踪

在多计算节点并行执行的场景下，由于节点算力差异及任务规模不一致，计算指令的完成顺序往往与其发射顺序不一致，呈现出典型的乱序完成 (Out-of-Order Completion) 特性。为在异步执行条件下维持 DAG 语义的一致性，系统引入了基于指令标识的全链路追踪机制。

每条计算指令在生成阶段被分配唯一的 24-bit 指令标识 C\_ID，该标识作为指令生命周期的追踪锚点，贯穿 DAG 建模、调度决策、节点执行以及结果回传等所有阶段。调度层在执行阶段将 C\_ID 与具体计算节点进行绑定，并在指令完成时依据 C\_ID 对完成事件进行识别与聚合。

各计算节点在完成指令执行后，将携带对应 C\_ID 的完成信息上报至调度层。调度层无需保证完成事件的顺序性，而是将来自不同节点的完成事件统一封装为 STATUS 流指令批量回传至主机侧。

主机驱动代码在接收 STATUS 信息后，仅需依据 C\_ID 在 DAG 图中索引对应节点并标记完成，而无需关心完成事件的回传顺序。该机制使系统能够在充分释放硬件并发能力的同时，保持 DAG 依赖关系的正确推进，并实现计算指令标识的自动回收。

### （3）异常处理机制与执行状态监控

为增强系统在异步并行执行环境下的鲁棒性，执行模型在指令完成回传路径中引入了统一的执行状态编码机制。计算节点在指令执行完成的同时，上报对应的执行状态，以辅助主机侧进行精确的异常定位与处理。

系统采用 2-bit 执行状态码对计算结果进行编码，其中包括正常完成、数值溢出、非法操作以及执行超时等典型异常情况。执行状态与对应的 C\_ID 一并上报至调度层，并通过 STATUS 流指令统一回传主机。

主机驱动代码在更新 DAG 状态时，可依据执行状态对异常算子进行精确定位，并触发相应的重试或报错流程。通过将异常处理机制与 DAG 更新过程相结合，系统能够在逻辑层面隔离异常结果，避免错误沿依赖链传播，从而维持整体计算过程的正确性与可控性。

## 第四章 张量数据管理机制与实现

在基于张量分块的多计算单元执行系统中，张量被划分为大量细粒度数据块，并在调度器与多个计算节点之间动态流转。随着任务规模与并行度的提升，系统中同时在途的数据块数量迅速增加，数据在主机、调度器及计算节点之间呈现出频繁迁移与并发访问的特征。因此，如何对分块张量数据进行有效管理，成为影响系统执行效率与正确性的关键问题。

在任务调度过程中，调度决策不仅取决于计算资源的空闲状态，还依赖于输入数据是否已驻留在目标节点。若缺乏统一的数据标识与元数据管理机制，调度器难以准确判断数据位置与复用关系，从而可能导致重复数据传输、增加通信开销并带来潜在一致性风险。

针对上述问题，本章围绕多节点环境下的张量分块数据管理机制展开设计。首先建立统一的数据标识与元数据管理机制，用于记录张量块的驻留位置与副本状态；在此基础上设计支持块稀疏张量的统一表示与传输方法；并在 FPGA 资源受限条件下实现轻量化的数据一致性管理策略。

围绕上述目标，本章将系统介绍张量数据管理机制的实现。4.1 节给出张量块的数据标识与元数据管理方案；4.2 节介绍块稀疏张量的统一表示方法及其序列化与传输流程；4.3 节讨论多节点环境下的数据一致性与正确性保障机制；4.4 节通过实验评估数据管理机制对数据本地性、通信开销及系统正确性的影响。

### 4.1 面向张量块的数据标识与元数据管理

在张量分块执行模式下，调度器对数据状态的感知是数据传输与任务调度的基础。为使调度器准确区分张量块并判断其可用性，本节介绍数据标识机制与元数据管理方案，为后续数据迁移与一致性控制提供支撑。

#### 4.1.1 张量块 ID 分配机制

在本系统中，张量被划分为固定大小的数据块作为最小管理单元。该粒度有助于简化调度器对数据状态的感知与管理，使数据标识、驻留判断及传输决策均在统一粒度下进行。

张量块大小的选择需权衡数据并行度、调度灵活性及元数据开销。块规模过小会增加数据块数量，提高管理复杂度；块规模过大会降低数据复用机会。综合

FPGA 片上存储容量、DMA 传输粒度及系统规模等约束，本系统选取  $128 \times 128$  作为默认块大小。

在分块及任务划分后，系统需对任务与数据对象进行统一标识管理，以避免动态调度下的数据或任务混乱。为此，本系统为任务与数据对象分别分配唯一标识 (ID)，并采用一致的 ID 分配与回收策略进行生命周期管理。任务 ID 与数据块 ID 维护独立的资源池，仅在各自语义空间内保证唯一性，支持并行管理并避免跨类型冲突。下文以数据块 ID 为例说明管理机制，任务 ID 设计与之一致。本节 ID 管理机制的整体流程如图4-1所示。

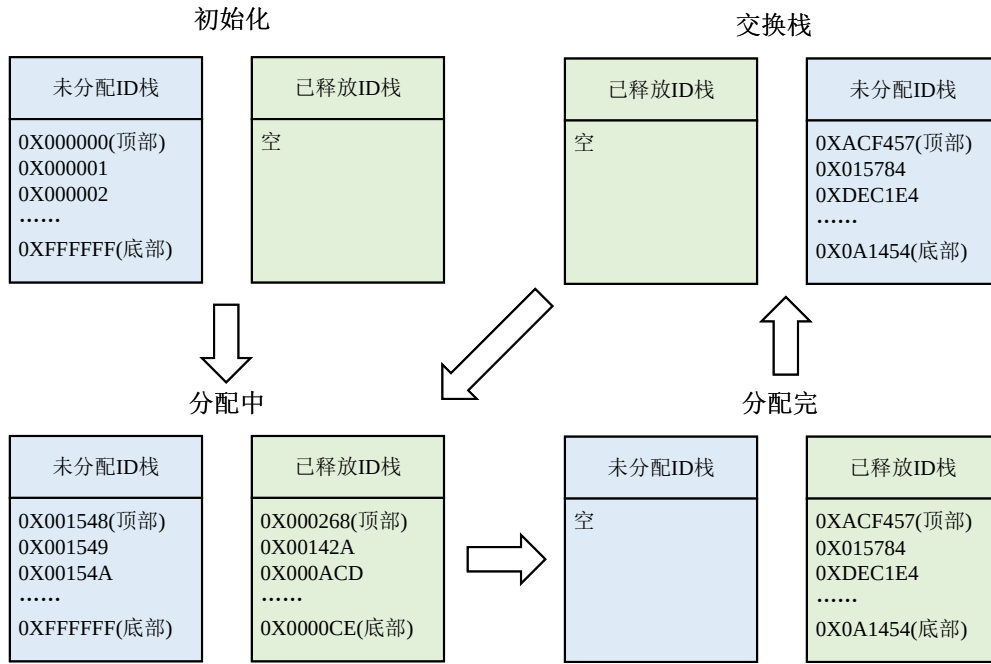


图 4-1 ID 管理示意图

### (1) ID 生成与分配机制

数据 ID 的生成与分配由主机侧张量数据预处理与管理子系统统一负责。系统在运行时维护两张基于栈 (Stack) 结构的 ID 管理表：可用 ID 表与已释放 ID 表。可用 ID 表用于存放尚未分配的 ID，而已释放 ID 表用于记录已完成任务或已释放数据块对应的 ID。

每个数据 ID 长度为 3 字节，系统最多支持  $2^{24}$ （约 1600 万）个唯一 ID。系统初始化时，可用 ID 表预填充全部可分配 ID，已释放 ID 表为空。

在任务或数据块创建时，运行时管理模块从可用 ID 表弹出一个 ID 并分配给新对象。该过程无需复杂查找或映射操作，在较低管理开销下保证 ID 唯一性。

### (2) ID 回收与复用机制

当数据块生命周期结束并释放相关资源后，系统将对对应 ID 回收并压入已释放 ID 表以供复用。

若运行过程中可用 ID 表被耗尽，系统将交换两张 ID 管理表的角色：原已释放 ID 表作为新的可用 ID 表，而原可用 ID 表被清空并作为新的已释放 ID 表。随后系统继续执行 ID 的分配与回收流程。

通过该机制，系统在有限 ID 空间内实现循环复用，避免 ID 随系统运行时间无限增长，同时保证任意时刻数据对象标识的唯一性。

### （3）ID 生命周期管理与系统优势

基于栈结构的 ID 分配与回收机制，使数据对象 ID 与其运行生命周期严格对应，实现显式且可控的生命周期管理。该设计结构简单、管理开销低，且不依赖复杂数据结构或动态内存管理，适用于 FPGA 资源受限的调度系统。

#### 4.1.2 元数据表结构设计

在完成张量块的统一标识之后，系统需要在运行过程中持续维护张量块的状态信息，以支撑调度器对数据可用性与数据迁移行为的判断。为此，本系统在调度层设计并维护块级元数据表，用于集中记录系统中已知张量块的关键信息。元数据表作为连接数据管理与调度决策的重要桥梁，其结构设计直接影响系统运行时的查询效率与实现复杂度。

元数据表以张量块 ID 作为索引键，每条记录对应一个逻辑张量块，主要包含以下几类信息：其一，数据块的唯一标识，用于在系统中区分不同张量块；其二，数据块当前的驻留位置，用于指示该数据是否已经存在于某一计算节点或调度层缓存中；其三，数据块的访问状态，用于反映该数据是否处于可读、更新中或不可用状态；其四，与副本管理相关的辅助信息，用于描述系统中是否存在多个数据副本及其基本情况；其五：数据块在调度层缓冲区的版本号以及物理地址，用于标识数据块在调度层的物理地址以及副本是否过期等信息。通过上述字段，调度器能够在不访问实际数据内容的情况下，对数据块的可用性和迁移需求进行快速判断。具体字段设计如表4-1所示：

考虑到调度层基于 FPGA 实现，元数据表采用定长结构化字段设计，以避免复杂动态数据结构带来的硬件开销。同时，所有查询与更新操作均以张量块 ID 为索引，通过简单查表即可完成，以降低控制逻辑复杂度并提高运行时确定性。

元数据表的更新由数据迁移、任务执行及数据回收等事件触发。当数据块被传输至新的计算节点或被任务访问时，调度层相应更新其驻留位置与状态信息，从而在系统范围内维持统一的数据状态视图。

表 4-1 元数据表字段设计

| 字段名称         | 位偏移 (Bit) | 宽度   | 类型  | 默认值   | 描述                                          |
|--------------|-----------|------|-----|-------|---------------------------------------------|
| Reserved     | 63–61     | 3 位  | –   | 3'b0  | 保留位                                         |
| location_map | 60–56     | 5 位  | r/w | 5'b0  | 位置映射字段，表示数据位置信息                             |
| Reserved     | 55–50     | 6 位  | –   | 6'b0  | 保留位                                         |
| status       | 49–48     | 2 位  | r/w | 2'b0  | 数据状态：00:INVALID; 01:INIT; 10:READY; 11:WAIT |
| version      | 47–40     | 8 位  | r/w | 8'h0  | 全局版本号，高位为标记位                                |
| data_size    | 39–32     | 8 位  | r/w | 8'h0  | 数据大小，单位为 KB                                 |
| base_addr    | 31–0      | 32 位 | r/w | 32'h0 | 数据起始地址，仅调度层映射有效时使用                          |

#### 4.1.3 数据驻留与副本管理策略

在多计算单元并行执行场景下，同一张量块可能被多个任务访问。为减少重复传输并提高数据复用效率，本系统允许数据块以驻留或副本形式存在。

当张量块首次被传输至某一计算节点后，该数据被视为驻留于该节点。若后续任务所需数据已驻留在目标节点，调度器将优先复用本地数据，从而避免额外的数据迁移。

对于副本生成，本系统采用受控副本策略：仅当目标节点不存在所需数据且数据迁移成本可接受时，调度器才触发新的数据传输操作，从而形成数据副本。副本数量上限及回收条件由调度器根据系统运行状态统一管理。

当数据块不再被后续任务访问时，主机端将其标记为可回收状态，并通过 FREE\_DATA 指令完成数据 ID 回收与数据释放。回收过程中调度层同步更新元数据，以保证数据状态一致性。具体一致性维护机制将在 4.3 节进一步讨论。

## 4.2 稀疏压缩与数据传输机制

在多计算单元并行执行的张量计算系统中，数据传输开销往往成为影响整体执行效率的重要因素。一方面，张量分块执行模式显著增加了系统中数据块的数量，使得跨节点数据迁移更加频繁；另一方面，实际计算任务中大量张量具有不同程度的稀疏特性，若直接以稠密形式进行传输，将造成带宽资源的浪费。因此，有必要在数据管理层面面对张量块进行统一的稀疏表示与高效传输。

本节围绕张量块在系统中的传输过程，介绍稀疏张量的统一表示方法、数据

序列化与传输流程，以及在调度层引入的环形缓冲区机制。通过上述设计，系统能够在保证数据正确性的前提下，降低跨节点通信开销，并实现数据传输与调度处理的解耦，为后续调度策略的稳定执行提供了基础支持。

#### 4.2.1 块稀疏张量的统一表示

在本系统中，张量以固定大小的数据块作为最小管理单元。考虑到张量计算任务中普遍存在的稀疏性特征，系统在数据传输阶段采用块级稀疏表示方式，以减少不必要的数据传输量。具体而言，系统采用块压缩稀疏行 (Block Compressed Sparse Row, BCSR) 格式对张量块进行表示。具体压缩格式如图4-2所示：

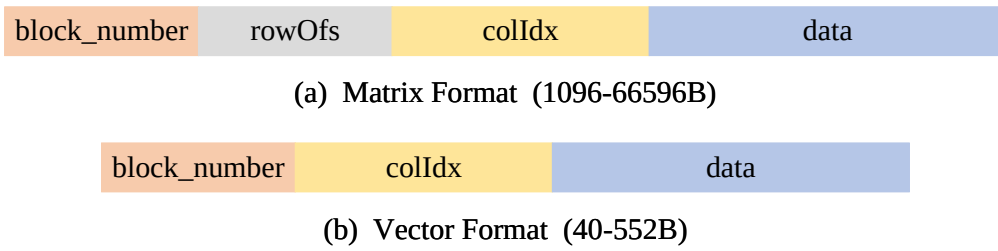


图 4-2 BCSR 压缩示意图

该图展示了 BCSR 下两种块表示形式的结构与尺寸特征：图4-2 (a) 所示矩阵格式包含非零块数量 (block\_number)、行偏移 (rowOfs)、列索引 (colIdx) 与数据 (data) 模块；图4-2 (b) 所示向量格式则省略 rowOfs，仅保留 block\_number、colIdx 与 data。二者均以减少数据传输量为核心目标。

BCSR 以子块为基本单元进行索引与存储。本系统将每个  $128 \times 128$  张量块进一步划分为  $4 \times 4$  子块，从而在保持较好压缩效果的同时，与固定块粒度的数据管理和调度机制保持一致。与元素级稀疏表示相比，块级稀疏可避免传输与解析阶段的粒度转换开销，简化调度层处理逻辑。

从概率角度分析，当张量元素以稀疏度  $\rho$  随机分布时，任意  $s \times s$  子块至少包含一个非零元素的概率为：

$$P_{nz}(s, \rho) = 1 - (1 - \rho)^{s^2} \quad (4-1)$$

由式(4-1)可知，随着子块尺寸  $s$  增大，即使  $\rho$  较低，非零子块概率也会快速上升，导致块级稀疏表示逐渐退化为接近稠密形式，压缩收益显著下降。因此子块尺寸需在索引开销与压缩有效性间进行权衡。

实现中，每个非零子块通过所属张量块的 Block ID 与稀疏存储项关联，确保稀疏数据在传输、缓存与接收过程中仍能统一标识与管理。本系统选择 BCSR 主要因其结构简单、索引明确，能以较低控制开销在 FPGA 上实现。

本系统采用 BCSR 的主要目的是降低数据传输开销，而非针对稀疏计算本身优化。因此未引入复杂压缩或重排策略，以保持简洁性与可扩展性。

块级稀疏表示主要适用于具有显著稀疏或结构化稀疏特征的张量，例如科学计算中的稀疏线性系统、图计算邻接矩阵，以及经剪枝处理的神经网络权重。在这些场景下，张量块中存在大量全零或近零子块，提供良好压缩空间。对于高密度张量，BCSR 将退化为带有额外索引开销的稠密形式，其传输优势显著减弱。因此本系统针对实际应用中广泛存在的大规模稀疏张量进行针对性优化。

#### 4.2.2 数据的序列化与传输

在完成张量块的稀疏表示后，数据需要从主机或计算节点传输至调度层，以支持后续调度与执行。为此，系统设计统一的数据序列化与传输机制，将张量块的结构信息与数值数据封装为可传输的数据单元。

在发送端，系统首先对张量块的元数据与数据内容进行处理。元数据包括 Block ID 等必要状态信息；数据内容为稀疏压缩后的数值数据及结构信息。通过优先解析元数据，调度层能够在完整数据到达前更新数据状态或触发后续处理。

从传输量角度分析，设张量块大小为  $B \times B$ ，子块大小为  $s \times s$ ，则子块总数：

$$N = \left(\frac{B}{s}\right)^2 \quad (4-2)$$

设式(4-2)中非零子块数量为  $k$ ，单个元素占用字节数为  $w$ ，则 BCSR 表示下单个张量块的传输数据量可表示为：

$$T_{\text{BCSR}} = S_{\text{idx}} + k \cdot s^2 \cdot w \quad (4-3)$$

式(4-3)中  $S_{\text{idx}}$  为索引区的固定开销，包括非零块数量、行偏移与列索引信息。相比之下，稠密表示下的传输数据量为：

$$T_{\text{RAW}} = B^2 \cdot w \quad (4-4)$$

当  $k \ll N$  时，BCSR 表示能够显著减少传输数据量；当非零子块比例较高时，索引开销会削弱其优势。该模型刻画了稀疏结构与传输效率之间的关系，并为后续实验分析提供理论依据。

序列化后的数据在物理布局上由索引区与数据区顺序组成，但二者在系统中作为整体进行存储与传输。索引区采用固定结构，如表4-2所示，包括非零块数量、行偏移及列索引信息；数据区则存储对应非零块的数值数据，其大小随稀疏度变化。最终存储大小为索引区与数据区之和。

在序列化实现上，各组成区域均按照 64bit 边界进行对齐，以保证数据在跨节点传输过程中的结构清晰性与访问一致性。该策略避免了对字节级精确寻址的依赖，有助于简化序列化与反序列化逻辑，提高系统实现的可维护性与稳定性。

表 4-2 序列化结构示意图

| 区域名称 | 内容说明      | 大小      | 对齐方式     |
|------|-----------|---------|----------|
| 索引区  | 非零块数量     | 8B      | 64bit 对齐 |
| 索引区  | 行偏移信息     | 64B     | 64bit 对齐 |
| 索引区  | 列索引信息     | 1KB     | 64bit 对齐 |
| 数据区  | 非零块数据     | 0B-64KB | 64bit 对齐 |
| 合计   | 索引区 + 数据区 | 可变      | 统一对齐     |

该统一序列化机制同样适用于向量数据的表示与传输。对于向量而言，其序列化格式与张量块保持一致，但由于向量仅包含单一行结构，因此不包含行偏移信息，其索引区相应地简化为非零元素数量与列索引描述，从而在保持接口与处理流程一致性的同时，减少了不必要的结构开销。

数据传输过程采用异步方式进行，发送端在完成数据单元的序列化后即可发起传输请求，而无需等待调度层的即时响应。该设计使数据传输能够与计算任务并行执行，从而有效降低通信延迟对系统整体吞吐性能的影响。

4.2.3 基于环形缓冲区的数据接收与缓存机制

在多源数据并发到达的场景下，调度层需要同时接收来自主机与多个计算节点的数据块。由于数据到达时间具有不确定性，且调度器的处理能力受限，若在数据到达时立即进行处理，容易导致调度层阻塞或资源竞争。为此，系统在调度层引入了一种基于环形缓冲区的统一数据接收与缓存机制，用于解耦数据到达过程与调度处理过程。其工作机制如图4-3所示。

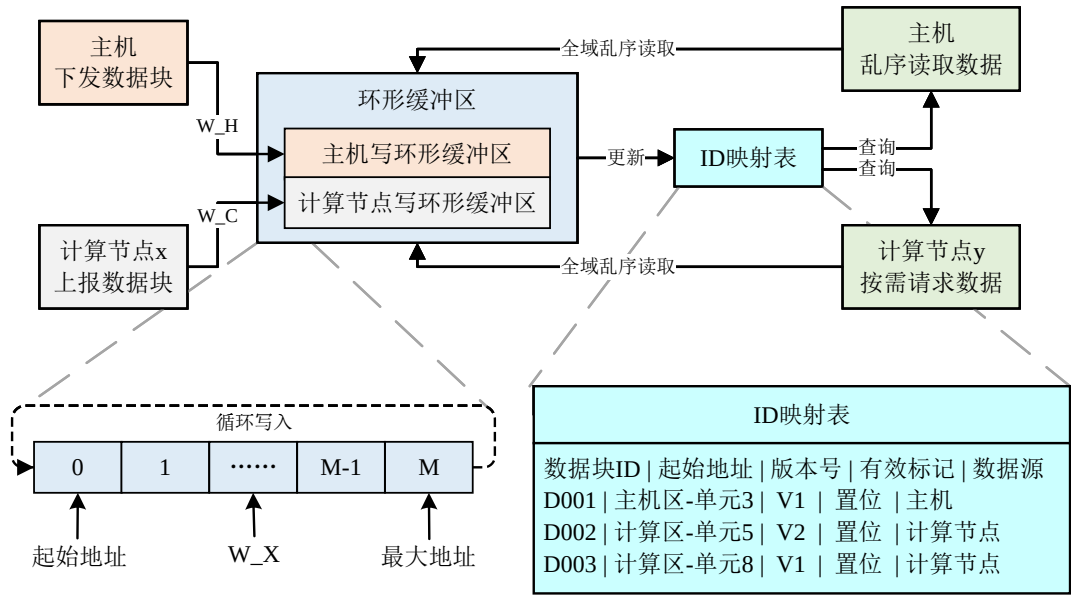


图 4-3 环形缓冲区机制示意图

与传统 FIFO 缓冲结构不同, 该环形缓冲区在逻辑上划分为两个独立写入区域, 分别用于主机与计算节点的数据写入。主机侧区域用于接收主机下发的数据块, 计算节点侧区域用于接收各计算节点回传的数据。数据写入完成后, 系统通过 ID 映射表记录数据块在缓冲区中的起始地址、版本号及有效标记, 以支持后续模块按需访问。

环形缓冲区采用顺序写入与循环覆盖策略。当写指针到达缓冲区末尾时自动回绕至起始位置继续写入, 实现空间复用。写入阶段主机与计算节点分别使用各自的写入区域以避免冲突; 读取阶段, 各模块可访问整个缓冲区, 但具体访问位置由 ID 映射表统一管理。

在读取机制上, 调度层通过 ID 映射表定位数据地址与长度并按需读取, 而非按写入顺序消费数据。因此, 该结构本质上是一种顺序写入、随机访问的循环存储结构, 而非传统 FIFO 队列。

该设计实现了数据写入与调度处理的解耦, 使调度器能够按照自身处理节奏访问已到达的数据块, 而不受数据到达顺序或突发写入行为影响。数据版本号的更新与一致性控制将在后续一致性模型中进一步讨论。

#### 4.2.4 数据迁移流程与 DMA 实现

在完成张量块数据的接收与缓存后, 调度层需要根据任务执行需求, 将尚未驻留于目标计算节点的数据块迁移至对应的计算节点。该过程发生于系统运行期间, 是调度决策驱动的数据再分布行为, 与主机侧数据下发过程相互独立。

数据迁移流程由调度层统一触发, 当调度器判定某一任务所需的输入张量块尚未驻留于目标计算节点时, 将依据元数据表中记录的块位置与状态信息, 生成相应的数据迁移请求。该请求由 DMA 控制模块执行, 实现张量块数据从源节点经调度层缓冲区到目标计算节点的直接传输。为避免数据迁移过程对任务调度与执行形成阻塞依赖, DMA 操作采用异步方式进行, 其完成状态通过事件或标志位反馈至调度层, 用于驱动后续调度决策。

在跨节点数据迁移过程中, 系统对元数据更新与 DMA 操作的执行顺序进行了严格约定。具体而言, 调度层仅在确认对应 DMA 传输完成后, 才会更新张量块在元数据表中的驻留状态与版本信息, 从而保证调度决策始终基于真实的数据分布情况。

通过将数据传输与状态更新解耦, 并以完成事件作为状态转换的唯一触发条件, 系统避免了因状态提前更新而导致的数据不一致或潜在死锁问题。相关的一致性维护策略与无死锁调度机制将在第 4.3 节中进一步讨论。

### 4.3 数据一致性与正确性保障机制设计

在多计算节点并行执行的张量计算系统中，数据一致性是保证任务正确执行的基础。由于系统支持张量块在主机、调度层以及多个计算节点之间的动态流动与复制，若缺乏统一的一致性约束，极易出现读写冲突、副本失效或调度决策基于错误数据状态的问题。为此，本系统在调度层引入集中式的一致性控制机制，通过明确的数据块状态机、元数据驱动的副本管理策略以及对调度层缓存有效性的判定，构建了一套简化但完备的数据一致性模型。

#### 4.3.1 一致性模型定义

本系统采用一种由调度层集中管理的 SWMR 一致性模型。在该模型下，任意时刻每个张量块最多只允许一个写者，同时可被多个任务以只读方式访问。所有对数据块的访问权限均由调度层通过元数据状态进行统一裁决，而不依赖分布式一致性协议或节点间协商。

从生命周期角度看，一个张量块在系统中的状态演化可抽象为若干离散阶段，包括初始化 (INIT)、就绪 (READY)、传输中 (WAIT) 以及无效 (INVALID)。其更细粒度的逻辑生命周期状态如图4-4所示：

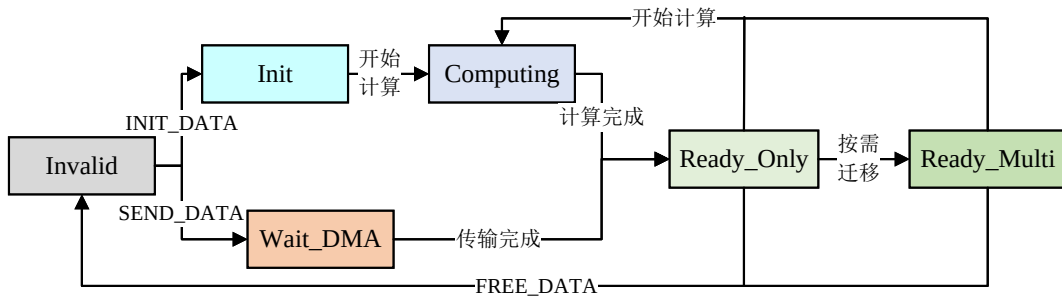


图 4-4 张量块生命周期及副本演化的逻辑状态机示意图

系统以逻辑生命周期状态机的形式刻画了张量块在初始化、传输、计算、副本扩散及释放等阶段的状态演化过程。其中，Wait\_DMA 与 Computing 分别表示数据块处于 DMA 传输和计算生成过程中的不可访问阶段；Ready\_Only 与 Ready\_Multi 用于区分数据块在系统中仅存在单一副本或存在多个一致副本的逻辑情形。该状态机用于描述数据一致性语义及副本演化规律，而非直接对应具体实现中的元数据状态字段。

在具体实现中，为降低元数据表的存储与控制开销，系统并未为图中每一个逻辑状态分别维护独立的实现状态，而是将其压缩映射为有限数量的实现态，用于快速判定数据块的可访问性及其参与调度的条件。逻辑状态与实现状态之间的对应关系如表4-3所示：

当初始数据由主机下发至调度层时，系统在元数据表中为对应张量块建立条

表 4-3 逻辑生命周期状态与实现状态的映射关系

| 逻辑状态（图4-4）  | 实现状态（元数据表） | 数据访问语义 |
|-------------|------------|--------|
| Invalid     | INVALID    | 不可访问   |
| Init        | INIT       | 不可访问   |
| Computing   | INIT       | 不可访问   |
| Wait_DMA    | WAIT       | 不可访问   |
| Ready_Only  | READY      | 可读     |
| Ready_Multi | READY      | 可读     |

目，并根据数据来源（如 INIT\_DATA 或 SEND\_DATA）进入 Init 或 Wait\_DMA 状态，同时更新元数据状态为 INIT 或 WAIT。待 DMA 传输完成后，状态由 Wait\_DMA 转为 Ready\_Only，并建立调度层位置映射。

在运行过程中，调度器可将处于 Ready\_Only 状态的张量块下发至一个或多个计算节点，从而形成多个一致副本并进入 Ready\_Multi 状态。对于只读任务，多个副本可被并行访问而不会产生一致性问题。

当任务需要对张量块执行写操作时，系统进入 Computing 状态，将元数据状态更新为 INIT，同时清除除目标计算节点外的所有副本，包括调度层缓存中的副本。在计算完成前，该数据块不允许被其他任务访问。写入完成后，调度层确认结果并恢复为 Ready\_Only 状态，此时系统仅保留写任务生成的唯一有效副本。

当调度器接收到 FREE\_DATA 指令时，将释放该张量块在调度层及各计算节点上的所有副本，并将元数据状态更新为 INVALID，从而结束其生命周期。通过上述状态机与副本管理策略，系统能够明确维护每个张量块的访问语义，避免并发读写冲突。

#### 4.3.2 元数据驱动的副本管理与状态更新机制

为支撑前述一致性模型与数据块生命周期状态机，系统在调度层维护一张集中式元数据表，即表4-1，用于记录每个张量块的全局状态及其副本分布。每条元数据项包含实现状态标志、位置映射信息以及与调度层缓存相关的辅助字段。调度器在任务调度、数据迁移及一致性判定时均以该表作为唯一可信信息源。

在副本生成与迁移过程中，系统对元数据更新与数据传输的顺序进行严格约束。无论数据来自主机还是计算节点，调度层仅在 DMA 传输完成后才更新张量块状态并建立位置映射。在跨节点迁移时亦遵循同样原则，即在传输完成前不会提前更新目标节点的驻留状态，以避免调度决策基于未就绪数据。

写操作是副本管理的关键环节。当调度器为任务授予写权限时，将通过元数据状态标记该数据块处于写入生成阶段，并清除除目标计算节点外的所有副本位

置映射，包括调度层缓存中的副本。在写入完成并经调度层确认之前，其他任务对该数据块的访问均不被允许。

由于所有状态更新与副本管理均由调度层统一触发，系统无需依赖分布式锁或节点间协商。该元数据驱动机制在支持多副本并行读取的同时保证写入排他性，并在写入完成后收敛为唯一有效副本，从而实现 SWMR 一致性模型。

#### 4.3.3 调度层缓存一致性与无死锁保证

除计算节点上的数据副本外，调度层环形缓冲区中暂存的数据同样纳入一致性管理范围。由于环形缓冲区采用顺序写入与循环覆盖的空间复用方式，仅依赖写入地址难以判断缓存数据是否已被覆盖。因此，本系统在调度层引入版本号机制，用于辅助判定缓存副本的有效性。图4-5给出了环形缓冲区在连续写入与回绕过程中的逻辑划分示意。

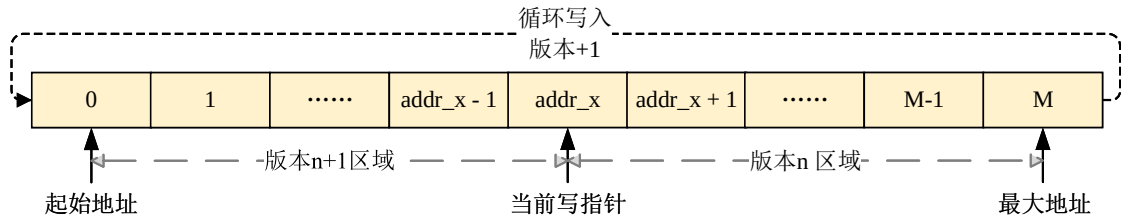


图 4-5 环形缓冲区版本号与地址判定示意图

如图4-5所示，环形缓冲区在逻辑上划分为多个连续版本区域。当写指针完成一次完整回绕时，缓冲区版本号递增。对于版本号为  $n$  的数据块，其有效地址位于对应版本区域内。当当前版本仍为  $n$  时，说明数据尚未被覆盖；当版本号变为  $n+1$  时，需要结合记录地址与当前写指针位置判断是否发生覆盖；若版本差值大于 1，则可直接判定该缓存副本已失效。

通过版本号与地址联合判定，系统无需显式维护缓存失效状态即可判断副本有效性，从而避免读取已被覆盖的数据。需要注意的是，调度层缓存映射标记仅表示数据曾写入缓冲区，其实际可用性仍由版本号与地址判定结果决定。

在一致性控制方面，系统采用集中式状态管理。任务不会主动请求多个资源，而由调度器在调度阶段统一检查所需数据块状态；若数据处于写入生成阶段，则延迟调度而非进入阻塞等待。由于数据状态遵循单向生命周期状态机，且不存在任务间循环依赖，系统能够从机制上避免死锁与活锁。

综上，通过基于版本号的缓存有效性判定与集中式状态管理，系统在多节点环境下实现了可靠的数据一致性保证，同时保持较低的实现复杂度与运行开销。

## 4.4 数据管理实验与分析

本节针对所提出的数据驻留机制、稀疏传输方法及 SWMR 一致性模型，从数据本地性、带宽利用效率以及一致性正确性三个维度进行系统评估，以验证其在实际运行场景中的性能收益与语义正确性。

### 4.4.1 数据本地性与跨节点传输开销分析

在固定任务执行轨迹（round-robin, 4 节点）的条件下，本节比较不同数据驻留与副本复用策略对跨节点数据搬运行为的影响。通过保持执行节点序列一致，仅改变数据管理策略，从而隔离调度因素并评估数据位置管理与副本复用机制对数据本地性及传输开销的影响。

#### （1）实验目标与对照策略

为避免调度策略差异对结论造成干扰，实验固定每个任务的执行节点序列，仅考察数据驻留与副本策略对搬运路径的影响。对比策略如下：

- 1) G1(No-Residency): 节点侧不驻留输入数据，任务访问缺失触发搬运；
- 2) G2(Single-Copy Residency): 系统内每个数据块至多保留一个有效副本，新节点访问时触发迁移并使旧副本失效；
- 3) G3(Multi-Copy Reuse): 允许数据在多个节点保留副本以提高命中率，节点缓存采用容量上限与 FIFO 淘汰策略。

在 G3 基础上进一步引入调度层复用强度参数 `sched_factor`。为便于对比，将 G3 划分为两种配置：

- 1) G3(s0): `sched_factor=0`;
- 2) G3(tuned): 在搬运字节数不超过最优值 5% 的条件下，选择最小 `sched_factor` 作为 tuned 配置。

实验仅统计任务输入 A/B 的搬运，不计输出 C 以及控制事件，以聚焦输入复用带来的跨节点传输变化。

#### （2）实验工作负载

任务按照 round-robin 在 4 个节点间分配，每个任务访问两个输入块 (A、B)，其 ID 从工作集 (working set) 中采样。访问模式包括 Uniform（均匀随机）与 Zipf（长尾热点）两类分布。通过工作集大小  $W$  控制复用强度，并以归一化指标  $ws/cap = W/capacity$  对实验结果进行分桶统计。

#### （3）统计指标

实验统计以下指标用于刻画跨节点搬运行为：

- `bytes_per_task`: 平均任务触发的数据搬运字节数；
- `miss_breakdown`: 缺失访问路径组成 (remote hit、scheduler hit、cold fetch)；

uplink\_ratio: uplink 在总搬运字节中的比例;

peak\_replica\_blocks: 系统运行过程中副本数量峰值。

这些指标均基于调度层元数据查询与事件记录获得, 能够直接反映数据驻留策略对传输路径与代价的影响。

#### (4) 结果与分析

##### 1) 不同驻留策略的搬运代价对比

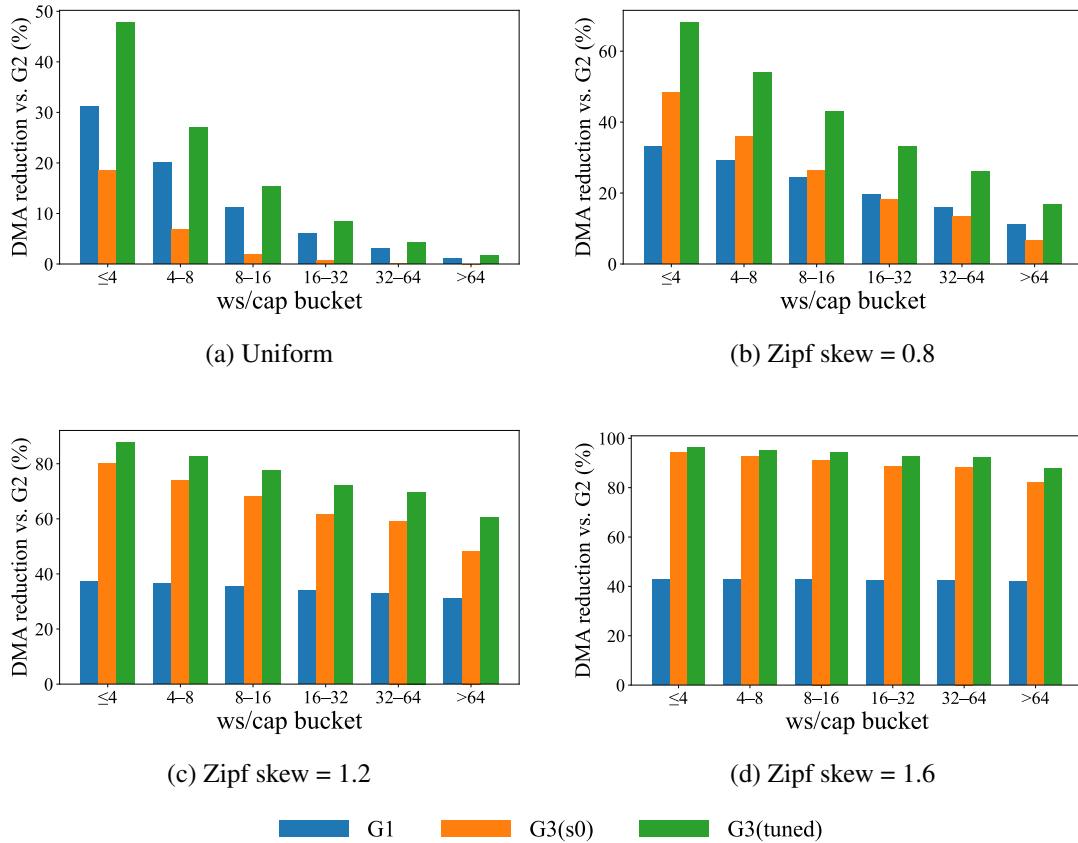


图 4-6 不同驻留与副本策略下的任务平均数据搬运代价对比 (以 G2 为基准)

表 4-4 不同工作负载下任务平均搬运代价对比 (中位数)

| Workload | ws/cap | G2 (KB) | G3(tuned) (KB) | Reduction |
|----------|--------|---------|----------------|-----------|
| Uniform  | ≤ 4    | 399     | 208            | 47.8%     |
| Zipf 0.8 | ≤ 4    | 446     | 142            | 68.1%     |
| Zipf 1.2 | ≤ 4    | 515     | 63             | 87.7%     |
| Zipf 1.6 | ≤ 4    | 590     | 22             | 96.2%     |

图4-6 给出了在 Uniform 与 Zipf 不同偏斜度下, 各策略在不同 ws/cap 分桶中的 bytes\_per\_task 相对降幅 (以 G2 为基准), 表4-4 给出了高复用场景 (ws/cap 4) 下的代表性绝对搬运代价。

实验结果表明，在 Zipf 热点场景中，G3(tuned) 可显著降低跨节点数据搬运，skew 1.2 时约 60-96%，表明多副本机制有效提升了数据复用率。

在 Uniform 场景中，G3 的收益随 ws/cap 增大迅速收敛，由约 47.8% 降至约 1.6%，表明在接近一次性访问模式下，多副本复用收益有限。

此外，G1 在部分场景下的较低搬运开销来源于以 cold fetch 替代跨节点回传，而非数据复用带来的优化。

## 2) 缺失访问组成与 uplink 压力分析

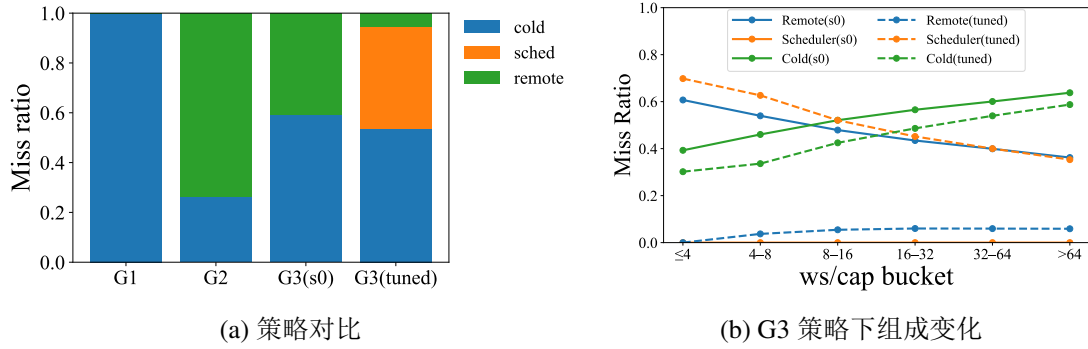


图 4-7 Zipf skew = 1.2 条件下缺失访问路径组成分析

图4-7 展示了 Zipf skew = 1.2 条件下缺失访问路径的组成差异。不同策略呈现出明显不同的访问特征：G1 的缺失几乎全部由 cold fetch 构成，G2 主要来源于 remote hit，而 G3 通过多副本机制显著降低了 remote hit 比例。进一步地，G3(tuned) 引入调度层复用，使部分跨节点访问在调度层被吸收，从而减少对远端回传的依赖。

结合图4-6 与表4-4 可知，尽管 G3(tuned) 在部分场景下 cold\_fetch\_ratio 略有上升，其整体搬运开销仍显著低于 G3(s0) 与 G2，表明性能提升主要来源于远端回传与重复传输的减少。

从 ws/cap 变化趋势看，G3(s0) 的缺失访问始终由 remote hit 与 cold fetch 构成，且随着工作集增大逐渐向 cold fetch 转移。相比之下，G3(tuned) 在小 ws/cap 区间内以 scheduler hit 为主，而随着工作集增大，调度层缓存逐渐饱和，缺失访问逐步转化为 cold fetch。同时，其 remote\_hit\_ratio 始终维持较低水平，进一步降低了数据迁移与 uplink 压力。

## 3) sched\_factor 扫描与代价-收益权衡

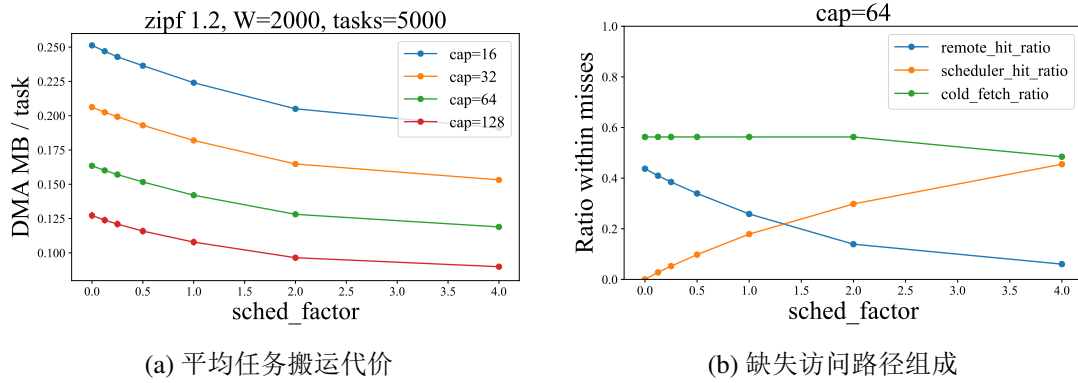


图 4-8 G3 中 sched\_factor 的影响

图4-8 给出了 Zipf skew = 1.2 条件下 sched\_factor 的扫描结果。如图4-8(a), 随着 sched\_factor 增大, bytes\_per\_task 单调下降但边际收益递减, 且节点侧缓存容量越大, 其对调度层复用的敏感性越低。

如图4-8(b), 从缺失访问组成来看, sched\_factor 增大使 scheduler\_hit\_ratio 上升、remote\_hit\_ratio 下降, 而 cold\_fetch\_ratio 基本保持稳定, 表明调度层缓存主要通过吸收跨节点复用流量降低回传开销。

表4-5 给出了基于“最优搬运字节 5% 容忍范围”的 sched\_factor 推荐结果。对于 Zipf(skew 1.2) 的中高复用场景, 多数分桶推荐较大的 sched\_factor (通常为 4); 而在 Uniform 且 ws/cap 较大的场景中, 推荐值逐步下降至 0, 表明此时调度层复用收益有限, 应弱化或关闭该机制。

表 4-5 基于搬运代价容忍准则的 sched\_factor 推荐配置

| workload | ws/cap | n_scen | rec. sf | workload | ws/cap | n_scen | rec. sf |
|----------|--------|--------|---------|----------|--------|--------|---------|
| uniform  | ≤4     | 2      | 4       | zipf 0.8 | ≤4     | 2      | 4       |
| uniform  | 4-8    | 4      | 4       | zipf 0.8 | 4-8    | 4      | 4       |
| uniform  | 8-16   | 6      | 4       | zipf 0.8 | 8-16   | 6      | 4       |
| uniform  | 16-32  | 8      | 2       | zipf 0.8 | 16-32  | 8      | 4       |
| uniform  | 32-64  | 8      | 0       | zipf 0.8 | 32-64  | 8      | 4       |
| uniform  | >64    | 12     | 0       | zipf 0.8 | >64    | 12     | 2       |
| zipf 1.2 | ≤4     | 2      | 4       | zipf 1.6 | ≤4     | 2      | 1       |
| zipf 1.2 | 4-8    | 4      | 4       | zipf 1.6 | 4-8    | 4      | 4       |
| zipf 1.2 | 8-16   | 6      | 4       | zipf 1.6 | 8-16   | 6      | 4       |
| zipf 1.2 | 16-32  | 8      | 4       | zipf 1.6 | 16-32  | 8      | 4       |
| zipf 1.2 | 32-64  | 8      | 4       | zipf 1.6 | 32-64  | 8      | 4       |
| zipf 1.2 | >64    | 12     | 4       | zipf 1.6 | >64    | 12     | 4       |

#### 4) 存储开销权衡

此外, 实验统计还记录了系统运行过程中副本数量的峰值 (peak\_replica\_blocks),

用于反映副本策略带来的存储开销上界。结果表明，G3(tuned) 通常通过增加副本数量来换取显著的数据搬运削减。

例如，在 Zipf skew = 1.2 且  $ws/cap \leq 4$  的场景中，副本峰值由 G2 的约 410 增加至 G3(tuned) 的 512，而 bytes\_per\_task 的中位数降幅约为 87.7%。该结果体现了典型的“在跨节点传输中以存储换带宽”的权衡关系。

#### (5) 一致性验证

为验证硬件模块 data\_movement 与 Python 行为级参考模型在数据驻留判定与跨节点搬运路径选择上的一致性，本文采用“同一驱动向量、同一判据对比”的验证方法。在固定 round-robin (4 节点) 执行轨迹下，参考模型根据工作负载生成逐任务驱动向量，包含 best\_node 及输入 A/B 对应的 move\_valid、move\_id、move\_loc 等字段，并输出期望的搬运统计与事件序列。

参考模型遵循 G1/G2/G3 的语义定义：本地命中不触发搬运；未命中时触发下发传输 (Scheduler → 目标节点)；仅当存在远端副本时，触发回传 (远端节点 → Scheduler) 与下发的组合路径。在 G3 中引入调度层缓存，命中时仅发生下发传输，缓存容量由  $sched\_factor \times capacity$  控制。

硬件验证中，Testbench 使用相同驱动向量驱动 data\_movement 模块，并从 send2r\* (下发) 与 get\* (回传) 端口采集 RTL 事件流。

一致性验证采用两级判据：1) 计数一致性：比较各通道 send2r0..3 与 get0..3 的事件计数；2) 事件流一致性：比较事件序列是否逐条一致。事件以“通道名 + 数据块 ID”表示，不要求周期对齐，但要求顺序与内容一致，从而保证关键调度决策与参考模型一致。

在 post-implementation 时序仿真下对代表性配置 (如 G3、Zipf skew = 1.2、 $W = 2000$ 、 $capacity = 64$ 、 $sched\_factor = 0.25$ ) 进行验证。RTL 输出的 send2r/get 事件计数 (send: 415/410/401/389, get: 252/159/113/81) 与参考模型完全一致，且事件序列逐条匹配。结果表明，硬件实现与参考模型在搬运控制路径上保持一致，为后续性能指标统计提供了可靠性保障。

#### (6) 结果分析

综合上述实验结果，多副本驻留与调度层复用在高热点、高复用场景中能够带来稳定且显著的性能收益。在 Zipf 分布 (尤其 skew  $\geq 1.2$ ) 且  $ws/cap$  较小的分桶内，G3(tuned) 相较于 G2 显著降低了 bytes\_per\_task，其主要来源于缺失访问路径的重构：跨节点回传 (remote\_hit 与 uplink) 减少，而调度层命中 (scheduler\_hit) 增加，从而降低重复搬运开销。

然而，该收益以更高的副本开销为代价。G3(tuned) 通常对应更大的 peak\_replica\_blocks，且在接近一次性访问模式 (如 Uniform 或  $ws/cap$  较大) 下，其边

际收益迅速下降,此时缺失访问主要转化为 cold\_fetch,增大 sched\_factor 难以进一步降低搬运代价。

因此,对于以张量计算及深度学习训练/推理为代表的高热点、高复用负载,采用 G3(tuned) 的“多副本驻留 + 受控调度层复用”策略更为合适;而在复用较弱的场景中,应在接近最优搬运代价的容忍范围内选择最小 sched\_factor,以控制调度层缓存开销并避免无效资源占用。

#### 4.4.2 稀疏率对数据传输带宽利用的影响

为验证第 4.2 节提出的稀疏压缩传输机制在数据通路上的有效性,本节以 XDMA 主机到 FPGA 的单通道写传输为对象,系统评估在不同稀疏模式与稀疏率条件下,稠密表示 (RAW) 与块压缩稀疏表示 (BCSR) 在传输时间与带宽利用率方面的差异。

##### (1) 实验设置与数据集构造

实验采用参数化矩阵生成与传输流程:矩阵边长设为

$$\text{size} = k \times \text{BIG\_BLOCK}, \quad (4-5)$$

并在多个  $k$ 、稀疏率 density,以及块划分参数 BIG\_BLOCK / SMALL\_BLOCK 组合上进行扫描。其中, BIG\_BLOCK 表示张量块大小, SMALL\_BLOCK 表示块内子块大小。PCIe 实际可用带宽经测试约为 3.1 GB/s。

为覆盖不同类型的稀疏特征,本节构造两类代表性工作负载:

1) 随机元素级稀疏 (Random): 以给定元素密度生成随机稀疏矩阵,并转换为稠密数组存储;

2) 结构化块级稀疏 (BSR): 以 BIG\_BLOCK 为粒度,以概率 block\_prob = density 随机选择非零大块,并在被选中的大块内填充稠密随机值。该构造使得整体元素密度与 density 同阶,同时呈现显著的结构化稀疏特征。

在表示方式上, RAW 采用逐元素稠密存储; BCSR 则首先按 BIG\_BLOCK 对矩阵进行分块,并在每个大块内以 SMALL\_BLOCK 为粒度扫描子块。若子块包含至少一个非零元素,则记录其列索引并追加对应子块数据。BCSR 的二进制布局由固定头部、行指针、列索引及数据区组成,并进行 16B 对齐打包。详细字段定义与布局方式见第 4.2 节相关内容。

##### (2) 测量方法与评价指标

传输通过 XDMA 工具在 h2c\_0 通道执行主机到设备的写操作,并从工具输出中解析 “written in Xs” 得到单次传输耗时。每组参数配置重复多次取平均值 (本实验设定 REPEAT = 10)。

在此基础上定义时间加速比为：

$$\text{Speedup} = \frac{t_{\text{RAW}}}{t_{\text{BCSR}}}, \quad (4-6)$$

并分别对 Random 与 BSR 两类数据集得到 speedup\_random 与 speedup\_bsr。

同时，引入带宽相关指标：

$$\text{BW} = \frac{\text{bytes}}{t}, \quad \text{BW}_{\text{eq}} = \frac{\text{raw\_bytes}}{t_{\text{BCSR}}}, \quad (4-7)$$

式(4-7)中  $\text{BW}_{\text{eq}}$  表示将压缩收益折算回原始张量规模后的等效带宽，即在保持原始张量规模不变的前提下，采用 BCSR 压缩传输所对应的等效 PCIe 传输带宽，用于刻画稀疏压缩对 PCIe-FPGA 通道整体吞吐能力的提升效果。

### (3) 实验结果与讨论

表4-6 与表4-7 分别汇总了在固定矩阵规模  $k = 64$ 、块大小  $\text{BIG\_BLOCK} = 128$ 、子块大小  $\text{SMALL\_BLOCK} = 4$  的代表性参数配置下，不同稀疏率条件下 RAW 与 BCSR（随机稀疏与结构化稀疏）在传输数据量、实际传输时间、压缩率、时间加速比及等效带宽等关键指标的实验结果，其中 RAW 表示未压缩张量传输，其数据量固定为 268.44 MB，平均传输时间约为 84 ms。表中列出了 BCSR 表示在不同稀疏率下的实际结果，为后续趋势分析提供总体参考。

表 4-6 不同稀疏率下 BCSR 的传输数据量与传输时间

| density | BCSR (MB) <sub>rand</sub> | BCSR (MB) <sub>bsr</sub> | BCSR (ms) <sub>rand</sub> | BCSR (ms) <sub>bsr</sub> |
|---------|---------------------------|--------------------------|---------------------------|--------------------------|
| 0.005   | 21.47                     | 1.79                     | 7.123                     | 0.724                    |
| 0.01    | 40.94                     | 3.92                     | 13.186                    | 1.240                    |
| 0.02    | 75.74                     | 6.32                     | 24.122                    | 2.105                    |
| 0.05    | 153.01                    | 15.77                    | 48.643                    | 4.718                    |
| 0.1     | 222.50                    | 27.22                    | 69.961                    | 8.920                    |
| 0.2     | 265.38                    | 54.31                    | 82.307                    | 16.798                   |
| 0.5     | 273.02                    | 137.24                   | 85.441                    | 43.472                   |
| 1.0     | 273.02                    | 273.02                   | 86.156                    | 85.480                   |

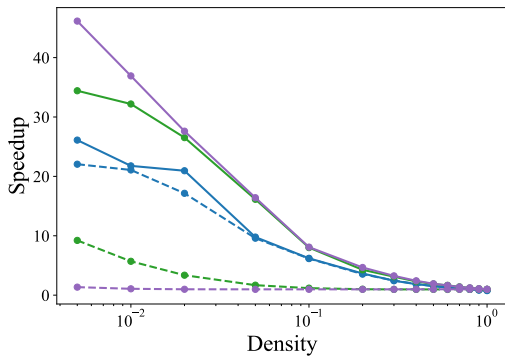
#### 1) 加速比随稀疏率变化趋势

图4-9(a) - (d) 分别给出了在不同矩阵规模 ( $k = 32, 64$ ) 与不同块尺度  $\text{BIG\_BLOCK} = 32, 128$  条件下，传输加速比随稀疏率变化的趋势。

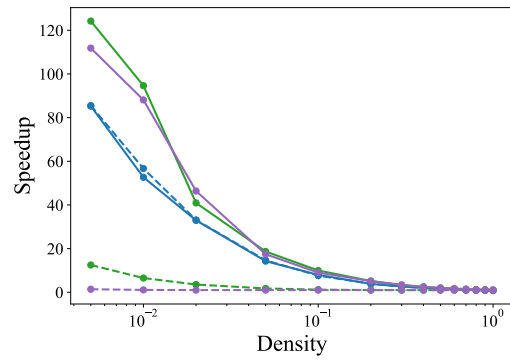
结果表明，对于随机稀疏，传输加速比随稀疏率升高迅速下降至 1，而结构化块级稀疏在低中密度下仍保持高加速比。增大  $\text{BIG\_BLOCK}$  可进一步提高结构化稀疏在低密度区域的加速效果，而随机稀疏提升有限。

表 4-7 不同稀疏率下 BCSR 的压缩率、时间加速比与等效带宽

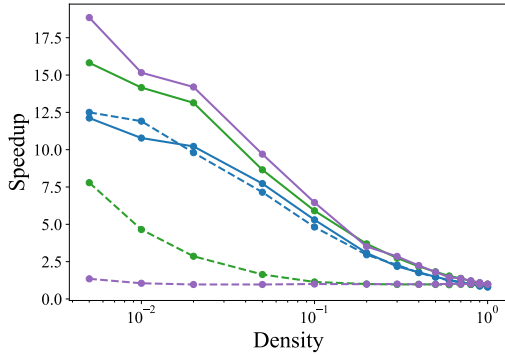
| density | CR <sub>rand</sub> | CR <sub>bsr</sub> | Speedup <sub>rand</sub> | Speedup <sub>bsr</sub> | BW <sub>eq,bsr</sub> (GB/s) |
|---------|--------------------|-------------------|-------------------------|------------------------|-----------------------------|
| 0.005   | 12.50              | 149.88            | 11.8                    | 116.94                 | 370.77                      |
| 0.01    | 6.56               | 68.46             | 6.32                    | 68.09                  | 216.48                      |
| 0.02    | 3.54               | 42.49             | 3.48                    | 40.05                  | 127.52                      |
| 0.05    | 1.75               | 17.02             | 1.74                    | 17.93                  | 56.90                       |
| 0.1     | 1.21               | 9.86              | 1.18                    | 9.43                   | 30.09                       |
| 0.2     | 1.01               | 4.94              | 1.03                    | 5.00                   | 15.98                       |
| 0.5     | 0.98               | 1.96              | 0.99                    | 1.95                   | 6.17                        |
| 1.0     | 0.98               | 0.98              | 0.97                    | 0.98                   | 3.14                        |



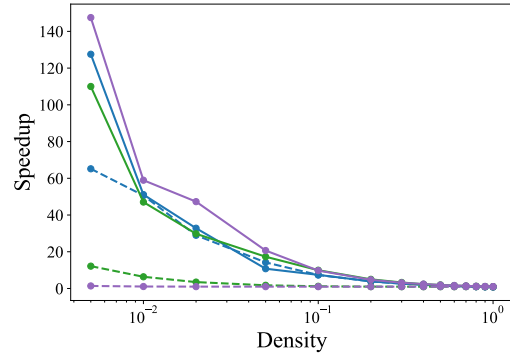
(a) k=64, BIG\_BLOCK=32



(b) k=64, BIG\_BLOCK=128



(c) k=32, BIG\_BLOCK=32



(d) k=32, BIG\_BLOCK=128

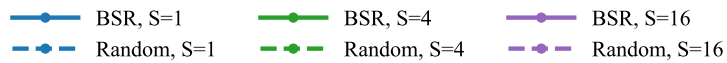


图 4-9 不同块划分参数下 Random 与 BSR 的传输加速比对比

## 2) 理论加速比与实际加速比的对比

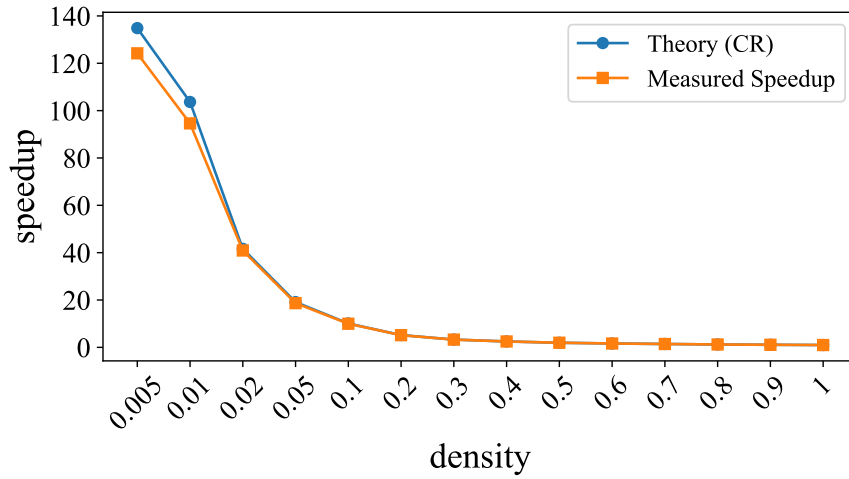


图 4-10 BSR 下理论压缩加速比与实际传输加速比对比

图4-10 对比了结构化稀疏 (BSR) 在不同稀疏率条件下，由压缩率 CR 推导得到的理论加速比与实际测得的时间加速比。

可以看出，在极低稀疏率（如 density=0.005、0.01）条件下，理论加速比明显高于实际测量值；随着稀疏率升高，二者差距逐步缩小，并在中等及较高密度区域趋于一致。

这一差异可由如下传输模型解释：

$$t \approx t_0 + \frac{\text{bytes}}{\text{BW}}, \quad (4-8)$$

式(4-8)中  $t_0$  表示 XDMA 传输中的固定启动与调度开销。当压缩后数据量较小时，该固定开销在总传输时间中占比较高，从而削弱压缩带来的时间收益；随着数据量增大，该开销被逐步摊薄，实际加速比逐渐接近理论预测值。

### 3) 块参数对加速比的影响

图4-11(a) 与 (b) 分别给出了在 density=0.01 与 density=0.2 条件下，结构化稀疏在不同 BIG\_BLOCK 与 SMALL\_BLOCK 组合下的传输加速比分布情况。

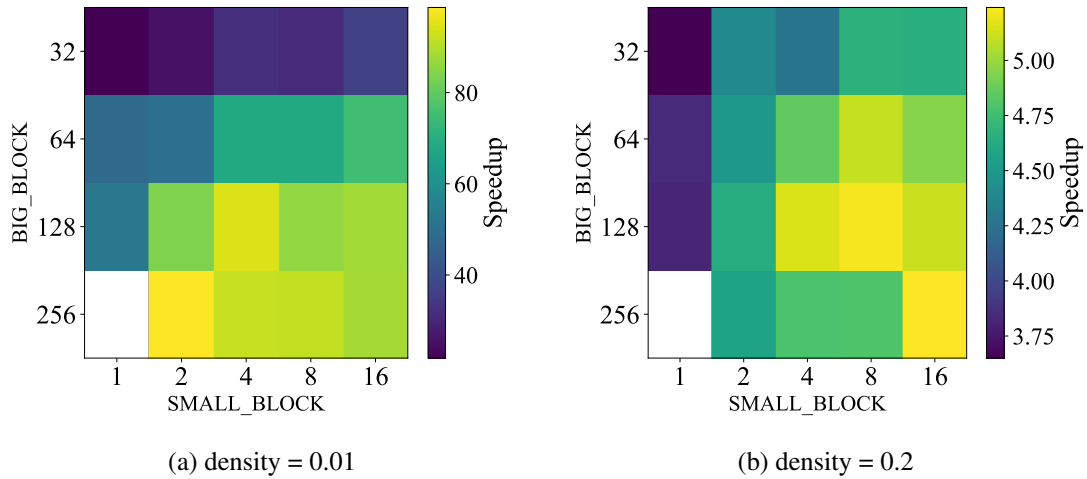


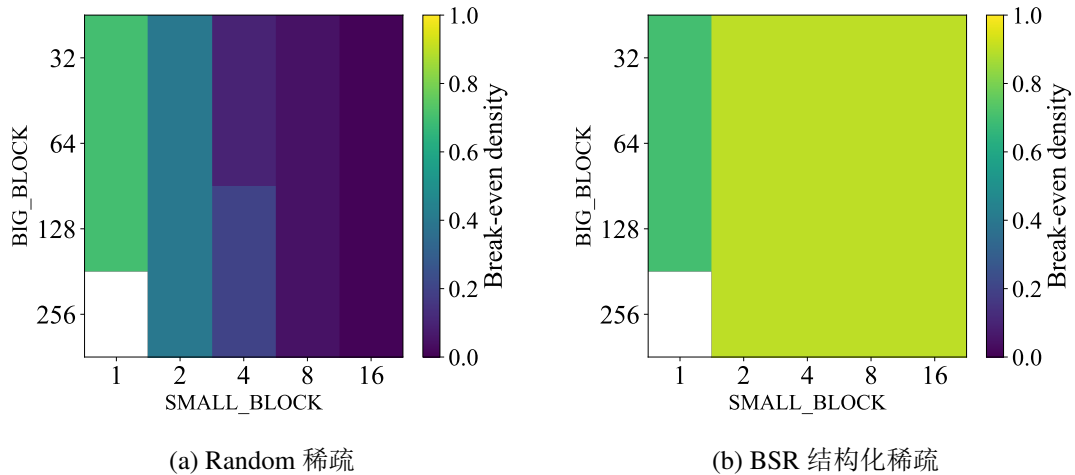
图 4-11 不同稀疏率下 BCSR 块参数对传输加速比的影响

在低密度与中等密度条件下，最优块参数区域具有较强的一致性；对于较大的 BIG\_BLOCK，适中的 SMALL\_BLOCK（如 4 或 8）通常能够在零子块捕捉能力与索引开销之间取得较好平衡。过小的 SMALL\_BLOCK 会显著增加子块数量与索引开销，而过大的 SMALL\_BLOCK 则可能降低零块捕捉效率。

#### (4) 适用性讨论

综合上述实验结果可知，在当前 PCIe-FPGA 单通道 XDMA 传输环境下，BCSR 在具有显著稀疏或结构化稀疏特征、且数据规模较大的张量工作负载中，能够有效降低传输数据量并显著提升等效带宽利用率。

图4-12(a)(b) 分别给出了在不同块参数组合下，随机稀疏 (Random) 与结构化块级稀疏 (BSR) 对应的、可实现正加速的稀疏率适用范围 (Break-even density)。

图 4-12 不同稀疏类型下块参数组合对正加速稀疏率范围的影响 ( $k = 64$ )

可以看到，随机稀疏的正加速区域对块参数高度敏感，在多数参数组合下仅

在较低稀疏率区间内才能保持正加速；而结构化稀疏在几乎所有块参数组合下，其正加速适用稀疏率均稳定维持在较高水平，表现出显著更强的参数鲁棒性。

该结果表明，相比随机稀疏，结构化块级稀疏在实际工程场景中能够在更宽的稀疏率区间内稳定获益，对块参数选择不敏感，更适合作为通用的数据压缩与传输优化手段。

#### 4.4.3 SWMR 一致性与副本管理正确性验证

##### (1) 实验目的

为验证第 4.3 节提出的集中式 SWMR 一致性模型及元数据驱动的副本管理机制，本节构建仿真环境，对多节点系统中的关键一致性性质进行检查，包括：多副本只读一致性、单写者唯一性、缓存过期保护以及系统可收敛性。

##### (2) 实验平台与配置

实验采用 Vivado 仿真环境，针对被测模块 `data_movement` 与 `ring_buffer` 分别构建 Verilog 测试平台。其中，一致性与副本管理验证平台 (CASE1~CASE3) 面向 SWMR 语义及副本迁移/释放行为，系统配置为 4 个计算节点 (NODES=4)，张量块 ID 空间大小为 1024 (ID\_SPACE=1024)，每个数据块的校验数据宽度为 64 bit (DATA\_W=64)。

测试平台在 `ready` 信号上引入随机抖动，并设置多级超时与排空判定机制 (如 `TIMEOUT_CYC`、`DRAIN_HARDMAX` 等)，以暴露潜在的边界错误或死锁风险。该验证平台后文简称为平台 A，其关键仿真参数及含义汇总如表 4-8 所示。

表 4-8 仿真验证平台关键参数设置

| 参数            | 取值    | 含义                            |
|---------------|-------|-------------------------------|
| NODES         | 4     | 计算节点数量                        |
| ID_SPACE      | 1024  | 数据块 ID 范围 (1..1023)           |
| DATA_W        | 64    | 校验数据宽度 (bit)                  |
| FUZZ_OPS      | 200   | 随机操作次数                        |
| READY_PCT     | 85%   | <code>ready</code> 抖动概率       |
| TIMEOUT_CYC   | 4000  | 等待引擎稳定/完成超时                   |
| DRAIN_HARDMAX | 40000 | <code>drain</code> 排空硬超时      |
| STABLE_CYC    | 32    | 判定写入结束的稳定周期                   |
| QUIET_NEED    | 12    | <code>drain</code> 结束所需连续静默周期 |

回绕覆写与过期判定验证平台 (CASE4) 面向调度层环形缓冲区的过期读取保护 (stale safety)。该平台仅启用单通道 (如 `fifo0`)，并选取一组代表性 ID (0~31) 构造连续写入与回绕后读出场景，以便在波形与日志中清晰定位“回绕发生—过期判定—有效读取”的关键过程，后文简称为平台 B。

### (3) 参考模型与判定方法

平台 A 在调度层侧维护参考模型 (golden model), 用于刻画每个数据块的最新版本数据及其在各节点间的副本分布状态, 并基于对 DUT 事件输出的实时监测进行一致性判定。其整体结构与判定流程如图4-13 所示。

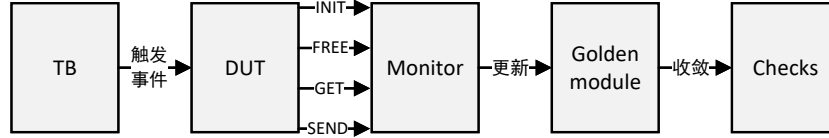


图 4-13 仿真验证平台结构与判定流程示意图

参考模型主要维护以下两类核心状态：

- 1) 最新版本数据 `latest[id]`：当测试平台触发写入生成场景时生成，用于区分新旧版本数据；
- 2) 节点侧副本状态 `rep_valid[node][id]` 与 `rep_data[node][id]`：分别表示节点是否持有该 ID 的有效副本及其对应数据内容，用于一致性校验。

为获取副本迁移与生命周期事件，测试平台对 `data_movement` 模块输出的多个 FIFO 读端口进行监测。由于 FIFO 输出可能一次携带两个 ID（按 `{16'h0, id1, 16'h0, id0}` 打包），monitor 在解包 `rd_data` 后更新参考模型。主要观测事件包括 INIT、FREE、GET 以及 `send2r` 等，用于刻画副本生成、释放、迁移及环形路径相关调度行为。

在判定策略上，平台实现以下三类检查机制：

- 1) 广播一致性检查 (`broadcast_check`)：对指定 ID 比较观测副本掩码与期望掩码，并校验所有有效副本数据均等于 `latest[id]`；
- 2) 迁移依赖检查 (`assert_move_get`)：当检测到环形路径调度事件且存在可用源副本时，必须观测到对应 GET 事件；
- 3) 排空与稳定性检查：在 `busy` 释放后等待内部 FIFO 计数稳定并最终排空，用于检验系统在随机场景下不会进入不可收敛状态。

平台 B 采用“连续写入触发回绕 + 延迟后读出”的确定性流程，并结合版本有效性信号与数据校验进行判定，其核心判据包括版本有效性判定、有效数据逐拍校验以及无效条目保护机制。

### (4) 实验用例设计与结果

为覆盖第 4.3 节提出的核心一致性语义与关键状态跃迁，本实验结合定向测试用例与随机 fuzz 测试进行验证，重点检查多副本一致性、单写者唯一性以及系统在随机操作条件下的可收敛性。

#### 1) 副本一致性与单写者语义验证

首先构造数据块在多个节点存在一致副本的场景，发起 move 请求后系统应通过 GET 事件从源节点回传数据，并通过 SEND 传输使目标节点获得副本。其关键波形如图4-14 所示。

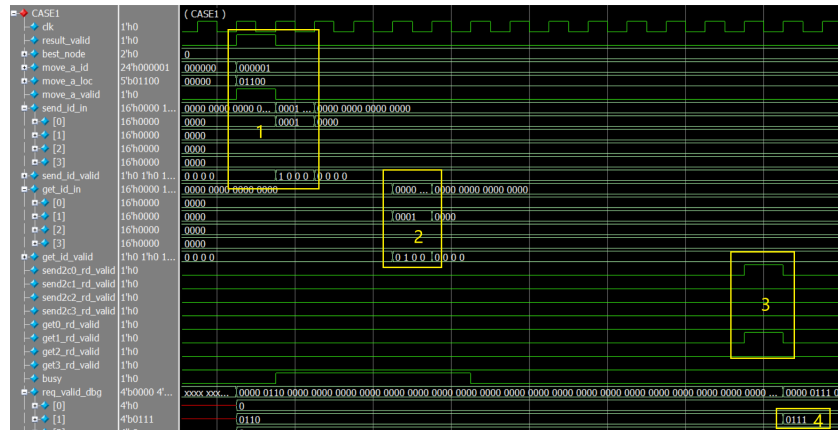


图 4-14 副本迁移过程关键波形

当 result\_valid 与 move\_a\_id 有效时，DUT 根据输入的 best\_node 与数据位置信息触发调度，系统首先从存在副本的节点产生 GET 事件，随后向目标节点发起 SEND 传输，使目标节点获得一致副本。monitor 根据事件流更新参考模型并进行一致性校验，副本掩码由 0110 变为 0111，表明目标节点成功获得副本且所有副本数据保持一致。

进一步构造写入生成场景，验证单写者唯一性语义。当 result\_valid 与 init\_c\_id 同时有效时，目标节点触发 INIT 事件生成新版本数据，调度层同时对其余节点上的旧副本触发 FREE 操作。关键波形如图4-15 所示。

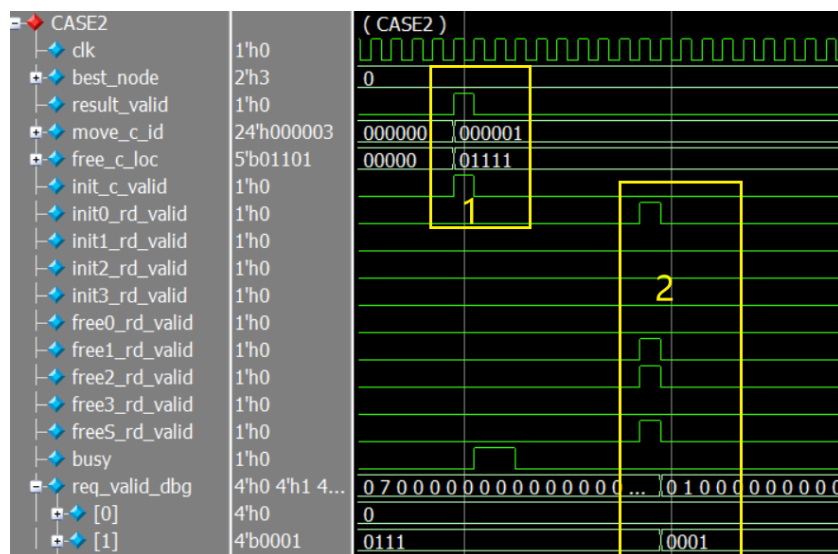


图 4-15 写入生成与旧副本失效波形

仿真结果表明，写入完成后系统状态逐步收敛为仅目标节点持有该数据块的

唯一有效副本，且所有读取数据均与参考模型中的 `latest[id]` 保持一致，验证了单写者唯一性语义的正确性。

此外，在 `ready` 抖动条件下随机生成操作序列 (`FUZZ_OPS=200`)，覆盖 `move-only`、`write-only` 以及混合操作三种运行模式。仿真过程中未出现超时、永久 `busy` 或队列无法排空现象，表明在随机操作条件下系统能够稳定运行并最终收敛。

## 2) 调度层环形缓冲区过期判定验证

为验证第 4.3.3 节提出的“版本号 + 写指针”联合判定机制，构造连续写入触发环形缓冲区回绕的典型场景，使早期写入的数据地址区间被后续写入覆盖。随后对部分早期写入的数据块发起读取请求，以检查系统是否能够正确识别过期缓存条目。

其关键读出阶段波形如图 4-16 所示。

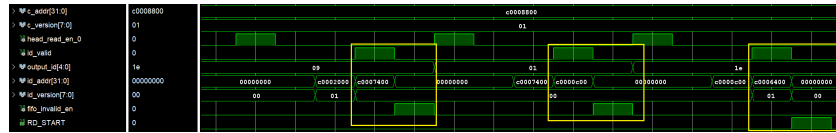


图 4-16 缓存版本有效性判定波形

当读取请求命中旧版本缓存条目时，系统根据缓存版本信息将其判定为无效，并通过 `fifo_invalid_en` 信号记录对应 ID，从而避免读取已被覆写的数据。当命中新版本或仍处于有效区间的条目时，DUT 输出 `RD_START` 信号并产生正常的的数据流输出。

仿真结果表明，旧版本缓存条目能够被正确识别并拒绝读取，而有效条目能够正常完成数据读取并通过一致性校验，验证了缓存过期判定机制的正确性。

## (5) 小结

实验结果表明，所提出的 **SWMR** 一致性模型及元数据驱动的副本管理机制能够在多节点环境下正确维护副本一致性与生命周期管理。

在副本迁移与写入生成场景中，系统能够保证单写者语义并保持副本数据一致；在随机操作与背压扰动条件下，系统未出现死锁或不可收敛状态。同时，调度层环形缓冲区通过版本号与写指针联合判定，能够有效识别并避免读取被覆写的数据，从而保证缓存机制在提升访问效率的同时不破坏系统一致性。

## 第五章 多因素任务调度策略与实现

在多 FPGA 计算节点协同执行的张量计算系统中，任务调度不仅需要考虑计算负载均衡问题，还必须同时兼顾数据本地性、节点内存资源以及跨节点数据迁移开销等多种约束因素。在缺乏成熟操作系统与运行时支持的 FPGA 环境下，调度策略的设计不仅影响系统性能，且关系到系统的可实现性与运行稳定性。

在前一章中，本文围绕数据管理与一致性控制机制，对张量块的驻留、副本管理以及跨节点数据迁移过程进行了系统设计与实验验证，为调度决策提供了可靠的数据状态与代价评估基础。在此基础上，本章进一步面向多 FPGA 计算单元环境，构建一种基于 DAG 全局依赖的协同调度模型：由主机端负责维护张量计算任务的全局依赖关系并生成就绪任务流，由 FPGA 调度层针对就绪任务进行多计算节点的调度决策。

不同于以理论最优或复杂优化模型为目标的调度算法研究，本文关注在现实硬件资源受限条件下的工程可行性。调度决策以数据迁移代价为核心依据，并综合考虑节点队列负载与内存压力，通过一系列启发式约束与判定机制，在保证调度逻辑简单、时序可控的前提下，实现数据本地性优先与负载均衡之间的折中。

本章首先对张量计算进行 DAG 依赖关系构建，然后对多加速器环境下的调度问题进行建模，明确调度目标与约束条件；并给出多因素调度决策模型的设计思路，详细说明其在 FPGA 调度器中的实现方式；最后通过实验分析，对所提出调度策略在数据本地性利用、负载均衡以及整体调度效果方面的表现进行验证。

### 5.1 面向多计算单元的调度问题建模

本文采用“主机端建模、调度层决策、计算节点执行”的分层设计思想，将复杂的任务依赖分析与调度决策过程进行功能划分。在该框架下，主机端负责构建张量计算任务的全局依赖关系并生成就绪任务流，调度层仅对已满足依赖条件的任务进行调度决策，从而显著降低调度问题在 FPGA 侧的复杂度。

#### 5.1.1 基于 DAG 的就绪任务模型

在张量计算任务中，算子之间通常存在明确的数据依赖关系，例如矩阵乘、卷积等算子之间存在输入输出依赖。本文在主机端使用 DAG 对计算任务进行建模，其中图中的节点表示具体的计算任务，边表示任务之间的数据依赖关系。通

过 DAG 模型，可以清晰描述任务的执行顺序及其对输入数据的依赖条件。主机根据计算图结构、算子依赖以及数据可用性，动态维护任务的就绪状态。

具体而言，主机端负责对张量计算任务进行分块与算子展开，并构建对应的 DAG 结构。图中的节点表示可调度的计算任务，边表示任务之间的数据依赖关系。当某一任务的所有前驱任务均已完成，且其所需输入数据已处于可访问状态时，该任务被判定为就绪任务 (ready task)，并由主机下发至调度层。

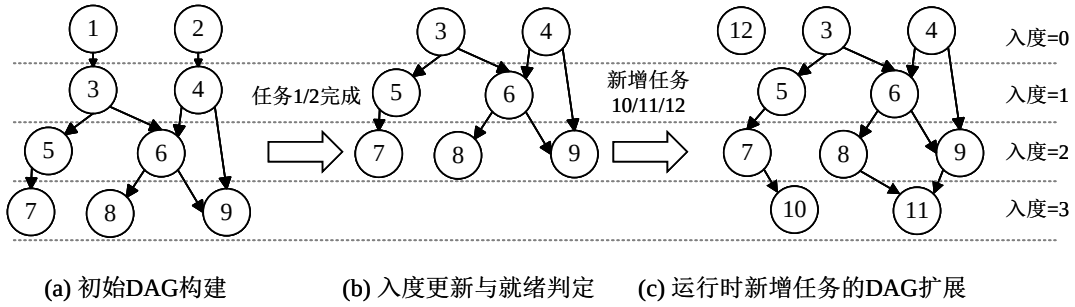


图 5-1 基于 DAG 的任务依赖维护与就绪任务生成示意图

如图5-1所示，系统在主机端维护任务依赖 DAG，并通过入度信息判定任务的就绪状态。当任务执行完成后，其后继节点的入度被更新，入度为零的任务被判定为就绪并发送至调度层。在系统运行过程中，新的计算任务亦可动态加入 DAG，其依赖关系与已有任务统一维护，从而支持张量计算任务的动态调度需求。调度层并不维护完整的 DAG 结构，也不参与依赖解析过程，而是仅接收来自主机端的就绪任务集合。

基于上述设计，调度问题可被抽象为：在给定一组就绪任务的前提下，依据各计算节点的当前状态与数据分布情况，为每个任务选择合适的执行节点。该建模方式在保持任务依赖语义正确性的同时，显著简化了调度器的实现复杂度，避免了在 FPGA 侧实现复杂的图遍历与依赖更新逻辑，符合资源受限 FPGA 平台的工程实际并提高了系统的可实现性与运行稳定性。

需要指出的是，将 DAG 构建与依赖管理放在主机端并非功能上的妥协，而是一种基于工程可行性的设计取舍。复杂控制逻辑更适合在软件环境中实现，而 FPGA 侧则更适合承担高吞吐、低控制复杂度的调度与执行任务。该分工方式符合软硬件协同系统的设计原则，也为后续系统扩展提供了良好的灵活性。

### 5.1.2 调度目标与约束

在上述就绪任务模型的基础上，调度层需要根据系统当前状态为每个就绪任务选择合适的计算节点。由于系统同时受到计算资源、数据分布以及存储资源等多方面因素的影响，调度目标并非单一指标的优化，而是在多种约束条件下实现综合性能的折中。

首先,从优化目标角度出发,调度策略应优先降低跨节点数据迁移开销。由于张量计算任务通常伴随较大规模的数据访问,频繁的数据搬移将显著增加通信开销并降低整体执行效率。因此,在调度决策中,应将数据本地性作为核心考量因素,尽量将任务分配至已有输入数据驻留的计算节点。

其次,调度过程需要兼顾各计算节点的负载状况。为避免部分节点过载而其他节点空闲的情况,系统引入节点队列长度或队列时间作为负载近似指标,在调度决策中对节点计算压力进行评估。通过在数据本地性与节点负载之间进行权衡,调度器能够在保证数据复用的同时,维持系统整体的执行并行度。

再次,内存资源构成调度决策的关键约束条件之一。由于 FPGA 计算节点的片上或板级存储资源有限,任务调度必须确保目标节点具备足够的可用内存以容纳任务执行所需的数据。本文在调度模型中将内存约束分为两类:一类为调度可行性的硬约束,当节点剩余内存不足时,相关节点直接被排除在候选集合之外;另一类为影响调度倾向的软约束,通过对数据迁移代价进行加权,间接反映节点内存压力对调度决策的影响。

此外,调度过程还需满足系统一致性与实现层面的工程约束。调度器必须遵循第四章所定义的数据一致性模型,避免在数据处于写入或不可访问状态时触发任务调度。同时,考虑到 FPGA 平台在资源与时序方面的限制,调度决策逻辑需保持结构简单,避免引入复杂的控制流、迭代优化或高代价运算,以确保调度过程能够在有限时钟周期内完成。

综合上述目标与约束,本文将调度问题建模为一个在多重工程约束条件下的启发式决策过程,不追求理论意义上的全局最优解,而是在保证系统可实现性与运行稳定性的前提下,实现数据本地性、负载均衡与资源利用之间的合理折中。

最后,本章所建模的调度问题并非全局任务调度,而是在主机端已完成任务依赖解析的前提下,对就绪任务进行执行节点选择的工程调度问题。通过这种分层建模方式,系统在保持调度语义正确性的同时,有效控制了 FPGA 调度器的实现复杂度,为后续多因素调度策略的设计与实现提供了清晰的问题边界。

## 5.2 多因素调度决策模型设计

在上一节完成了面向多计算单元的 DAG 任务调度问题建模之后,调度器已具备对任务依赖关系、就绪条件以及基本调度约束的形式化描述。然而,仅依赖任务就绪性进行调度,难以充分发挥多节点 FPGA 系统的硬件潜力,尤其是在存在数据分布不均、节点负载差异以及内存资源受限等现实条件下。

为此,本节进一步从数据本地性、节点负载状态以及内存资源约束等多个维度出发,构建面向工程实现的多因素调度决策模型。该模型以数据迁移代价为核

心度量, 引入对节点计算负载与内存压力的量化描述, 并在此基础上设计轻量级的综合调度决策机制, 在保证调度效果的同时兼顾 FPGA 实现复杂度。

### 5.2.1 数据本地性评分模型

数据本地性是影响多节点张量计算性能的关键因素之一。在多节点 SoC 或 NoC 架构中, 任务执行节点与数据驻留节点之间的空间距离直接决定了数据访问延迟。当任务所需数据已驻留在计算节点本地时, 可避免跨节点数据迁移, 从而显著降低任务启动延迟和执行开销; 反之, 若数据分布在调度层缓冲区或远端计算节点, 则不可避免地产生额外的数据搬运开销。

为此, 本文引入数据迁移代价 (Move Cost) 作为数据本地性的量化指标, 并将其作为调度决策的重要依据。该代价由调度层根据张量块的当前驻留位置以及预估的数据传输时间综合计算得到。

#### (1) 通用 NoC/NUMA 数据迁移代价模型

在 NoC、NUMA 以及多节点 SoC 系统中, 数据搬运延迟通常可以拆分为以下三个部分:

1) 启动与调度开销 (Setup Latency): 包括 DMA 配置、缓存一致性维护以及调度层与计算节点之间的控制交互, 该部分与数据规模无关。2) 路径相关延迟 (Hop Latency): 由数据在 NoC 中经过的路由跳数 (hop) 决定, 反映节点之间的物理或逻辑距离。3) 数据传输延迟 (Transfer Latency): 由链路带宽和数据规模共同决定, 通常与数据大小呈线性关系。

因此, 数据迁移代价的普适性计算模型可表示为:

$$Cost_{move} = L_{base} + H \times L_{hop} + Size \times C_{bw} \quad (5-1)$$

式(5-1)中:  $L_{base}$  表示固定启动延迟,  $H$  表示数据源节点与目标节点之间的 hop 数,  $L_{hop}$  表示单 hop 的路由延迟,  $Size$  表示待迁移数据规模,  $C_{bw}$  表示单位数据量的传输代价, 与 NoC 带宽相关。该模型在保证表达简洁的同时, 能够较好地刻画多节点系统中数据迁移的主要开销来源, 因而具有良好的通用性。

#### (2) 面向本系统的数据迁移代价模型

在本文所设计的系统中, 调度层对数据驻留位置进行了抽象划分, 数据仅可能处于以下三种状态之一: 1) 本地节点 (Local): 数据已驻留在目标计算节点本地存储中, 无需迁移; 2) 调度层缓冲区 (Scheduler Buffer): 数据位于调度层共享缓冲区, 与计算节点之间存在一次 NoC 传输; 3) 远端计算节点 (Remote Node): 数据驻留在其他计算节点, 需要跨节点传输。基于上述划分, 本文将节点间距离离散化为三个等级, 并用参数  $seg$  表示:  $seg = 0$ : 本地数据;  $seg = 1$ : 调度层缓冲区;  $seg = 2$ : 远端节点。

在此基础上, 结合公式(5-1), 可将数据迁移代价简化为以下形式:

$$Cost_{move} = \begin{cases} 0, & seg = 0, \\ L_{base} + seg \times L_{hop} + seg \times Size \times C_{bw}, & seg \neq 0. \end{cases} \quad (5-2)$$

式(5-2)中,  $L_{base}$  为 DMA 与调度启动代价,  $L_{hop}$  为跨层或跨节点的路径惩罚,  $C_{bw}$  为传输单位数据的代价,  $seg$  表示节点间距离。

该模型在保持线性复杂度的同时, 引入了对节点距离的刻画, 能够较真实地反映集中式调度下多节点架构的数据迁移行为。具有以下合理性特征:

1) 符合实际硬件行为: 启动代价、路径延迟和带宽开销分别对应真实系统中的 DMA 配置、NoC 路由以及链路带宽限制。2) 对小数据与大数据均有效: 对于小规模数据, 启动代价和 hop 延迟占主导; 对于大规模张量块, 带宽项成为主要瓶颈。3) 实现复杂度低: 模型采用线性表达形式, 便于在调度层进行快速估算, 适合硬件或轻量级固件实现。

在调度决策阶段, 调度器针对每个候选计算节点计算其对应的数据迁移代价, 并将其作为节点选择的主导因素之一。调度器优先选择数据迁移代价较小的节点执行任务, 从而: 提高数据本地命中率; 减少数据带宽压力; 降低整体任务启动与执行延迟。通过该策略, 系统能够在全局范围内减少不必要的数据搬运, 提升多节点张量计算的整体性能。

### 5.2.2 节点负载与内存压力建模

除数据本地性外, 计算节点的当前负载状态与可用内存资源同样是影响调度决策的重要因素。若调度器忽略节点负载情况, 可能导致任务在高负载节点排队等待, 从而增加整体执行延迟; 若忽略内存压力, 则可能引发数据无法放置或频繁回收的问题, 影响系统稳定性。

#### (1) 节点负载建模

本文使用节点任务队列的预完成时间作为节点负载的近似度量, 用以反映节点当前的计算繁忙程度。对于第  $i$  个计算节点, 其负载可表示为:

$$Q_i = \sum_{k \in \mathcal{T}_i} T_k \quad (5-3)$$

式(5-3)中,  $\mathcal{T}_i$  表示当前已调度至节点  $i$  但尚未完成的任务集合,  $T_k$  表示任务  $k$  的预估执行时间。

在实际系统中, 为避免复杂的队列建模与精确执行时间预测, 调度层维护一个累计的节点预完成时间计数器, 并在调度决策阶段将其作为节点负载指标。该建模方式在保证计算复杂度较低的同时, 能够保持节点负载的单调性, 满足调度比较需求。

## (2) 内存压力建模

FPGA 计算节点的片上与片外存储资源有限，内存占用情况直接影响任务是否能够被安全调度。本文对内存资源采用软约束与硬约束相结合的建模方式。

1) 内存硬约束：即可行性判定，当节点剩余内存不足以满足任务所需最小数据空间时，该节点应被直接排除在调度候选范围之外。对于节点  $i$ ，其内存可行性约束可表示为：

$$M_i^{free} \geq \alpha \cdot M^{task} \quad (5-4)$$

式(5-4)中： $M_i^{free}$ ：节点  $i$  的可用内存； $M^{task}$ ：任务所需内存； $\alpha$ ：冗余因子。

当上述条件不满足时，节点  $i$  被判定为不可调度节点。

## 2) 内存软约束（代价加权）

当节点内存满足基本可行性要求但剩余空间较少时，继续调度新任务可能导致系统后续调度空间受限。为此，本文引入内存压力系数  $k_i$ ，用于对数据迁移代价进行加权，其定义如下：

$$k_i = \begin{cases} 1, & \frac{M_i^{free}}{M_i^{max}} \geq \theta_1, \\ 1.5, & \theta_2 \leq \frac{M_i^{free}}{M_i^{max}} < \theta_1, \\ 2, & \frac{M_i^{free}}{M_i^{max}} < \theta_2. \end{cases} \quad (5-5)$$

式(5-5)中， $M_i^{max}$  表示节点  $i$  的最大内存容量， $M_i^{free}$  表示节点  $i$  的可用内存， $\theta_1$  与  $\theta_2$  为内存压力分段阈值。

加权后的有效数据迁移代价定义为：

$$Cost_{move,i}^{eff} = k_i \cdot Cost_{move,i} \quad (5-6)$$

该建模方式在不引入复杂内存预测模型的前提下，使调度器在决策过程中对内存紧张节点产生抑制作用，从而有效提升整体调度的稳定性与鲁棒性。

### 5.2.3 综合调度决策与迁移约束机制

在获得各计算节点的数据迁移代价、负载状态与内存可行性信息后，本文设计了一种以数据本地性为主导、负载均衡为补偿的启发式调度决策机制。该机制避免了全局优化或迭代搜索，适合在 FPGA 上以低延迟实现。

#### (1) 基准节点选择

调度器首先在所有满足内存可行性约束的节点中，选择数据迁移代价最小的节点作为基准节点：

$$i_{base} = \arg \min_{i \in V} Cost_{move,i}^{eff} \quad (5-7)$$

式(5-7)中,  $V$  表示满足内存硬约束的节点集合。

基准节点代表在当前调度条件下数据本地性最优的执行位置。

### (2) 负载差与迁移代价补偿判定

对于任意候选节点  $j$ , 其相对于基准节点的额外数据迁移代价定义为:

$$\Delta C_j = \max \left( 0, Cost_{move,j}^{eff} - Cost_{move,i_{base}}^{eff} \right) \quad (5-8)$$

同时, 节点负载差定义为:

$$\Delta Q_j = \max \left( 0, Q_{i_{base}} - Q_j \right) \quad (5-9)$$

为抑制频繁的数据迁移行为, 仅当节点  $j$  的负载优势足以补偿其额外数据迁移代价时, 才允许其参与调度候选。本文引入迁移判定阈值:

$$\Delta Q_j \geq \beta \cdot \Delta C_j \quad (5-10)$$

式(5-10)中,  $\beta$  为比例系数, 用于控制数据本地性与负载均衡之间的权衡关系。

### (3) 综合调度代价函数

在满足内存可行性与迁移约束条件的节点集合中, 调度器采用如下综合代价函数进行最终选择:

$$Cost_i^{total} = Q_i + Cost_{move,i}^{eff} \quad (5-11)$$

最终调度节点为:

$$i^* = \operatorname{argmin}_i (Cost_i^{total}) \quad (5-12)$$

该决策过程在保证数据本地性优先的前提下, 引入负载补偿机制, 有效避免任务长期集中在单一节点, 提升系统整体并行度。

上述调度模型具有以下特点: 1) 计算复杂度低: 调度决策复杂度为  $O(N)$ , 其中  $N$  为计算节点数量; 2) 实现友好: 模型仅涉及加法、比较与移位运算, 适合 FPGA 实现; 3) 调度稳定性强: 迁移约束机制有效抑制频繁数据迁移, 避免系统状态振荡。因此, 该综合调度机制在工程实现复杂度与调度效果之间取得了良好折中。

需要特别指出的是, 上述多因素调度决策模型中涉及的各项核心负载描述参数, 包括数据迁移代价模型中的固定启动延迟  $L_{base}$ 、路径惩罚因子  $L_{hop}$ 、单位数据传输代价  $C_{bw}$ , 以及控制本地性与负载权衡的调度平衡系数  $\beta$  等在理论设计阶段均保持抽象的符号化形式。这种参数化的建模方式, 使得本调度框架在理论上具备了良好的普适性。调度器不仅能够通过动态配置这些参数来描述不同拓扑结构的多节点系统, 如片上网络 NoC、总线拓扑或分布式集群等, 还能够灵活适应具有不同通信计算比 (CCR) 的复杂张量负载特征, 从而在架构层面保障了

系统的可迁移性。关于这些实际负载参数在本文特定多 FPGA 原型平台下的具体量化依据与测定理念，将在第 6 章实验验证部分予以深入剖析。

### 5.3 FPGA 调度器实现

在前述调度模型设计的基础上，本文在 FPGA 上实现了一套面向多计算节点的硬件调度器，用于完成就绪任务的节点选择、数据搬移判定以及调度状态维护等功能。该调度器以有限状态机为核心控制结构，通过模块化设计将任务解析、数据代价计算、调度决策以及状态更新等功能解耦，实现了在有限硬件资源条件下的高效调度决策。

本节将从调度流程、内部结构、硬件代价以及调度状态维护机制等方面，对 FPGA 调度器的实现进行详细说明。

#### 5.3.1 调度流程与状态更新机制

FPGA 调度器整体采用顺序控制与并行计算相结合的设计方式，其调度流程由统一的状态机驱动完成。整体状态转移关系如图5-2 所示。

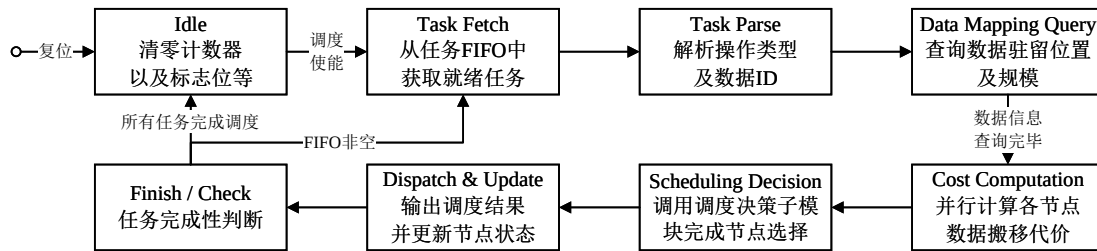


图 5-2 调度决策状态机示意图

在系统上电并完成初始化后，调度器首先进入 Idle 状态，等待系统使能信号。当调度器被使能后，状态机转移至 Task Fetch 状态，从任务 FIFO 中依次获取主机端下发的就绪任务。

任务获取完成后，调度器进入 Task Parse 状态，对任务的操作类型以及其输入输出数据标识进行解析。解析结果将用于后续调度路径选择与数据查询过程。针对不同算子类型（如标量运算、累加运算等），状态机在该阶段对调度流程进行路径裁剪，从而避免不必要的映射查询与代价计算，以降低整体调度开销。

完成任务解析后，调度器进入 Data Mapping Query 状态，通过数据映射查询接口，依次获取任务涉及的数据块在系统中的驻留位置及对应的数据规模信息。该状态下的数据查询结果将作为后续代价计算的输入。

在获得完整的数据位置信息后，状态机转移至 Cost Computation 状态。在该状态中，调度器并行计算各数据块在不同计算节点之间的数据搬移代价，并在此

基础上生成每个节点对应的综合数据搬移代价。该并行计算过程在同一状态周期内完成，不引入额外的状态切换。

随后，调度器进入 **Scheduling Decision** 状态，调用独立的调度决策子模块，综合考虑当前各计算节点的负载状态、剩余存储资源以及任务的数据搬移代价，完成计算节点的选择并生成最终调度决策。

调度决策完成后，状态机进入 **Dispatch & Update** 状态。一方面，调度器向后续执行模块输出任务调度信息及必要的数据搬移指令；另一方面，同步更新内部维护的节点状态信息，为后续任务调度提供最新的状态依据。

在单个任务调度完成后，状态机进入 **Finish / Check** 状态，判断任务 **FIFO** 中是否仍存在待调度任务。若存在未完成任务，则状态机返回 **Task Fetch** 状态继续调度流程；否则，调度器输出处理完成标志并重新进入 **Idle** 状态。

本节所描述的状态更新过程主要关注调度流程中状态切换的时序关系，而状态数据的来源及其可信度问题将在后续小节中进一步讨论。

### 5.3.2 FPGA 调度器内部结构设计

在上一节中，本文从时序角度介绍了调度器在任务获取、决策与结果输出过程中的整体运行流程。本节进一步从结构角度对 **FPGA** 调度器的内部组成进行分析，说明各功能模块的划分方式及其相互关系。如图5-3所示，从结构上看，**FPGA** 调度器主要由以下几个功能模块构成：任务解析与控制模块、数据位置查询与搬移代价计算模块、多因素调度决策模块、节点状态维护模块以及结果与指令输出模块。

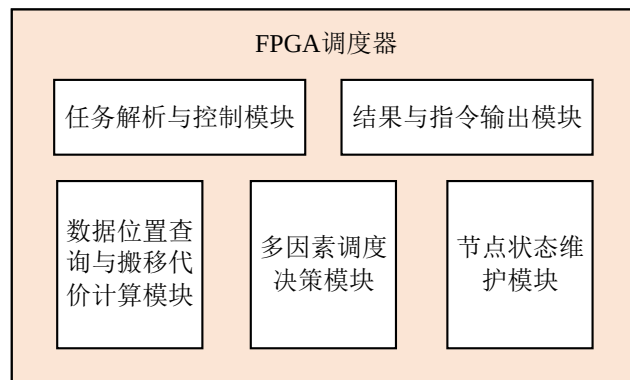


图 5-3 FPGA 调度器内部功能模块结构示意图

任务解析与控制模块负责驱动调度流程，通过统一状态机协调任务请求、解析及各阶段触发顺序，对输入任务的操作类型与数据标识进行解析，并协同完成数据查询与调度决策。

数据位置查询与搬移代价计算模块用于获取输入数据的驻留位置，并基于数据规模与目标节点计算搬移代价。其运算仅涉及移位、加法与比较等简单整数操

作，各节点计算彼此独立，可在 FPGA 上并行展开，从而降低调度延迟。

多因素调度决策模块为核心单元，综合数据搬移代价、节点负载与内存压力，对候选节点进行并行评估，并选取综合代价最小的节点。其关键路径主要由比较级数决定，而非节点数量线性增长，具有良好的并行扩展性。

节点状态维护模块用于管理节点负载与内存状态。一方面，在调度后进行预测性更新；另一方面，在接收真实反馈时进行校正，以避免状态长期漂移。

结果与指令输出模块负责生成执行控制信息，包括数据搬移请求与计算指令下发控制。对于涉及数据搬移的任务，在指令下发前引入数据就绪同步，以保证执行正确性，同时保持数据通路 with 指令通路解耦。

总体而言，该 FPGA 调度器结构规整、模块划分清晰，支持通过模块复制实现横向扩展，具备良好的可扩展性。

### 5.3.3 调度状态的预测更新与反馈校正机制

在多节点调度系统中，调度决策的触发频率通常高于节点状态的上报频率，且状态反馈在传输与处理过程中存在延迟。因此，若调度器仅依赖周期性返回的真实状态进行决策，在高并发或任务密集场景下容易基于过期信息作出判断，从而影响调度效率。相反，若完全依赖内部状态估计而忽略真实反馈，则预测误差会随时间累积，导致状态逐渐偏离真实运行情况。

针对上述问题，本文在调度器中设计了一种结合预测更新与反馈校正的节点状态维护机制，以协调调度决策的实时性需求与节点状态准确性之间的矛盾。该机制的核心思想是在调度决策阶段通过预测更新保证调度响应的及时性，并在获取真实状态反馈后通过校正操作消除预测误差的长期影响。

为形式化描述上述机制，本文将节点  $i$  在时刻  $t$  的状态表示为：

$$S_i(t) = (L_i(t), M_i(t)) \quad (5-13)$$

式(5-13)中， $L_i(t)$  和  $M_i(t)$  分别表示节点  $i$  在时刻  $t$  的负载与可用内存。

调度器内部维护对应的预测状态：

$$\hat{S}_i(t) = (\hat{L}_i(t), \hat{M}_i(t)) \quad (5-14)$$

用于支撑调度决策过程。

当调度器在时刻  $t$  将任务  $k$  分配至节点  $i$  时，根据任务的计算负载需求  $\Delta L_k$  及内存占用需求  $\Delta M_k$ ，对节点预测状态进行即时更新：

$$\hat{L}_i(t^+) = \hat{L}_i(t) + \Delta L_k, \quad \hat{M}_i(t^+) = \hat{M}_i(t) - \Delta M_k \quad (5-15)$$

当调度器接收到来自计算节点的真实状态反馈  $S_i^{real}(t_f)$  时，直接采用真实状态对内部预测状态进行覆盖，即：

$$\hat{S}_i(t_f) \leftarrow S_i^{real}(t_f) \quad (5-16)$$

从而在系统运行过程中持续校正预测误差，避免节点状态估计随时间发生漂移。

通过将预测更新与反馈校正机制相结合，调度器在保证调度决策及时性的同时，具备对状态估计误差的自我修正能力，使系统能够在复杂任务负载与动态资源变化场景下保持稳定且可靠的调度性能。

## 5.4 调度策略实验

### 5.4.1 实验环境与参数指标设计

#### (1) 实验环境设计

为验证前文提出的多因素调度策略在多 FPGA 节点环境下的性能与稳定性，本文在自建硬件调度原型系统上构建了一套可复现的实验环境。实验采用“离线生成工作负载 + 硬件仿真执行 + 结果统计分析”的流程，对不同调度策略在多种系统状态与负载条件下进行对比评估。

系统规模固定为 4 个计算节点，对应 4 个 FPGA 计算单元。调度输入为就绪任务流，任务依赖关系由主机端离线解析，调度器仅负责对就绪任务进行节点选择，从而避免在 FPGA 侧引入复杂的 DAG 解析逻辑，整体设计与主机—调度层—计算节点分层架构一致。

任务流及系统初始状态由 Python 脚本自动生成，包括节点初始负载与内存状态、数据驻留位置与副本分布，以及任务类型与数据依赖等信息。所有实验均通过固定随机种子控制，确保不同策略间工作负载一致，从而保证结果的可对比性与可复现性。

数据层面共维护 512 个张量对象，分为向量与矩阵两类，其规模分布差异明显，以模拟实际计算中的数据访问特征。数据初始状态为节点本地、调度层缓冲区或缺失，并允许多副本存在，以反映多节点环境下的数据本地性与共享特性。

此外，引入参数  $skew \in [0, 1]$  控制任务负载分布。当  $skew = 0$  时，任务计算开销近似随机分布，RR 调度可自然实现负载均衡；当  $skew = 1$  时，高开销任务集中于部分节点，形成极端负载不均场景，用于评估调度策略的适应能力。

调度过程中，调度器基于当前可观测状态与数据分布进行决策，并维护节点负载与内存的预测值。通过控制状态更新频率，构造理想与状态漂移两类场景，用于评估策略在不完备信息下的鲁棒性。

实验结果由仿真平台统一采集，包括任务完成时间 (Makespan)、数据搬移

量、节点负载分布及调度开销等指标，从性能、数据本地性与系统稳定性等维度对调度策略进行综合分析。表5-1 汇总了主要参数及取值范围。

表 5-1 实验工作负载与系统参数配置说明

| 参数名称             | 参数取值           | 参数说明                     |
|------------------|----------------|--------------------------|
| num_nodes        | 4              | FPGA 计算节点数量              |
| tasks            | 500 / 2000     | 单次调度实验中的任务总数             |
| seed             | 1, 2, 3        | 控制任务流与数据分布生成的随机种子        |
| skew             | 0, 0.3, 0.6, 1 | 控制计算任务负载不均衡程度            |
| num_data_objects | 512            | 系统中维护的全局张量数据对象数量         |
| frac_matrix      | 0.45 / 0.60    | 矩阵类张量在所有数据对象中的比例         |
| vec_size_range   | [1, 9]         | 向量类张量的数据规模范围             |
| mat_size_range   | [100, 255]     | 矩阵类张量的数据规模范围             |
| p_absent         | 0.20 / 0.35    | 张量数据在初始状态下所有节点缺失的概率      |
| replica_dist     | 1–4 replicas   | 张量数据在不同节点上的初始副本数量分布      |
| node_mem_kb      | 32 / 64 / 128  | 单个 FPGA 节点的最大可用内存容量 (MB) |

## (2) 调度参数标定

为了使调度策略的仿真评估结果能更有效地反映底层系统的运行特征，本实验中输入的任务流计算延迟与数据通信代价基于自研硬件控制通路及计算核心的测定参数进行标定：

1) 算子计算时间的参数标定：任务流中不同算子的预期执行时间对齐自研计算核心的硬件实际执行时序。在仿真配置中，各典型算子的基准时间参数定义为：矩阵乘算子约为  $2000\ \mu\text{s}$ ，矩阵加与矩阵累加算子为  $180\ \mu\text{s}$ ，列向量乘行向量算子为  $25\ \mu\text{s}$ ，矩阵乘列向量算子为  $4\ \mu\text{s}$ ，行向量乘矩阵算子为  $45\ \mu\text{s}$ ，数乘向量算子为  $100\ \mu\text{s}$ 。此外，数乘矩阵、行向量乘列向量、向量加法及向量累加等轻量级算子的执行时间统一标定为  $2\ \mu\text{s}$ 。算子间的计算开销呈现异构特征，时间窗跨度为  $2\ \mu\text{s} \sim 2000\ \mu\text{s}$ 。

2) 数据搬移代价的参数标定：本实验环境下设计的硬件互联链路物理带宽约为 10 Gbps，控制通路的单次 DMA 配置与软硬件控制握手固有延迟  $L_{base}$  测定约为  $1.5\ \mu\text{s}$ 。对于 1 KB 的张量数据块，其在 10 Gbps 带宽下的理论网络传输耗时为：

$$T_{pure} = \frac{1024 \times 8\ \text{bits}}{10 \times 10^9\ \text{bps}} \approx 0.819\ \mu\text{s} \quad (5-17)$$

结合控制启动开销，单次搬移 1 KB 数据的端到端实际延迟在  $2.3\ \mu\text{s}$  左右。为便

于调度层固件进行硬件友好的快速线性估算，本实验将单位数据传输代价  $C_{bw}$  定量标定为  $2.0 \mu\text{s}/\text{KB}$ ，用以近似表征细粒度传输场景下的有效带宽边界。

基于上述硬件时序与带宽限制，多因素调度模型中的控制常数（如调度平衡系数  $\beta$ ）在本实验场景下确立为推荐值 8。当远端节点的队列空闲时窗优势达到额外通信开销的 8 倍以上时，系统才允许执行跨节点任务迁移。该参数的选择能够较好地适应自研算子的大跨度时序分布，用以在负载均匀性与跨节点传输开销之间寻求相对合理的性能平衡。

#### 5.4.2 整体性能对比分析

在节点状态信息保持准确的理想条件下，本文对多因素调度策略 (Multi) 与三种对比策略——轮询调度 (RR)、本地优先调度 (Local) 以及负载均衡调度 (LB)——进行了整体性能对比分析。所有调度策略均在完全一致的任务流、数据分布及系统初始状态下运行，并通过固定随机种子与统一参数配置生成工作负载，从而保证实验结果在不同策略之间具有良好的公平性与可比性。

为避免不同任务规模对统计结果产生混合影响，本文分别在 500 tasks 与 2000 tasks 两种典型负载规模下对各调度策略进行评估。表中数据以“均值  $\pm$  标准差”的形式表示，用于刻画各策略在相同任务规模下的平均性能及其稳定性。

表 5-2 不同调度策略的整体性能统计结果 (skew=0.0)

| Strategy           | Makespan (ms)      | Avg Input (KB)      | Throughput        |
|--------------------|--------------------|---------------------|-------------------|
| RR (500 tasks)     | 7.692 $\pm$ 1.324  | 87.962 $\pm$ 14.667 | 0.067 $\pm$ 0.012 |
| RR (2000 tasks)    | 26.025 $\pm$ 2.681 | 72.431 $\pm$ 26.848 | 0.078 $\pm$ 0.008 |
| Local (500 tasks)  | 8.776 $\pm$ 1.232  | 0.000 $\pm$ 0.000   | 0.058 $\pm$ 0.009 |
| Local (2000 tasks) | 39.858 $\pm$ 6.984 | 1.263 $\pm$ 2.967   | 0.052 $\pm$ 0.009 |
| LB (500 tasks)     | 6.531 $\pm$ 0.886  | 89.803 $\pm$ 13.822 | 0.078 $\pm$ 0.011 |
| LB (2000 tasks)    | 23.611 $\pm$ 2.085 | 72.791 $\pm$ 26.930 | 0.085 $\pm$ 0.008 |
| Multi (500 tasks)  | 6.587 $\pm$ 0.971  | 2.139 $\pm$ 1.096   | 0.078 $\pm$ 0.012 |
| Multi (2000 tasks) | 23.562 $\pm$ 2.111 | 1.976 $\pm$ 1.667   | 0.086 $\pm$ 0.008 |

表 5-3 不同调度策略的整体性能统计结果 (skew=1.0)

| Strategy           | Makespan (ms)    | Avg Input (KB)   | Throughput    |
|--------------------|------------------|------------------|---------------|
| RR (500 tasks)     | 64.001 ± 0.000   | 125.711 ± 17.112 | 0.008 ± 0.000 |
| RR (2000 tasks)    | 250.003 ± 0.000  | 109.856 ± 47.792 | 0.008 ± 0.000 |
| Local (500 tasks)  | 56.377 ± 7.002   | 0.057 ± 0.342    | 0.009 ± 0.001 |
| Local (2000 tasks) | 255.116 ± 33.732 | 2.807 ± 5.598    | 0.008 ± 0.001 |
| LB (500 tasks)     | 35.368 ± 1.009   | 121.628 ± 15.467 | 0.014 ± 0.000 |
| LB (2000 tasks)    | 137.774 ± 2.129  | 105.255 ± 45.470 | 0.015 ± 0.000 |
| Multi (500 tasks)  | 35.593 ± 1.028   | 4.646 ± 2.405    | 0.014 ± 0.000 |
| Multi (2000 tasks) | 137.996 ± 2.022  | 3.670 ± 3.251    | 0.014 ± 0.000 |

表5-2 给出了负载均匀分布 (skew=0.0) 条件下的性能结果。在 2000 tasks 场景中, Multi 的平均任务完成时间为 25.09 ms, 与 LB(25.10 ms) 基本一致, 显著优于 RR(28.52 ms) 和 Local(40.90 ms)。同时, Multi 在保持吞吐率的同时显著降低数据输入规模, 体现出其在负载均衡与数据本地性之间的有效权衡。

表5-3 展示了高度负载不均 (skew=1.0) 条件下的结果。此时各策略 makespan 均明显上升, 其中 RR 与 Local 由于缺乏全局负载感知, 性能退化尤为显著。相比之下, Multi 与 LB 仍保持稳定表现。在 2000 tasks 条件下, Multi 的 makespan 为 137.996 ms, 与 LB(137.774 ms) 接近, 但显著优于 RR(250.003 ms) 和 Local(255.116 ms)。此外, Multi 在数据输入规模上仍优于 LB, 表明其在负载倾斜场景下仍能有效降低跨节点数据传输。

在此基础上, 后续将结合节点任务分布与数据传输特征, 对不同调度策略在多种负载条件下的行为进行进一步分析。

#### (1) 任务完成时间分析

图5-4 展示了不同 skew 下, 各调度策略在不同任务规模下的任务完成时间呈现一致的相对性能规律, 以及绝对数值随 skew 增大显著上升。

500 tasks 条件下, skew 从 0 增至 1 时, 各策略 makespan 整体单调上升。各策略性能排序基本稳定: LB 与 Multi 最优, 其次为 RR, Local 最差。LB 与 Multi 差异较小, 表明二者在轻中度负载下均可有效缓解数据倾斜带来的性能退化。

2000 tasks 条件下, 系统负载升高, makespan 对 skew 更为敏感。RR 由约 26 ms(skew=0) 升至约 250 ms(skew=1.0), Local 由约 40 ms 升至约 255 ms, LB 与 Multi 由约 23 ~ 24 ms 增至约 136 ~ 137 ms。此外, 高负载下, 数据倾斜对性能的放大效应更明显。skew=1.0 时, Multi 相比 Local 与 RR 的 makespan 分别降低约 46% 和 45%, 在严重负载不均衡场景下仍具备较强度调度适应能力。

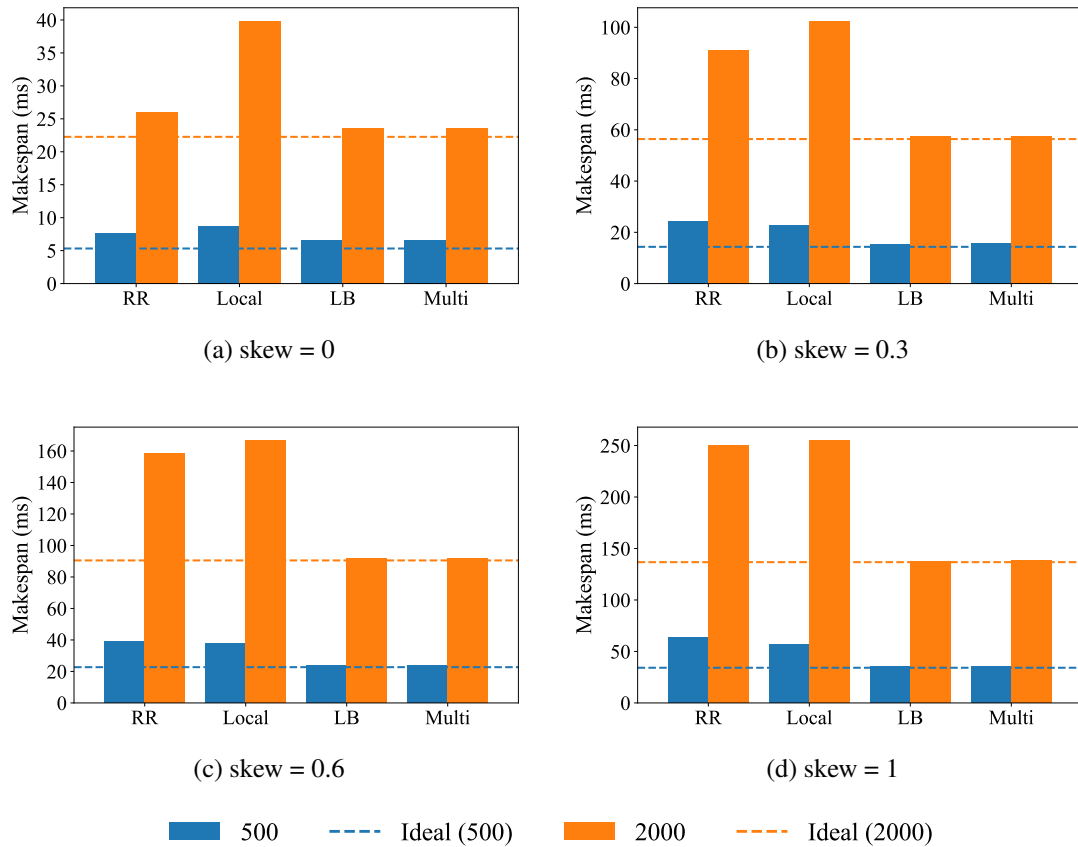


图 5-4 不同调度策略的任务完成时间对比

与 Ideal 值对比可见, skew 增大使各策略与理想下界的差距逐渐拉大, RR 与 Local 尤为明显。RR 在低 skew 时接近理想分配, 但负载不均衡加剧后快速劣化; 而 LB 与 Multi 在各 skew 条件下均更接近理想线, 在任务分配与数据局部性优化上更逼近理论最优。

总体而言, 数据倾斜是影响系统 makespan 的关键因素。具备负载感知的 LB 和 Multi 调度策略, 可在不同 skew 下稳定获得更低完成时间, 在高负载与高倾斜场景下表现出更强的性能优势与鲁棒性。

## (2) 输入数据搬移开销分析

图5-5 给出了不同调度策略在两种任务规模下的平均输入数据搬移量对比结果。从不同 skew 条件下的子图可以看出, 各策略在数据搬移量上的相对关系基本保持稳定, 说明该指标对负载倾斜程度的敏感性较低。

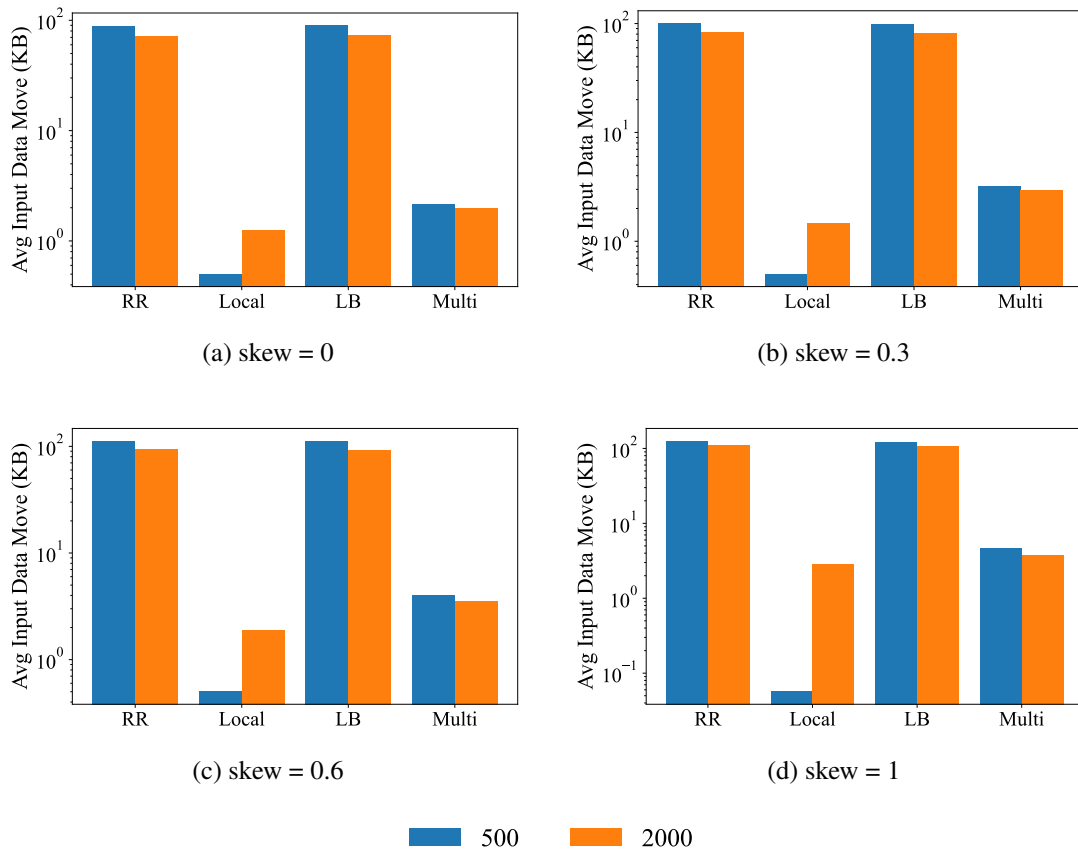


图 5-5 不同调度策略的平均输入数据搬移量对比

500 tasks 条件下, RR 与 LB 策略的平均输入数据搬移量分别为  $87.96 \pm 14.67$  KB 和  $89.80 \pm 13.82$  KB, 处于高数据搬移区间。相比之下, Multi 策略的平均输入数据搬移量仅为  $2.14 \pm 1.10$  KB, 相较 LB 降低约 97.6%, 表明其能够稳定抑制不必要的数据迁移。Local 策略在 500 tasks 条件下几乎不产生跨节点输入数据搬移, 其平均输入搬移量为 0 KB, 符合数据本地性策略的预期。

2000 tasks 条件下, RR 与 LB 策略的平均输入数据搬移量分别为  $72.43 \pm 26.85$  KB 和  $72.79 \pm 26.93$  KB, 且波动幅度进一步增大。Multi 策略在该条件下的平均输入搬移量为  $1.98 \pm 1.67$  KB, Local 策略的平均输入搬移量为  $1.26 \pm 2.97$  KB, 在不同任务规模及不同 skew 条件下均保持了稳定的低搬移水平。

在高数据倾斜条件 (skew=1.0) 下, 各策略的输入数据搬移量整体略有上升, 但相对关系仍保持一致。例如, RR 与 LB 的平均输入搬移量分别约为  $125.71 \pm 17.11$  KB 和  $121.63 \pm 15.47$  KB, 而 Multi 仍维持在较低水平 (约  $4.65 \pm 2.41$  KB), 进一步说明其在不同负载分布条件下均具备良好的数据局部性优化能力。

总体而言, 输入数据搬移量对负载倾斜的敏感性较低, 但不同调度策略在数据局部性利用能力上的差异始终存在, 并构成影响系统性能的重要因素之一。

### (3) 系统吞吐率分析

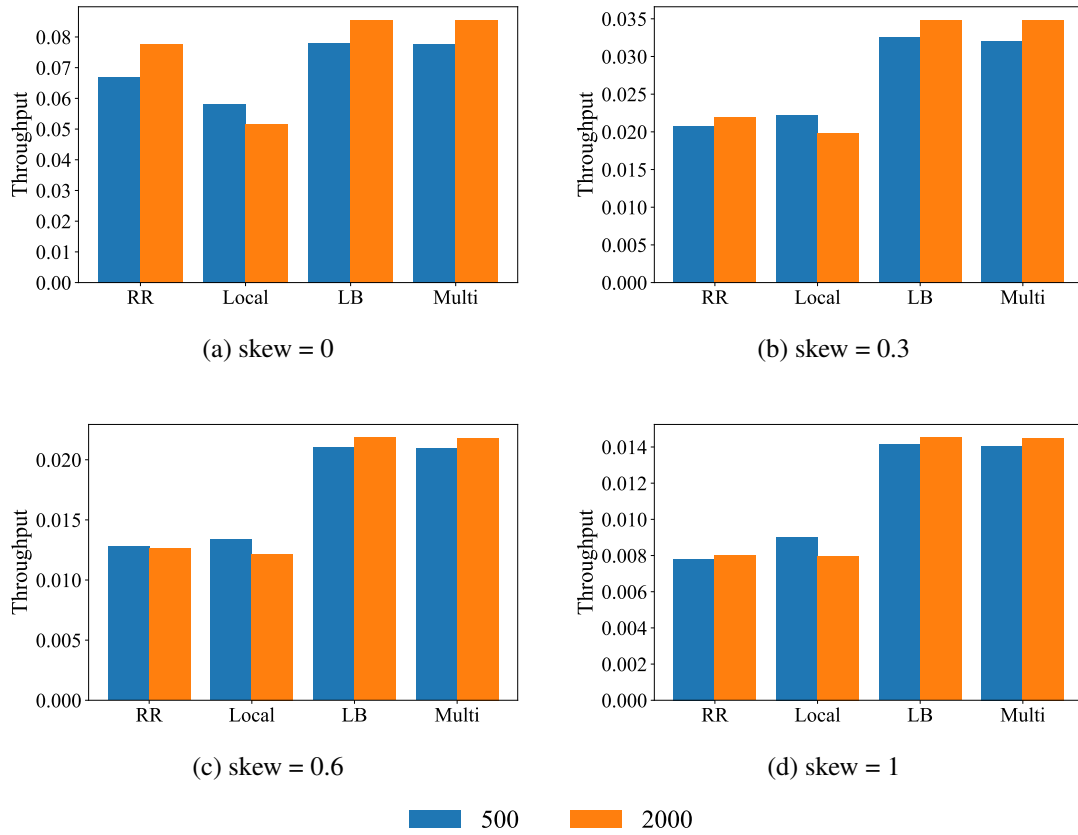


图 5-6 不同调度策略的系统吞吐率对比

图5-6 展示了不同调度策略在两种任务规模及不同 skew 条件下的系统吞吐率对比结果。从各子图可以看出，随着 skew 增大，所有策略的吞吐率均呈下降趋势。这主要是由于矩阵乘法任务比例随 skew 增大而提高，在相同任务数量下增加了整体计算开销，从而导致 makespan 增大并进一步降低系统吞吐率。

在 500 tasks 条件下，LB 与 Multi 策略在各 skew 场景下始终保持较高吞吐率，并整体优于 RR 与 Local。例如，在 skew=0 时，LB 与 Multi 的平均吞吐率分别为  $0.074 \pm 0.012$  和  $0.071 \pm 0.011$  tasks/ $\mu$ s；随着 skew 增大，其吞吐率虽有所下降，但仍明显高于其他策略，表现出更优的调度效率。

当任务规模提升至 2000 tasks 时，LB 与 Multi 的吞吐率在低 skew 条件下进一步提升，分别达到  $0.080 \pm 0.008$  和  $0.081 \pm 0.009$  tasks/ $\mu$ s，且在不同 skew 条件下始终保持接近水平。相比之下，RR 与 Local 的吞吐率下降更为明显，其中 Local 在各条件下整体吞吐率较低且变化幅度有限，表明其并行执行效率持续受限，而 RR 受 skew 的影响最大，在 skew 增大时其吞吐率明显下降。

总体而言，吞吐率的变化主要受任务计算复杂度的影响，而非仅由负载分布

决定；在此基础上，具备负载感知能力的 LB 与 Multi 策略仍能够在不同 skew 条件下保持更高的系统执行效率。

#### (4) 吞吐率与数据搬移关系分析

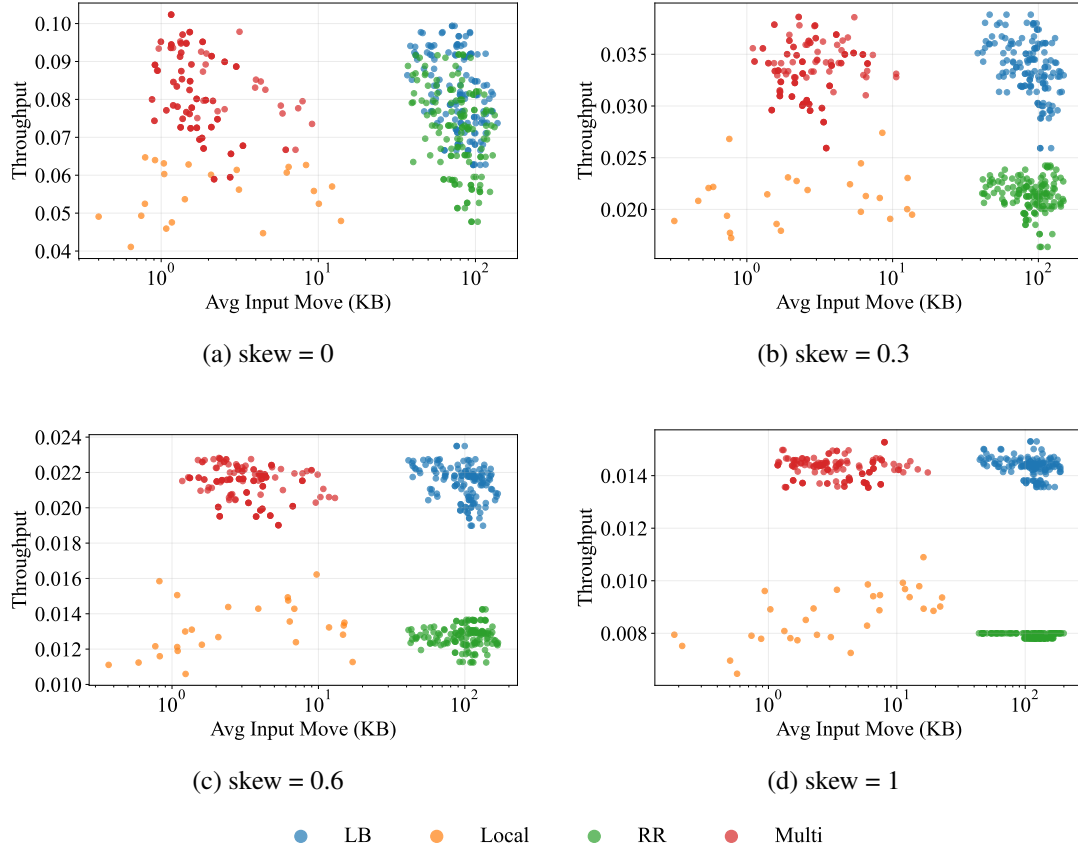


图 5-7 系统吞吐率与平均输入数据搬移量之间的关系

图5-7 从散点分布角度展示了不同 skew 条件下系统吞吐率与平均输入数据搬移量之间的关系。随着 skew 增大，各策略在平面中的分布逐渐呈现出更清晰的结构性差异。

具体而言，RR 策略始终位于高数据搬移区域，但其吞吐率随 skew 增大明显下降，由“高搬移-高吞吐”逐步转变为“高搬移-低吞吐”，表明其对任务计算复杂度变化较为敏感。Local 策略则始终集中于低搬移区间，吞吐率整体较低且变化幅度有限，说明其性能主要受调度策略本身限制，而对 skew 的响应较弱。

相比之下，LB 策略在低 skew 条件下分布较为离散，而随着 skew 增大，其散点逐渐向“高搬移-高吞吐”区域集中，表现出更稳定的负载调度能力。Multi 策略则在各 skew 条件下均稳定分布于“低搬移-高吞吐”区域，且随着 skew 增大其分布进一步收敛，体现出在不同任务结构下更强的一致性与鲁棒性。

总体而言，不同调度策略在“数据搬移-吞吐率”空间中的分布轨迹揭示了

其对任务复杂度变化的响应机制，其中 **Multi** 在效率与数据局部性之间始终保持最优权衡。

### 5.4.3 动态收敛性分析

在实际系统运行过程中，任务负载与数据分布往往呈现出显著的非均匀特征。与前文基于统计结果的性能对比分析不同，本节进一步从调度过程的动态演化角度，对不同调度策略的收敛行为与稳定性进行分析。考虑到不同 **skew** 条件下的整体性能趋势在前文已得到充分验证，本节选取 **skew = 1** 的典型非均衡场景作为代表案例，对调度过程的动态收敛行为进行深入分析。

Case0038\_sk3 来源于统一参数空间下的实验配置 **sweep**，其 **skew = 1.0**，任务总数为 2000，单节点最大可用内存为 64 MB。在任务类型分布上，矩阵类计算任务占比为 60%，整体计算开销较高。同时，在数据分布方面，该 case 采用非均匀的数据副本配置，其中约 70% 的数据仅存在单一副本，三副本及以上的数据比例不足 5%，并设置较高的数据缺失概率  $p_{\text{absent}} = 0.35$ ，从而在调度初期形成显著的数据访问集中与节点负载不均衡现象。

为定量刻画系统负载分布情况，本文引入负载不均衡度 (Backlog Imbalance) 作为衡量指标。设在任务  $t$  调度完成后，第  $i$  个节点的负载为  $B_i(t)$ ，则系统在时刻  $t$  的负载不均衡度定义为：

$$\text{Imbalance}(t) = \frac{\sigma(B_1(t), B_2(t), \dots, B_N(t))}{\mu(B_1(t), B_2(t), \dots, B_N(t))} \quad (5-18)$$

式(5-18)中， $\mu(\cdot)$  表示各节点负载的算术平均值， $\sigma(\cdot)$  表示负载的标准差。该指标通过对标准差进行归一化，有效反映系统负载分布的相对均衡程度。当不均衡度接近 0 时，系统处于均衡状态；反之则表明任务负载高度集中于部分节点。

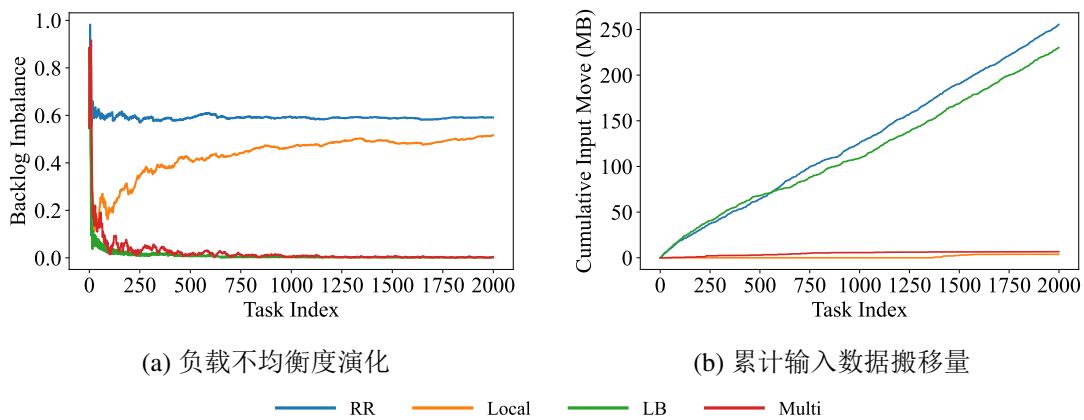


图 5-8 不同调度策略的负载不均衡度与数据搬移量演化 (**skew = 1.0**)

如图5-8(a)所示，各调度策略在初期均表现出较高不均衡度，随后呈现不同

收敛趋势。**Local** 策略初期略有改善, 但随后不均衡度回升并稳定在约 0.45–0.50, 说明其依赖数据本地性, 难以持续优化全局负载。**RR** 策略虽能快速降低不均衡度, 但最终稳定在约 0.58–0.60, 表明其在高度不均衡场景下优化能力有限。

**LB** 策略可在短时间内将不均衡度降至 0.05 以下, 并保持稳定, 体现出良好的负载调节能力。**Multi** 策略同样可在约 200 个任务内将不均衡度降至接近 0, 但存在一定波动, 源于其在负载均衡与数据迁移成本之间进行动态权衡。

图5-8(b) 显示数据搬移情况。**RR** 与 **LB** 在调度过程中持续产生数据迁移, 累计输入搬移量近似线性增长, 最终分别约为 255 MB 和 230 MB。**Local** 几乎不产生数据迁移, 但以负载不均衡为代价。**Multi** 在初期通过少量迁移缓解不均衡, 后期增长显著放缓, 总搬移量控制在约 10 MB, 明显低于 **RR** 与 **LB**, 接近 **Local**。

综合来看, **Multi** 在接近 **LB** 的负载均衡效果下, 大幅降低了数据迁移开销, 实现了较好的权衡; **RR** 与 **Local** 分别受限于负载感知不足和过度依赖本地性, 而 **LB** 虽均衡效果好, 但数据迁移代价较高。

#### 5.4.4 状态误差下的稳定性分析

在实际多节点调度系统中, 调度器所依赖的节点状态信息往往存在执行时间扰动与反馈延迟, 从而导致调度决策基于的状态与真实运行状态之间产生偏差。为评估所提出“预测更新与反馈校正机制”在非理想状态下的有效性与稳定性, 本节在前述仿真平台基础上, 引入执行时间抖动与状态反馈机制, 对调度策略在状态误差条件下的表现进行分析。

##### (1) 实验环境

基于 SystemVerilog 搭建调度系统测试平台, 核心为 `tb_scheduler_system_real.sv` 模块。该测试平台通过构建预测执行轨 (`model`) 与真实执行轨 (`real`) 双轨机制, 对调度器在不同状态感知条件下的行为进行仿真评估。实验采用的案例为 `case0024_sk1`, 该案例在负载分布、数据局部性及资源约束等方面均处于中等复杂度区间, 能够较好地反映实际分布式系统中的典型运行特征。在此基础上, 引入如下关键建模因素:

1) 执行时间扰动比例 (Jitter Percent): 通过参数 `JITTER_PCT` 在任务执行时间中引入随机扰动, 用于模拟实际系统中的执行不确定性;

2) 反馈控制模式 (Correction Mode): 通过参数 `CORR_MODE` 控制调度器所使用的状态来源, 包括仅使用预测状态、使用真实状态以及周期性校正等方式;

3) 周期反馈间隔 (Correction Period): 在周期反馈模式下, 通过 `CORR_PERIOD_US` 控制状态更新的时间间隔;

4) 节点性能动态变化 (Dynamic Node Variation): 在仿真过程中周期性引入

节点执行速度变化，通过参数 PAUSE 控制速度变化幅度，用于模拟系统运行时的负载波动或资源竞争。

### (2) 实验设计

为评估调度策略在不同状态感知条件下的性能表现，从反馈机制的可用性、准确性及时效性三个维度出发，构建了四类典型实验模式：

#### 1) 开环模式 (OPEN\_LOOP, 记为 OL)

该模式用于刻画无反馈条件下的开环系统行为，作为分析调度策略在不确定性影响下性能表现的参考基线。

#### 2) 有偏反馈模式 (BIASED\_FEEDBACK, 记为 FB-B)

该模式用于模拟实际系统中节点仅能基于经验或统计信息进行时间估计，而无法准确感知执行扰动的情形。

#### 3) 完全反馈模式 (PERFECT\_FEEDBACK, 记为 FB-P)

该模式对应理想化的完备状态感知系统，用于作为性能分析的上界参考。

#### 4) 周期反馈模式 (PERIODIC\_FEEDBACK, 记为 FB-T)

该模式用于刻画实际系统中由于监控频率受限、通信开销或系统延迟所导致的非实时状态感知场景。

各模式通过对执行时间扰动及状态反馈机制的不同建模，用于刻画从理想情况到实际系统的多种运行场景。具体参数配置见表5-4：

表 5-4 不同实验模式下的参数配置

| 模式   | JITTER_PCT | CORR_MODE | CORR_PERIOD_US | PAUSE     | 状态来源         |
|------|------------|-----------|----------------|-----------|--------------|
| OL   | 20         | 0         | 1000           | 1.2/1.5/2 | 预测状态 (model) |
| FB-B | 20         | 1         | 1000           | 1.2/1.5/2 | 预测状态 (model) |
| FB-P | 20         | 1         | 1000           | 1.2/1.5/2 | 真实状态 (real)  |
| FB-T | 20         | 2         | 1000           | 1.2/1.5/2 | 周期校正 (model) |

### (3) 实验结果与分析

#### 1) Makespan 性能分析

在不同反馈模式及系统负载条件下，各调度策略的 Makespan 表现如图5-9所示。图中选取了性能扰动较大的 PAUSE = 2 与扰动较小的 PAUSE = 1.2 两种典型场景，以对比不同误差水平下系统行为的差异。

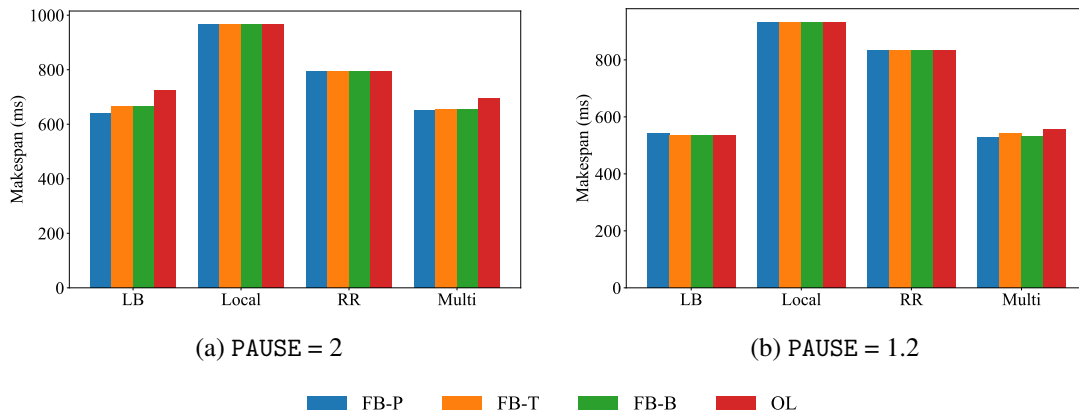


图 5-9 不同反馈模式与负载条件下调度策略的 Makespan

从调度策略横向对比的角度来看，在所有反馈模式与负载条件下，各策略的整体性能排序与理想模型分析结果基本一致，即  $LB \approx Multi > RR > Local$ 。其中，LB 与 Multi 策略始终表现出最优或近似最优的 Makespan，而 RR 次之，Local 策略性能最弱。这表明，无论采用何种反馈模式，所设计的调度策略均能够有效发挥作用，并保持稳定的相对性能优势。

从反馈模式纵向对比的角度来看，对于同一调度策略，尤其是 LB 和 Multi，不同反馈机制对其性能存在显著影响。在 OL 模式下，由于调度器完全依赖预测信息进行决策，缺乏对实际执行状态的校正能力，在存在执行时间扰动时会产生持续偏差，导致资源分配失衡，从而表现出最差的 Makespan。相比之下，FB-P 模式能够实时获取准确的系统状态，其性能始终作为整体最优基准。

进一步观察可以发现，FB-T 与 FB-B 模式的性能均仅略弱于 FB-P，且二者之间差异较小。这表明周期性反馈与基于预测的反馈在实际调度过程中具有相似的效果。其原因在于，相较于计算节点的执行时间，调度器的调度开销极小，因此反馈更新的时间粒度对整体调度结果影响有限，使得周期校正与实时预测在宏观性能上趋于一致。

结合不同扰动幅度进行分析可以看出，当  $PAUSE = 2$  时，不同反馈模式之间的性能差距显著扩大；而在  $PAUSE = 1.2$  的低扰动场景下，各反馈模式表现趋于接近。这说明反馈机制在高不确定性环境中尤为关键，能够有效提升调度策略在误差条件下的稳定性与鲁棒性。

## 2) Backlog 动态分析

为进一步分析不同反馈模式下系统的动态行为，对各节点 Backlog 随任务推进的变化过程进行了观察，图5-10选取了  $PAUSE = 2$  条件下 Multi 调度策略在不同反馈模式下的多节点 Backlog 增长情况。

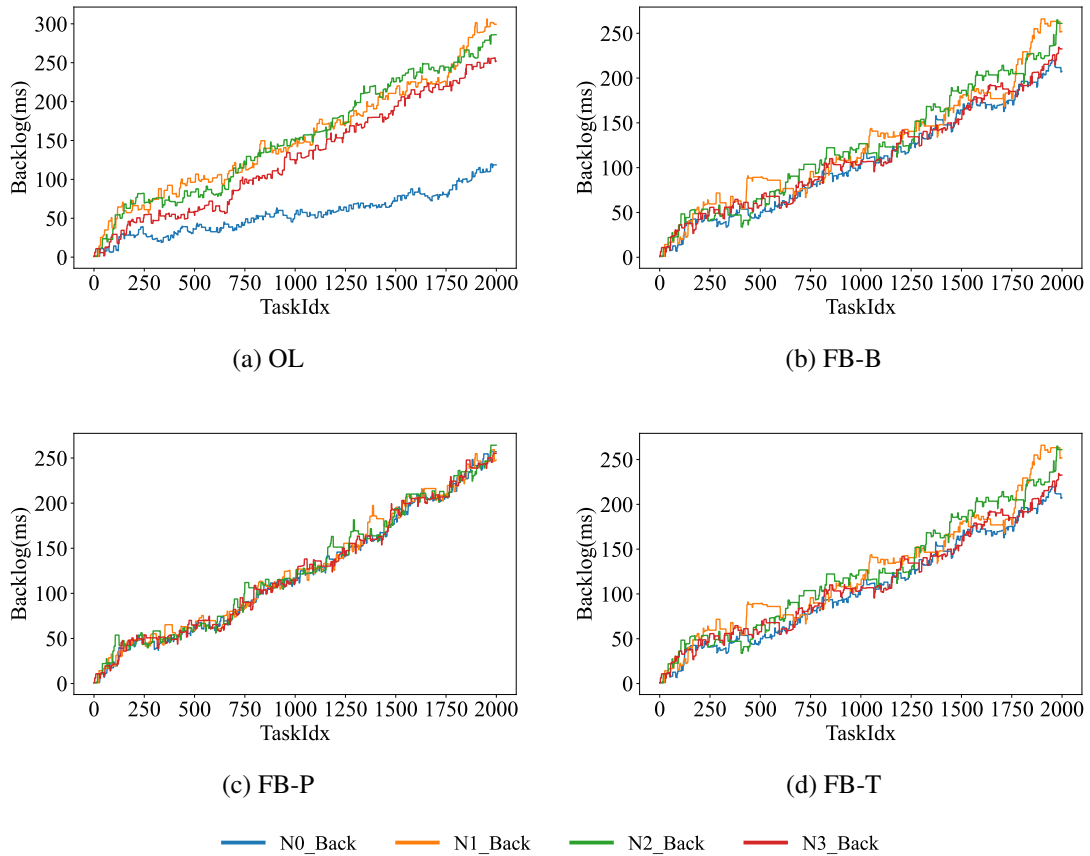


图 5-10 不同反馈模式下 Multi 调度策略在 PAUSE = 2 条件下各节点 Backlog 增长情况

OL 模式下，由于调度器始终基于预测状态进行决策，而未能感知执行时间扰动所带来的状态偏移，各节点的 Backlog 呈现出明显的不均衡增长趋势。部分节点出现持续积压，其他节点则利用率较低，反映出典型的开环系统失配问题。

相比之下，FB-P 模式下各节点 Backlog 曲线最为平滑且收敛性最好。由于调度器始终基于真实状态进行决策，系统能够及时响应负载变化，实现较为均衡的任务分配，从而避免了局部拥塞现象。

在 FB-B 与 FB-T 模式下，系统均引入了反馈机制，对预测误差进行一定程度的修正。可以观察到，两种模式下各节点 Backlog 曲线整体呈现较为一致的增长趋势，未出现 OL 模式中明显的节点失衡现象，说明反馈机制能够有效缓解状态偏移带来的负载不均问题。

尽管相较于 FB-P 模式仍存在一定程度的波动，节点间仍可观察到轻微差异，但整体稳定性已显著提升，系统运行状态保持在可控范围内。进一步对比可以发现，FB-B 与 FB-T 的表现高度相似，均未表现出明显优劣差异。

这一现象表明，在当前系统设定下，反馈更新的时间粒度对调度结果的影响有限。其根本原因在于，相较于计算节点的执行时间，调度器的调度开销极小，

使得周期性反馈与基于预测的反馈在宏观调度行为上趋于一致。因此，FB-T 在保持较低通信与实现开销的同时，能够达到接近 FB-B 乃至 FB-P 的稳定性表现，体现出良好的工程实用性。

#### （4）总结

综上，在引入执行时间扰动与状态感知误差的条件下，各调度策略在不同反馈模式下表现出显著差异。实验结果表明：

在调度策略层面，LB 与 Multi 始终取得最优或近似最优的 Makespan，表明所设计策略在不同状态感知条件下具有良好的有效性与稳定性。

在反馈机制层面，完备反馈（FB-P）构成性能上界，而开环模式（OL）在存在扰动时表现最差，说明缺乏状态校正会导致调度决策持续偏离真实系统状态。相比之下，引入反馈机制（FB-B、FB-T）能够显著缓解状态误差带来的负面影响，从而提升系统整体稳定性。

进一步分析表明，周期反馈（FB-T）与有偏反馈（FB-B）在性能及动态行为上表现接近，表明在计算时间远大于调度开销的系统中，反馈更新粒度对调度结果的影响有限，使周期性校正能够以较低开销逼近实时反馈的效果。

此外，随着系统扰动幅度与负载水平的增加，不同反馈模式之间的性能差距进一步扩大，表明在高不确定性环境下，反馈机制对于保障系统稳定性具有更加关键的作用。

## 第六章 系统实现与实验分析

本章在前文系统架构设计、数据管理机制与调度策略实现的基础上，给出完整系统原型的硬件实现，并通过一系列系统级实验对本文提出的张量计算调度系统进行验证与分析。实验重点评估数据-指令解耦执行语义下系统的运行行为与性能收益。

需要说明的是，本文所使用的计算节点为实验室自研加速器阵列，主要用于验证调度、通信与指令机制的时序行为及其系统开销。鉴于本文侧重系统层调度机制与数据-指令解耦执行语义，本章实验主要关注调度机制与数据/指令通路的系统级特性，而不涉及算子绝对计算性能的评估。

### 6.1 系统原型实现

为支撑后续实验验证，本文基于 FPGA 构建了张量计算调度系统原型。该原型系统实现了第四章所述的数据管理机制与第五章提出的多因素调度策略，并通过“主机-调度器-加速器”三层结构进行部署，形成完整的系统执行闭环。

#### (1) 硬件实现结构

图6-1 给出了系统在 FPGA 上的整体硬件实现结构。为提高图示可读性，对部分长距离连接进行了适当简化处理，但不影响整体结构与连接关系的表达。该结构基于 Vivado Block Design 构建，由核心 RTL 模块、接口模块以及存储模块等部分组成，其余为支撑系统运行的配套功能模块。

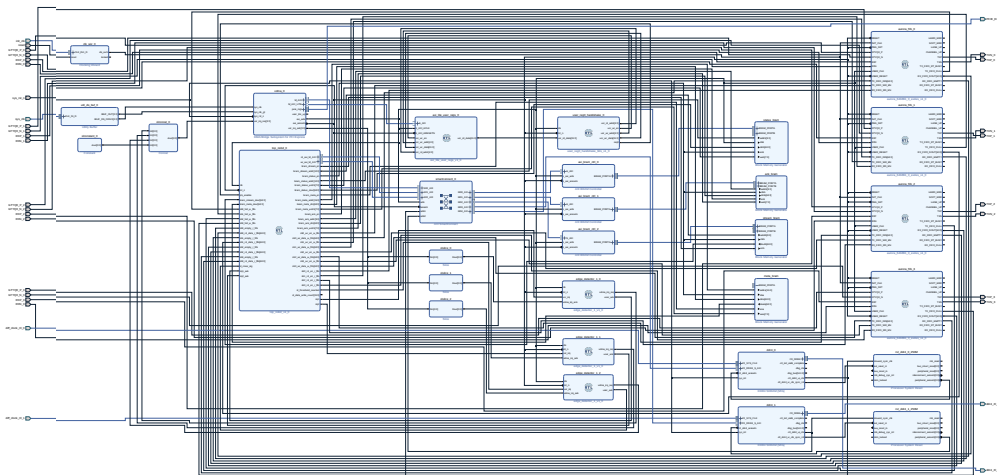


图 6-1 系统在 FPGA 上的整体硬件实现结构

其中，图左侧的最大模块为核心 RTL 模块，集成了调度控制逻辑与数据管理逻辑。调度部分负责任务选择与执行状态维护，数据管理部分负责缓存、迁移与一致性控制，两者协同完成系统控制与数据管理功能。图右侧第一列为接口模块，负责调度器与主机以及计算节点之间的数据与控制交互；第二列为存储模块，用于存储指令、元数据信息及计算数据，并为数据传输与调度执行提供支撑。

### (2) 板级实现与系统部署

图6-2 展示了系统在 FPGA 板卡上的物理实现。该调度板基于 Xilinx Virtex UltraScale+ FPGA 构建，集成板载 DDR 以及高速串行接口，用于支持主机通信、数据缓存以及节点间数据传输。

其中，QSFP/SFP/PCIE 等高速接口基于 FPGA 板载高速串行收发器实现，用于完成 FPGA 与 Host 及计算节点之间的数据通信；板载 DDR 用于数据缓存与中间结果存储。通过板上扩展接口连接计算节点，从而构建多节点调度执行环境。

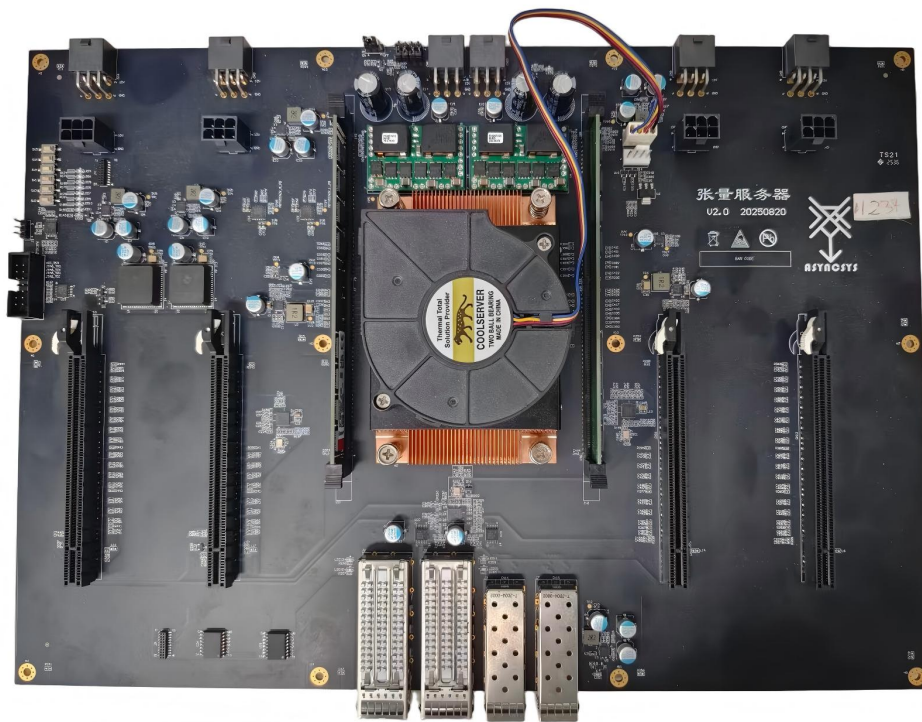


图 6-2 FPGA 调度板卡实物图

### (3) 资源利用情况

表6-1 给出了调度系统在 FPGA 上的资源利用情况。从结果可以看出，系统在 LUT、寄存器 FF 等通用逻辑资源上的占用比例较低，而 BRAM、IO 及 GT 等资源则主要用于数据缓存与高速通信相关功能，整体资源分布与系统以调度控制与数据通路管理为主的设计目标相一致。

其中,LUT 与寄存器资源主要用于实现调度控制逻辑与数据管理逻辑,BRAM

资源用于缓存指令、元数据及中间数据；DSP 资源占用极低，符合系统非计算密集型的设计特点。此外，GT、MMCM 及 IO 等资源的使用表明系统已集成高速串行通信及时钟管理等功能，能够支持多节点环境下的数据交互需求。

整体来看，当前设计在资源使用上保持了良好的功能-开销平衡，同时仍具备较大的资源余量，为后续扩展计算节点规模或增强调度功能提供了空间。

表 6-1 FPGA 资源利用情况

| 资源类型   | 使用量    | 可用量     | 利用率 (%) |
|--------|--------|---------|---------|
| LUT    | 113645 | 1728000 | 6.58    |
| LUTRAM | 10264  | 791040  | 1.30    |
| FF     | 153937 | 3456000 | 4.45    |
| BRAM   | 482    | 2688    | 17.93   |
| DSP    | 6      | 12288   | 0.05    |
| IO     | 251    | 702     | 35.75   |
| GT     | 10     | 76      | 13.16   |
| MMCM   | 3      | 16      | 18.75   |
| PLL    | 6      | 32      | 18.75   |

#### (4) 核心时序路径与决策延迟分析

除了上述静态资源利用率指标外，本节针对主调度状态机单元(task\_scheduler)及多节点决策子模块(scheduler\_decision\_4node)的硬件时序关键路径与决策延迟进行分析。

1) 调度决策时钟周期延迟：根据控制流状态机设计，双目算子（如矩阵乘/加）与标量算子（如数乘矩阵）从任务提取到结果输出分别固定消耗 10 个和 9 个时钟周期。具体表现为：状态机首先通过 4 个周期完成任务获取与锁存；随后，双目与标量算子分别经历 4 个和 3 个周期的元数据表流水线查询；最后通过 2 个周期执行并行决策并输出有效调度报文。由于系统采用数据与指令解耦的双通路架构，该数十纳秒级的固定时钟延迟在流式执行中可被前序任务的硬件计算时间重叠隐藏。

2) 组合逻辑深度与关键路径：子模块 scheduler\_decision\_4node 采用纯组合逻辑构建级联代价网络。由于包含 4 个节点的并发判决，其最长组合逻辑深度发生在从 32 位节点排队延迟信号输入，经过基准寻找、移位减法器校验、直至最终 4 路总代价判决输出的通路上。该通路未插入流水线寄存器打拍，其累加门延迟构成了调度器的核心时序关键路径（Critical Path）。

3) 最大工作频率( $F_{max}$ )预估：在系统所用硬件平台上，虽然核心决策子模块未进行流水线切分，但其全网完全由 32 位加减器、比较器以及移位逻辑构成，

不涉及专用硬件乘法器或除法硬核。经时序网络延迟预估,该4级深度的组合逻辑链总延迟可控制在  $3.5\text{ ns} \sim 4.2\text{ ns}$  以内,对应的最高理论工作频率 ( $F_{max}$ ) 在  $230\text{ MHz} \sim 280\text{ MHz}$  之间。在实际系统的硬件部署验证中,该调度模块的主时钟域统一配置为  $100\text{ MHz}$  (时钟周期  $5\text{ ns}$ )。预留了充足的时序余量,此时,10个周期的决策时延折合仅为  $100\text{ ns}$ ,在宏观算子流调度中属于较低的控制开销。

上述系统原型为后续实验提供了统一的运行平台。在此基础上,下一节将给出实验环境配置、工作负载设计及评估指标,用于对系统性能进行系统性分析。

## 6.2 实验环境与评估方法

### 6.2.1 实验目的

本节给出原型系统的实验平台、软硬件配置、工作负载设置及评估指标,用于保证后续实验结果具备可复现性与可比性。实验的核心目标包括:

(1) 验证系统在长时间、多 case 连续运行条件下的稳定性,观察是否出现死锁、卡死、执行中断或状态异常等问题;

(2) 基于系统实际运行结果,比较解耦机制下的任务完成时间与耦合语义对应完成时间之间的差异,验证解耦机制能否有效缩短整体完成时间;

(3) 结合运行日志中统计得到的通信相关指标,分析解耦机制带来的完成时间收益来源,重点观察被系统运行过程隐藏的通信开销规模;

(4) 在固定调度策略条件下,进一步比较不同通信压力场景中的收益表现,以说明当通信开销更突出时,解耦机制的作用更加明显。

综上,实验从系统稳定性、执行效率提升以及通信行为变化三个方面对所提出的解耦机制进行综合评估。

### 6.2.2 实验平台

实验运行在 Windows 平台主机上,Host 负责测试用例生成、运行时日志记录与结果汇总、数据预处理、DAG 依赖解析与维护等。硬件调度层采用 Xilinx Virtex UltraScale+ FPGA 板卡,实现指令分发、DMA 触发与链路管理等功能。实验平台的关键软硬件参数如表6-2所示。此外,为展示系统的实际部署形态,图6-3给出了调度板与多个计算节点的连接与运行状态。系统采用基于 FPGA 的调度板作为核心控制节点,通过板上扩展接口连接多个计算节点,实现多节点任务调度与数据通路管理。

表 6-2 实验平台关键参数

| 项目       | 配置                                                                                                                                              |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Host     | Intel Core i7-14700K<br>主频约 3.418 GHz; 内存约 68 GB<br>Windows Server 2016 / Windows 10(OS Build 19045)                                            |
| FPGA 调度器 | xcvu13p-fhgb2104-2-e<br>PCIe 3.0 $\times 16$ ; DDR4 $\times 2$ (总 16 GB, 3200 MHz)<br>QSFP28 $\times 2$ (最高 100 Gb); SFP+ $\times 2$ (最高 10 Gb) |
| 计算节点     | xc7k325tffg900-2                                                                                                                                |

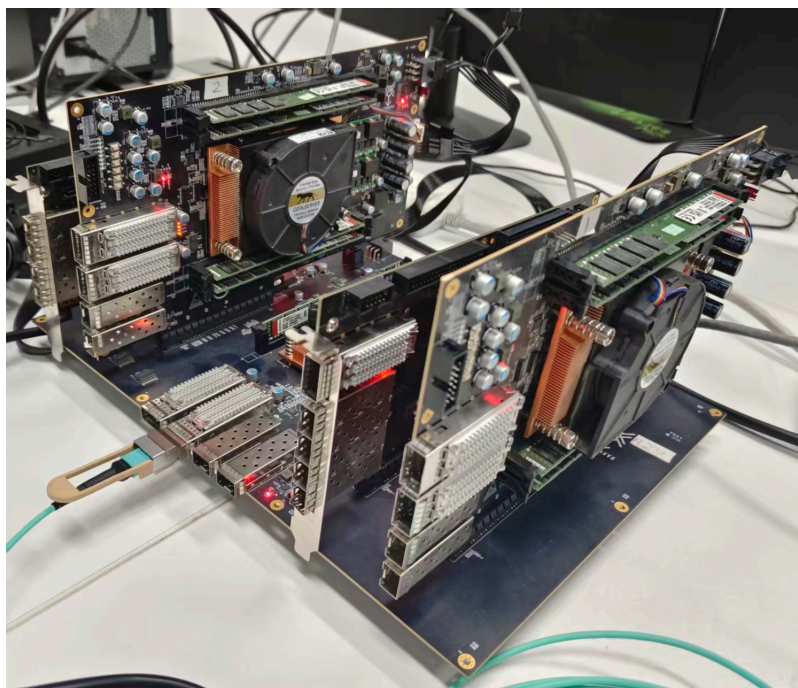


图 6-3 调度系统运行实物图

### 6.2.3 工作负载

实验采用批量生成的 case 集合作为输入, 通过 `stats.json` 与运行结果文件共同描述每个 case 的任务组成、数据分布特征及执行结果。实验任务覆盖矩阵乘、矩阵加、向量加、归约以及标量--向量/矩阵运算等多类算子; 不同 case 在矩阵类任务占比、初始副本分布、输出数据缺失比例与数据存在性等方面均存在差异, 从而形成不同程度的通信压力与数据准备负担。

其中重点选取 Multi 与 LB 两种策略进行分析。Multi 代表本文实现中面向通信隐藏与执行协同的调度方式; LB 则在部分工作负载下会暴露出更高的通信开

销，因此适合作为观察解耦收益边界与通信主导情形的对照对象。

#### 6.2.4 评估指标

本章所有性能数据均来源于系统实际运行结果以及运行日志统计。对于每个 case-strategy 实例，系统记录任务完成时间、计算时间以及通信相关统计信息。后续分析均建立在这些实验观测数据之上。本文主要采用以下指标：

##### (1) 基础观测量

系统运行过程中记录以下关键时间指标：

1)  $T_{\text{comp}}$ ：计算时间，由实验结果中的 computation time 统计项给出，用于刻画任务执行路径上的主要计算与调度推进时间；

2)  $T_{\text{comm}}^{\text{coupled}}$ ：耦合执行条件下的通信开销，表示通信完全暴露于关键路径时对执行时间的影响；

3)  $T_{\text{comm}}^{\text{decoupled}}$ ：解耦执行条件下的通信开销，表示在通信与计算重叠后仍暴露于关键路径的通信时间。

##### (2) 任务完成时间

任务完成时间表示系统完成某一 case 下全部任务所经历的总时间，用于衡量端到端执行效率。基于不同执行模式，定义为：

$$T_{\text{coupled}} = T_{\text{comp}} + T_{\text{comm}}^{\text{coupled}} \quad (6-1)$$

$$T_{\text{decoupled}} = T_{\text{comp}} + T_{\text{comm}}^{\text{decoupled}} \quad (6-2)$$

##### (3) 隐藏通信时间

为刻画解耦机制带来的通信隐藏效果，定义隐藏通信时间如公式 6-3 所示，该指标表示在系统运行过程中被成功隐藏的通信开销规模。

$$T_{\text{hidden}} = T_{\text{comm}}^{\text{coupled}} - T_{\text{comm}}^{\text{decoupled}} \quad (6-3)$$

##### (4) 完成时间降低比例

为评估解耦机制带来的端到端性能收益，定义完成时间降低比例，如公式 6-4 所示，该指标刻画出解耦机制相对于耦合执行在整体完成时间上的优化幅度。

$$\text{ReductionRatio} = \frac{T_{\text{coupled}} - T_{\text{decoupled}}}{T_{\text{coupled}}} = \frac{T_{\text{hidden}}}{T_{\text{coupled}}} \quad (6-4)$$

##### (5) 加速比

定义解耦执行相对于耦合执行的加速比，如公式 6-5 所示，该指标用于衡量解耦机制带来的整体执行效率提升。

$$\text{Speedup} = \frac{T_{\text{coupled}}}{T_{\text{decoupled}}} \quad (6-5)$$

上述指标从执行时间分解与整体性能两个角度刻画了解耦机制的作用效果，为后续性能分析提供统一的量化基础。

## 6.3 实验结果与分析

### 6.3.1 运行稳定性分析

#### (1) 实验覆盖规模与任务构成统计

为评估系统在大规模连续运行条件下的稳定性，本文对全部 sweep case 的覆盖规模及任务构成进行统计分析。表6-3 汇总了实验涉及的基础 case 数、skew 配置数、调度策略数，以及各类算子的累计任务数量。

表 6-3 实验覆盖规模与任务构成统计

| 统计项            | 数值     | 统计项            | 数值    |
|----------------|--------|----------------|-------|
| 基础 case 数      | 144    | MAT_COL_MUL 任务 | 37161 |
| skew 配置数       | 4      | ROW_MAT_MUL 任务 | 38969 |
| 调度策略数          | 4      | ROW_COL_MUL 任务 | 36609 |
| 成功完成实例数        | 2304   | SCALAR_VEC 任务  | 85484 |
| 总任务数           | 718000 | SCALAR_MAT 任务  | 33652 |
| MAT_MUL 任务     | 206887 | MAT_SUM 任务     | 36744 |
| MAT_ADD 任务     | 34227  | VEC_SUM 任务     | 88128 |
| COL_ROW_MUL 任务 | 35900  | VEC_ADD 任务     | 84239 |

由表6-3 可知，在 144 个基础 case、4 种 skew 配置及 4 类调度策略的组合下，系统共完成 2304 个策略实例，累计任务规模达到 718000。上述任务量与策略组合为验证系统在长时间连续运行条件下的稳定性提供了充分的实验基础。各类任务类型覆盖矩阵乘、矩阵加、向量归约、向量加以及多种行列变换相关算子，表明所采用的工作负载在规模与结构上具有较高的复杂性与代表性。

#### (2) 运行时行为与日志验证

为验证系统在运行过程中的稳定性，并分析其执行行为特征，本文对调度执行过程中产生的运行时日志进行分析。

图 6-4 展示了系统执行过程中的典型日志片段。从中可以观察到，系统持续维护非空的就绪队列 (READY QUEUE)，且不存在未满足依赖的任务 (PENDING TASKS 为空)，表明任务依赖关系能够被正确解析并持续推进。同时，日志中数据传输操作（如 xdma\_rw.exe 执行）与指令发送过程交替出现，说明系统在运行过程中能够协同推进数据通信与任务调度。

在该执行过程中，未出现任务停滞、重复调度或异常终止等情况，表明调度

器在持续运行条件下能够保持稳定的任务流动与执行推进。

```

[✓] SEND_DATA success
[✎] Executing: xdma_rw.exe h2c_0 write 0xc2000000 -f C:\Users\Administrator\Desktop\new_simple2.
1\send_files\send_data_linear_20260319_103640_883540.bin
16448 bytes written in 0.000132s
[✓] Linear DMA success
1 bytes written in 0.000004s
[✎] Executing: xdma_rw.exe h2c_0 write 0xc0000000 -f C:\Users\Administrator\Desktop\new_simple2.
1\send_files\init_data_inst_20260319_103640_891357.bin
40 bytes written in 0.000053s
1 bytes written in 0.000003s
[✓] INIT_DATA success
##### Batch Ingest: Vector Add (Task 1) #####
[✎] [READY QUEUE]: 5, 6, 7, 8, 9, 10, 11, 12
[✎] [PENDING TASKS]: None (All remaining tasks are READY)
#####
[✎] Submitted compute_id=1. Waiting... (timeout=30.0s)
[✎] Executing: xdma_rw.exe h2c_0 write 0xc0000000 -f C:\Users\Administrator\Desktop\new_simple2.
1\send_files\send_inst_inst_20260319_103640_899297.bin
24 bytes written in 0.000056s
1 bytes written in 0.000003s
[✓] SEND_INST success
*****[DEBUG] Parsed 1 statuses from 1:
task_ID: 5, C_ID: 17, C_SIZE: 66

##### Runtime: Processed 1 Completed Tasks #####
[✎] [READY QUEUE]: 6, 7, 8, 9, 10, 11, 12
[✎] [PENDING TASKS]: None (All remaining tasks are READY)
#####

```

图 6-4 运行时调度队列与通信行为日志

进一步地，为验证系统在大规模批量任务条件下的持续运行能力，图 6-5 给出了多 case 连续执行过程中的日志片段。

```

[SWEEP] DONE case_id=1
-----
[SWEEP] RUN case_id=2 skew_id=0 skew=0.0
-----
Starting : Multi Mode
-----
Result: Smart Finished.
Tasks: 500
Makespan: 6687.00 us
Throughput: 0.074772 task/us
TotalInputMoveKB: 1141, AvgInputMoveKB: 2.282 KB/task
-----
Starting : LB Mode
-----
Result: LB Finished.
Tasks: 500
Makespan: 6778.00 us
Throughput: 0.073768 task/us
TotalInputMoveKB: 40320, AvgInputMoveKB: 80.640 KB/task
=====
ALL TESTS COMPLETED
=====
[SWEEP] DONE case_id=2

```

(a) 批量任务执行过程

```

-----
Starting : Multi Mode
-----
Result: Smart Finished.
Tasks: 2000
Makespan: 137794.00 us
Throughput: 0.014514 task/us
TotalInputMoveKB: 6682, AvgInputMoveKB: 3.341 KB/task
-----
Starting : LB Mode
-----
Result: LB Finished.
Tasks: 2000
Makespan: 137468.00 us
Throughput: 0.014549 task/us
TotalInputMoveKB: 320973, AvgInputMoveKB: 160.487 KB/task
=====
ALL TESTS COMPLETED
=====
[SWEEP] DONE case_id=143
=====
[SWEEP] ALL CASES DONE.
=====
# close_project
# exit 0
INFO: [Common 17-206] Exiting Vivado at Tue Mar 17 02:43:54 2026...

```

(b) 全部任务完成状态

图 6-5 批量任务执行与整体完成状态日志

如图6-5(a)所示，系统能够按照 sweep 配置依次执行不同 case 及调度策略，在执行过程中各 case 顺序推进且未出现中断；图6-5(b)则表明，在全部任务完成后系统能够输出“ALL TESTS COMPLETED”与“ALL CASES DONE”等信息。

从运行日志来看,上述现象从执行过程与整体结果两个方面表明系统能够稳定推进任务执行。综合而言,系统不仅能够在单次执行过程中保持稳定运行,还能够多 case 批量调度条件下持续推进并完成全部任务,反映出系统在连续执行过程中的稳定运行能力。

### (3) 稳定性结果分析

在上述实验覆盖规模条件下,系统对全部 case 进行了连续运行,并对各策略实例的执行结果进行统计分析。统计结果表明,在全部 2304 个策略实例中,未观察到死锁、卡死、执行中断或由系统异常导致的结果缺失现象。

从整体实验结果来看,结合任务覆盖规模、算子累计数量以及实例完成情况可以看出,本文原型系统在长时间、多任务持续注入的条件下能够保持稳定运行,并维持一致的执行语义,说明所实现的任务调度与数据通路协同机制具备良好的持续运行能力。

## 6.3.2 解耦机制性能分析

本节分析解耦执行机制对任务完成时间的影响。首先给出不同任务规模与调度策略下的统计结果,如表6-4 所示;随后结合图示结果对性能变化趋势及其形成原因进行分析。

### (1) 整体性能统计结果

表6-4 汇总了不同任务规模与策略组合下的耦合完成时间、解耦完成时间及通信相关指标的平均结果。

表 6-4 不同任务规模与策略下的平均收益统计

| Tasks | Strategy | Case | $T_{\text{coupled}}$ | $T_{\text{decoupled}}$ | $T_{\text{comp}}$ | $T_{\text{comm}}^{\text{coupled}}$ | $T_{\text{comm}}^{\text{decoupled}}$ | $T_{\text{hidden}}$ |
|-------|----------|------|----------------------|------------------------|-------------------|------------------------------------|--------------------------------------|---------------------|
| 500   | Multi    | 288  | 25701.38             | 20540.53               | 20464.06          | 5237.33                            | 76.48                                | 5160.85             |
| 500   | LB       | 288  | 178299.19            | 26527.38               | 20309.24          | 157989.95                          | 6218.14                              | 151771.81           |
| 2000  | Multi    | 287  | 95741.11             | 77642.89               | 77555.24          | 18185.86                           | 87.65                                | 18098.21            |
| 2000  | LB       | 287  | 604654.56            | 83814.43               | 77441.65          | 527212.91                          | 6372.78                              | 520840.14           |

由表6-4 可以看出,在不同任务规模与调度策略条件下,解耦机制均能够降低任务完成时间。对于 Multi 策略,Tasks=500 与 Tasks=2000 条件下的完成时间均呈现稳定下降趋势,对应加速比约为 1.30;对于 LB 策略,完成时间下降幅度更为显著,对应加速比分别达到 7.17 与 8.59。

上述结果表明,解耦执行机制在不同配置条件下均能够带来一致的性能改善,但其改善幅度依赖于具体的任务结构与调度策略。

### (2) 性能差异的构成分析

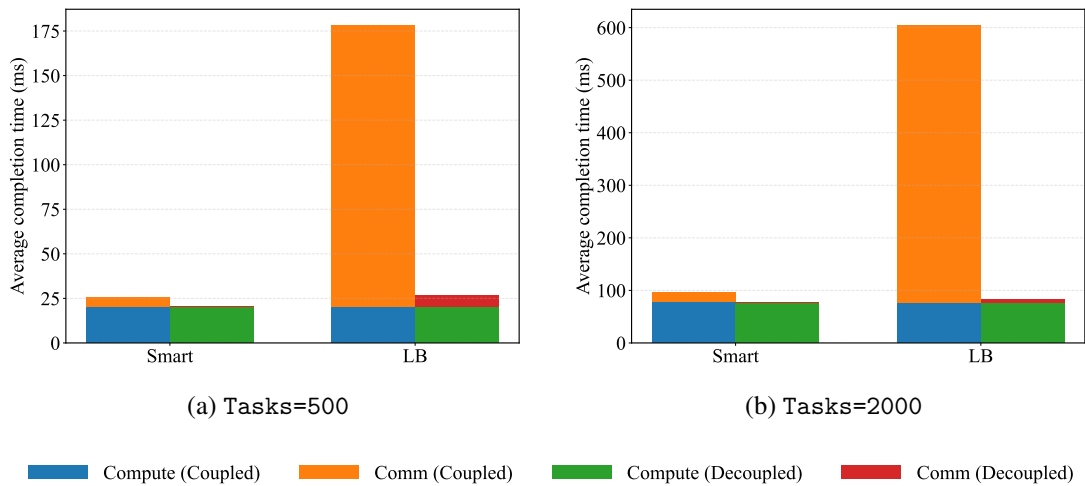


图 6-6 不同策略的完成时间组成比例

图6-6 给出了不同策略下完成时间的组成比例。从图6-6(a) 与图6-6(b) 可以看出，不同策略下通信在完成时间中的占比存在显著差异。对于 **Multi** 策略，通信占比较低，系统整体完成时间主要由计算路径决定；而在 **LB** 策略下，通信在完成时间中占据主导地位。

上述差异决定了解耦机制对整体性能的影响程度。当通信占比较低时，完成时间主要受计算路径限制，解耦带来的整体改善有限；而当通信占据主要时间开销时，解耦机制通过减少关键路径上的通信等待，能够显著缩短任务完成时间。

### (3) 性能分布特征

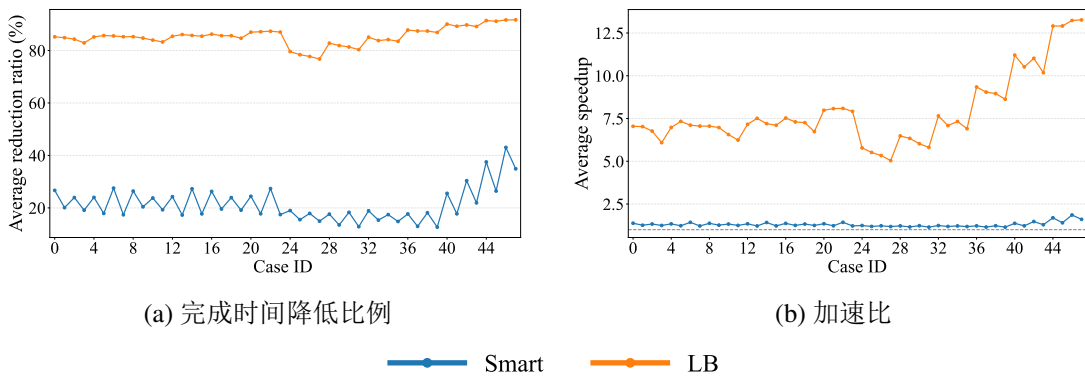


图 6-7 性能变化分布

图6-7 展示了不同 case 下完成时间降低比例与加速比的分布。结果表明，解耦机制在多数 case 中均带来性能提升，但不同 case 之间仍存在一定波动，说明任务构成与数据分布会影响其实际效果。总体来看，**LB** 策略提升更显著，而 **Multi** 策略提升较为稳定但幅度较小。

### 6.3.3 实验总结

本章从系统稳定性与解耦执行机制对任务完成时间的影响两个方面，对原型系统进行了系统性实验评估。实验结果表明，在 144 个基础 case、4 种 skew 配置与 4 类调度策略组合下，系统共完成 2304 个策略实例，累计处理任务 718000 个。在连续运行过程中，未观察到死锁、卡死或执行中断等异常，验证了系统在持续负载下的稳定运行能力。

进一步分析表明，解耦机制在不同任务规模与调度策略条件下均能够降低任务完成时间，其性能差异主要取决于通信在总完成时间中的占比：当通信占比较低时，性能受计算路径限制，提升幅度有限；当通信成为主导开销时，解耦机制通过减少关键路径上的通信等待，能够显著提升系统性能。

综合来看，所提出的数据-指令双通路解耦机制不仅能够支持系统稳定运行，而且能够有效降低关键路径上的通信开销，从而缩短任务完成时间，体现出良好的系统级性能提升效果与应用价值。

## 第七章 总结与展望

### 7.1 研究成果总结

本文针对多节点张量计算中通信开销大、调度灵活性不足等问题，基于数据与指令解耦思想，设计并实现了面向小规模多 FPGA 平台的张量计算调度原型系统。从系统架构、数据管理与调度策略三个层面验证了该方法的工程可行性。

在体系结构层面，本文构建了“主机-调度器-加速器”三层协同架构，通过显式区分数据与指令通路，实现了数据传输与指令调度的解耦执行，为通信与计算重叠奠定了结构基础。在运行时机制上，采用 DAG 建模张量任务并由就绪驱动触发执行，降低了调度控制复杂度并提升了系统并行性。

在数据管理方面，设计了基于元数据表的张量块管理机制，统一维护数据标识、驻留位置及一致性状态，并结合单写者多读者 (SWMR) 模型保障跨节点访问的正确性。同时，通过数据复用与驻留管理减少不必要的数据迁移。在调度策略方面，提出并实现了融合数据本地性、节点负载及内存占用的多因素加权调度方法，实现了任务在运行时的动态调整与分配。

在自研 FPGA 计算阵列上的验证结果表明，该架构最高可降低约 97% 的跨节点数据传输开销，并通过通信与计算重叠有效隐藏通信延迟。在通信主导的场景下，系统整体性能提升约 7-8 倍。同时，系统在连续运行过程中表现出良好的稳定性，验证了该设计在资源受限多节点环境中的工程可行性。

### 7.2 系统局限性与不足

尽管本文完成了数据-指令解耦调度系统的设计与实现，但仍存在一定的局限性，主要体现在以下几个方面：

首先，实验平台规模有限。本文的验证工作主要基于小规模多 FPGA 节点环境，节点数量与网络拓扑相对简单，尚未覆盖大规模集群或复杂互联结构下的调度行为，其性能收益在更大规模系统中的表现仍有待进一步验证。

其次，调度策略仍以启发式方法为主。多因素加权调度模型在工程实现上具有较好的可控性，但尚未引入更精细的性能建模或自适应优化机制，在复杂或极端负载场景下仍存在进一步优化空间。

此外，当前系统主要面向规则性较强的张量计算任务，对于复杂控制流、不规则访存模式及动态算子场景的支持仍较为有限；同时，对 CPU、GPU 与 FPGA

等异构计算资源之间差异性的考虑仍不充分，有待进一步拓展。

最后，系统原型构建基于理想化假设，缺乏异常容错机制。设计假定计算节点完全同构、模型静态可解析，未考虑节点失效、通信超时或任务重试等物理异常。若突发硬件故障，由于缺乏心跳探针、动态拓扑重构及检查点机制，会导致流水线挂起或调度终止，限制了工业级环境下的鲁棒性。

### 7.3 未来工作展望

针对上述不足，未来的研究工作可从以下几个方向展开：

(1) 扩展系统规模与实验场景。

在更大规模的 FPGA 或异构节点平台上验证数据-指令解耦调度机制的可扩展性，分析不同互联拓扑结构与通信带宽条件下的系统性能表现，并评估其在复杂负载场景下的稳定性与适应能力。

(2) 引入更智能的调度决策机制。

结合在线性能反馈、历史执行信息或学习型方法，对调度策略中的权重参数进行自适应调整，构建更精细的调度代价模型，从而进一步提升系统在复杂动态环境下的调度效率与资源利用率。

(3) 拓展对异构与复杂算子的支持能力。

研究在 CPU、GPU 与 FPGA 等异构计算资源协同环境中引入统一的解耦调度机制，并探索对不规则张量计算、动态图任务及复杂数据依赖场景的支持方式，提升系统的通用性与适应性。

(4) 加强与编译器和上层框架的协同。

将数据-指令解耦思想与编译器优化、算子生成及深度学习框架相结合，构建从编译期到运行时的协同优化路径，实现任务划分、数据布局与调度执行的统一设计，进一步提升系统整体执行效率与资源利用率。

(5) 引入轻量级容错与异常处理机制。

针对分布式环境下的不确定性，在维持数据与指令解耦高效性的前提下，研究并引入轻量级的节点心跳检测与网络超时重传协议；同时探索任务失败时的状态回滚与动态节点剔除接管机制，从而提升调度系统在物理节点异常情况下的稳健性与可用性。

通过上述方向的进一步研究，有望推动数据与指令解耦机制在更大规模、更复杂计算系统中的应用，为高效张量计算调度与数据管理提供更加完善且可扩展的系统支撑。

## 参考文献

- [1] Devlin J, Chang M W, Lee K, et al. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding[C]//Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. 2019: 4171-4186.
- [2] Brown T, Mann B, Ryder N, et al. Language Models Are Few-Shot Learners[J]. Advances in Neural Information Processing Systems, 2020, 33: 1877-1901.
- [3] Achiam J, Adler S, Agarwal S, et al. GPT-4 Technical Report[EB/OL]. 2023. <https://arxiv.org/abs/2303.08774>.
- [4] Chowdhery A, Narang S, Devlin J, et al. PaLM: Scaling Language Modeling with Pathways [J]. Journal of Machine Learning Research, 2023, 24(240): 1-113.
- [5] Zeng W, Ren X, Su T, et al. PanGu- $\alpha$ : Large-Scale Autoregressive Pretrained Chinese Language Models with Auto-Parallel Computation[EB/OL]. 2021. <https://arxiv.org/abs/2104.12369>.
- [6] Sun Y, Wang S, Feng S, et al. ERNIE 3.0: Large-Scale Knowledge Enhanced Pre-Training for Language Understanding and Generation[EB/OL]. 2021. <https://arxiv.org/abs/2107.02137>.
- [7] Du Z, Qian Y, Liu X, et al. GLM: General Language Model Pretraining with Autoregressive Blank Infilling[C]//Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2022: 320-335.
- [8] Guo D, Yang D, Zhang H, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning[EB/OL]. 2025. <https://arxiv.org/abs/2501.12948>.
- [9] 赵桦箏, 庞善民, 赵英海, 等. 面向多功能张量加速器的细粒度结构化稀疏设计[J]. 西安交通大学学报, 2024, 58(11): 176-184.
- [10] 张振兴, 文渊博, 吕涵祺, 等. 面向大语言模型训练的智能计算系统综述[J]. 计算机科学技术学报, 2025, 40(1): 6-41.
- [11] Ragan-Kelley J, Barnes C, Adams A, et al. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines[J]. ACM SIGPLAN Notices, 2013, 48(6): 519-530.
- [12] Abadi M, Barham P, Chen J, et al. TensorFlow: A System for Large-Scale Machine Learning [C]//12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16). 2016: 265-283.
- [13] Paszke A, Gross S, Massa F, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library[J]. Advances in Neural Information Processing Systems, 2019, 32.

- [14] Shoeybi M, Patwary M, Puri R, et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism[EB/OL]. 2019. <https://arxiv.org/abs/1909.08053>.
- [15] Rasley J, Rajbhandari S, Ruwase O, et al. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters[C]//Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2020: 3505-3506.
- [16] Moritz P, Nishihara R, Wang S, et al. Ray: A Distributed Framework for Emerging AI Applications[C]//13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). 2018: 561-577.
- [17] Smith J E. Decoupled Access/Execute Computer Architectures[J]. ACM SIGARCH Computer Architecture News, 1982, 10(3): 112-119.
- [18] Choquette J. NVIDIA Hopper H100 GPU: Scaling Performance[J]. IEEE Micro, 2023, 43(3): 9-17.
- [19] Jouppi N, Kurian G, Li S, et al. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings[C]//Proceedings of the 50th Annual International Symposium on Computer Architecture. 2023: 1-14.
- [20] 焦李成, 孙其功, 杨育婷, 等. 深度神经网络 FPGA 设计进展、实现与展望[J]. 计算机学报, 2022, 45(3): 441-468.
- [21] 孟庆昊, 边丽衡. 基于 FPGA 的神经网络硬件加速器研究综述[J]. 信号处理, 2025, 41(12): 1855-1873.
- [22] Bauer M, Treichler S, Slaughter E, et al. Legion: Expressing Locality and Independence with Logical Regions[C]//SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis. 2012: 1-11.
- [23] Augonnet C, Thibault S, Namyst R, et al. StarPU: A Unified Platform for Task Scheduling on Heterogeneous Multicore Architectures[C]//European Conference on Parallel Processing. 2009: 863-874.
- [24] Bosilca G, Bouteiller A, Danalis A, et al. PaRSEC: Exploiting Heterogeneity to Enhance Scalability[J]. Computing in Science & Engineering, 2013, 15(6): 36-45.
- [25] Fang T, Perez-Vicente A, Johnson H, et al. Deep Learning Scheduling on a Field-Programmable Gate Array Cluster Using Configurable Deep Learning Accelerators[J]. Information, 2025, 16(4): 298.
- [26] Shazeer N, Cheng Y, Parmar N, et al. Mesh-TensorFlow: Deep Learning for Supercomputers [J]. Advances in Neural Information Processing Systems, 2018, 31.
- [27] TensorFlow Team. XLA: Optimizing Compiler for Machine Learning[EB/OL]. 2017. <https://www.tensorflow.org/xla>.
- [28] Chen T, Moreau T, Jiang Z, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning[C]//13th USENIX Symposium on Operating Systems Design and Implemen-

- tation (OSDI 18). 2018: 578-594.
- [29] Ding Y, Yu C H, Zheng B, et al. Hidet: Task-Mapping Programming Paradigm for Deep Learning Tensor Programs[C]//Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. 2023: 370-384.
- [30] Zhao J, Li B, Nie W, et al. AKG: Automatic Kernel Generation for Neural Processing Units Using Polyhedral Transformations[C]//Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. 2021: 1233-1248.
- [31] Moreau T, Chen T, Vega L, et al. A Hardware--Software Blueprint for Flexible Deep Learning Specialization[J]. IEEE Micro, 2019, 39(5): 8-16.
- [32] Sharma H, Park J, Amaro E, et al. DNNWeaver: From High-Level Deep Network Models to FPGA Acceleration[C]//Workshop on Cognitive Architectures. 2016: 1-12.
- [33] Umuroglu Y, Fraser N J, Gambardella G, et al. FINN: A Framework for Fast, Scalable Binarized Neural Network Inference[C]//Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 2017: 65-74.
- [34] 裴建华, 唐俊, 周扬. YOLO 后处理算法在 FPGA 上的硬件加速实现[J]. 集成电路与嵌入式系统, 2025, 25(12): 76-82.
- [35] Kjolstad F, Kamil S, Chou S, et al. The Tensor Algebra Compiler[J]. Proceedings of the ACM on Programming Languages, 2017, 1(OOPSLA): 1-29.
- [36] Feng S, Hou B, Jin H, et al. TensorIR: An Abstraction for Automatic Tensorized Program Optimization[C]//Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. 2023: 804-817.
- [37] Huang Q, Kang M, Dinh G, et al. CoSA: Scheduling by Constrained Optimization for Spatial Accelerators[C]//2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA). 2021: 554-566.
- [38] Ham T J, Aragón J L, Martonosi M. DeSC: Decoupled Supply-Compute Communication Management for Heterogeneous Architectures[C]//Proceedings of the 48th International Symposium on Microarchitecture. 2015: 191-203.
- [39] Dennis J B. Data Flow Supercomputers[J]. Computer, 1980, 13(11): 48-56.
- [40] NVIDIA Corporation. NVIDIA Hopper Architecture Whitepaper[EB/OL]. 2022. <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c>.
- [41] OpenXLA Project. OpenXLA: XLA Compiler Documentation[EB/OL]. 2023. [https://openxla.org/xla/hlo\\_to\\_thunks](https://openxla.org/xla/hlo_to_thunks).
- [42] Zheng L, Jia C, Sun M, et al. Anso: Generating High-Performance Tensor Programs for Deep Learning[C]//14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). 2020: 863-879.
- [43] Li Z, Gao M. KAPLA: Scalable NN Accelerator Dataflow Design Space Structuring and Fast

- Exploring[C]//Proceedings of the 30th Asia and South Pacific Design Automation Conference. 2025: 8-15.
- [44] Dean J, Ghemawat S. MapReduce: Simplified Data Processing on Large Clusters[J]. Communications of the ACM, 2008, 51(1): 107-113.
- [45] Zaharia M, Chowdhury M, Franklin M J, et al. Spark: Cluster Computing with Working Sets [C]//2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 10). 2010: 10-17.
- [46] Sergeev A, Del Balso M. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow [EB/OL]. 2018. <https://arxiv.org/abs/1802.05799>.
- [47] Rajbhandari S, Rasley J, Ruwase O, et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models[C]//SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. 2020: 1-16.
- [48] Narayanan D, Harlap A, Phanishayee A, et al. PipeDream: Generalized Pipeline Parallelism for DNN Training[C]//Proceedings of the 27th ACM Symposium on Operating Systems Principles. 2019: 1-15.
- [49] Johnson H, Fang T, Perez-Vicente A, et al. Reconfigurable Distributed FPGA Cluster Design for Deep Learning Accelerators[C]//2023 IEEE International Conference on Electro Information Technology (eIT). 2023: 1-5.
- [50] Wang T, Geng T, Li A, et al. FPDeep: Scalable Acceleration of CNN Training on Deeply-Pipelined FPGA Clusters[J]. IEEE Transactions on Computers, 2020, 69(8): 1143-1158.
- [51] Zeng H, Prasanna V. GraphACT: Accelerating GCN Training on CPU-FPGA Heterogeneous Platforms[C]//Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 2020: 255-265.
- [52] 刘林东, 邬依林. 基于 HEFT 和 CPOP 的相关任务表调度算法[J]. 计算机系统应用, 2019, 28(03): 118-125.
- [53] Wu Q, Wu Z, Zhuang Y, et al. Adaptive DAG Tasks Scheduling with Deep Reinforcement Learning[C]//International Conference on Algorithms and Architectures for Parallel Processing. 2018: 477-490.
- [54] 宋俊苏. 分布式大数据的复杂任务调度与存储优化策略研究[J]. 数据科学与技术, 2025 (4): 138-146.
- [55] 赵志岩. 分布式异构算力环境下并发 DAG 任务调度算法研究[D]. 电子科技大学, 2025.
- [56] 左俊杰, 肖锋, 黄姝娟. 一种新的异构多核平台下多类型 DAG 调度方法[J]. 计算机应用研究, 2025, 42(2): 514-518.
- [57] 刘斌, 赵亦玄, 王辉, 等. 一种基于风险感知策略的多节点任务调度方法[J]. 电子与信息学报, 2022, 44(9): 3231-3240.
- [58] Jia Z, Padon O, Thomas J, et al. TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions[C]//Proceedings of the 27th ACM Symposium on Operating Systems Principles. 2019: 47-62.

- [59] 冉仕举. 张量网络[M]. 首都师范大学出版社, 2022.
- [60] 陈云霄, 李玲. 智能计算系统[M]. 机械工业出版社, 2024.
- [61] 陈国良. 并行算法的设计与分析[M]. 高等教育出版社, 2009.
- [62] Ye Z, Gao W, Hu Q, et al. Deep Learning Workload Scheduling in GPU Datacenters: A Survey [J]. ACM Computing Surveys, 2024, 56(6): 1-38.
- [63] Venieris S I, Bouganis C S. fpgaConvNet: A Framework for Mapping Convolutional Neural Networks on FPGAs[C]//2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). 2016: 40-47.
- [64] 李亿渊, 薛巍, 陈德训, 等. 稀疏矩阵向量乘法在申威众核架构上的性能优化[J]. 计算机学报, 2020, 43(06): 1010-1024.
- [65] AlEbrahim S, Ahmad I. Task Scheduling for Heterogeneous Computing Systems[J]. The Journal of Supercomputing, 2017, 73(6): 2313-2338.
- [66] Topcuoglu H, Hariri S, Wu M Y. Task Scheduling Algorithms for Heterogeneous Processors [C]//Proceedings of the Eighth Heterogeneous Computing Workshop (HCW'99). 1999: 3-14.
- [67] Culler D, Singh J P, Gupta A. Parallel Computer Architecture: A Hardware/Software Approach [M]. Gulf Professional Publishing, 1999.
- [68] Barney B. Introduction to Parallel Computing[J]. Lawrence Livermore National Laboratory, 2010, 6(13): 10.
- [69] Van Steen M, Tanenbaum A S. Distributed Systems[M]. Maarten van Steen Leiden, The Netherlands, 2017.

## 在学期间的研究成果

### 一、专利成果

1. 基于三层异构架构的张量处理器系统及其数据处理方法 [P]. 第一作者, 中国发明专利, 专利号: ZL202610003124.8, 申请日: 2026 年 01 月 05 日, 已授权。  
对应本文第三章张量计算调度系统总体架构

### 二、参与课题

1. 基于 RISC-V 指令集的全异步低功耗 SoC 研发与应用示范 (甘肃省科技重大专项计划, 编号:22ZD6GE016,2022.11-2025.11)
2. 异步集成电路芯片研究团队 (兰州大学中央高校优秀青年团队项目, 编号: lzuibky-2024-eyt02, 2024.01-2025.12)
3. 基于 Link-Joint 技术的全异步神经网络芯片设计与 FPGA 验证研究 (中央引导地方科技发展资金项目, 编号:25ZYJA016,2025.01-2027.012)

## 致 谢

三年研究生时光悄然落幕，我的硕士求学之路也即将圆满收官。回望这段充实且珍贵的历程，从初入校园的懵懂迷茫，到一步步夯实专业功底、完成科研实践，最终顺利毕业，离不开身边亲人、师长与同窗一路的陪伴、指引与帮扶，这份温暖与支持，我始终铭记于心，在此致以最真挚的谢意。

首先感恩我的父母，你们是我求学路上最坚实的依靠。三年来，你们用无私的付出与默默的牵挂，包容我学业的忙碌与偶尔的疲惫，你们从不苛求成绩，只愿我平安顺遂，这份深沉的养育与关爱，是我克服困难、坚持到底的底气，此生难忘，永怀感恩。

感谢我的女友，感谢你全程陪伴我走过这段研究生岁月。工科的学习与科研本就枯燥繁重，无数个泡在实验室、伏案钻研的日子，是你给予我温柔的理解、耐心的包容与坚定的鼓励，抚平我的焦虑，支撑我稳步前行，这份相守相伴，是我求学路上最珍贵的温暖。

衷心感谢我的导师王兆滨教授。作为我的导师，您始终牵挂我的学业成长，入学第一年便带我迈入科研的大门，帮我打下扎实的专业基础，您严谨的治学态度、务实的科研作风，深深影响着我，为我整个研究生阶段的学习树立了标杆，全程给予我的指导与关怀，让我受益终身，在此深表谢意。

感谢王老师门下的各位同门们，在一同学习交流的时光里，我们互帮互助、切磋心得，携手攻克学习与科研中的小难题，彼此鼓励、共同进步，这份纯粹的同门情谊，为我的研究生生活增添了诸多暖意，格外珍贵。

感谢导师何安平教授，在我研究生后两年的学习与科研实践中，您给予了我细致的指导与帮助，陪着我一步步推进科研内容、梳理学业成果，助力我稳步完成各项学业任务，顺利迈向毕业，感谢您的悉心点拨与无私支持。

感谢何老师门下的各位同窗伙伴，两年多的朝夕相处、并肩奋战，我们在实验室里共同钻研，在生活中彼此照应，一起度过了忙碌又充实的时光，也收获了真挚的友谊，感恩相遇。

最后，再次向所有关心、帮助过我的人致以最诚挚的谢意。三年研究生生涯圆满落幕，这段经历将成为我人生的宝贵财富，未来我定会带着这份感恩与期许，脚踏实地，步履不停，不负所有厚爱与期盼。